

A Fiat–Shamir Transformation From Duplex Sponges

Alessandro Chiesa	Michele Orrù
<code>alessandro.chiesa@epfl.ch</code>	<code>m@orru.net</code>
EPFL	CNRS

March 27, 2026

Abstract

We analyze a variant of the Fiat–Shamir transformation based on an ideal permutation. The transformation relies on the popular duplex sponge paradigm, and minimizes the number of calls to the permutation (given the information to absorb/squeeze). This closely models deployed variants of the Fiat–Shamir transformation, and our analysis provides concrete security bounds to guide security parameters in practice. Our results contribute to the ongoing community-wide effort to achieve rigorous, and ultimately formally verified, security proofs for deployed cryptographic proofs. Along the way, we find that indistinguishability (a property proven for many modes of operation, including the duplex sponge) is insufficient for establishing the knowledge soundness and zero knowledge of a non-interactive argument.

We additionally contribute `spongef`, an open-source Rust library implementing our Fiat–Shamir transformation. The library is interoperable across multiple cryptographic frameworks, and works with any choice of permutation. The library comes equipped with Keccak and Poseidon permutations, as well as several “codecs” for re-mapping prover and verifier messages to the permutation’s domain.

Keywords: Fiat–Shamir transformation; duplex sponge

Contents

1	Introduction	3
1.1	Our results	4
1.2	Application: a BCS transformation from duplex sponges	6
1.3	Open problems	7
1.4	Related work	7
2	Techniques	8
2.1	Review: starting point and the basic variant	8
2.2	DSFS: a Fiat–Shamir transformation based on the ideal duplex sponge	9
2.3	Limitations of indifferntiability	11
2.4	Soundness and knowledge soundness	12
2.5	Zero knowledge	14
2.6	Application: BCS transformation from a duplex sponge	16
3	Preliminaries	17
3.1	Notation	17
3.2	Basics	17
3.3	Duplex sponge in overwrite mode	18
3.4	Non-interactive arguments in oracle models	20
3.5	Interactive proofs	21
3.6	Fiat–Shamir transformation	23
4	Duplex-sponge Fiat–Shamir transformation	27
5	Security analysis	29
5.1	Data structure for query-answer trace	30
5.2	Backtracking procedure	31
5.3	Lookahead procedure	33
5.4	Prover transformation	34
5.5	Trace transformation	36
5.6	Bad events	38
5.7	Analysis of aborts	44
5.8	Proof of Theorem 5.1	46
6	Soundness and knowledge soundness	55
6.1	Proof of Theorem 6.1	55
6.2	Proof of Theorem 6.2	56
7	Zero knowledge	59
7.1	Definitions for zero knowledge	59
7.2	The simulator	60
7.3	Proof of Theorem 7.1	60
8	Implementation	68
8.1	Core library and software stack	68
8.2	Concrete security and ergonomics	69
A	On indifferntiability	71
A.1	Oracle constructions and indifferntiability	71
A.2	Limitations of indifferntiability	72
B	Example: codecs for Schnorr’s protocol	77
C	Bias of modular reduction	78
	Acknowledgments	79
	References	79

1 Introduction

The Fiat–Shamir transformation [FS86] is a technique that uses a hash function to convert a public-coin interactive protocol between a prover and a verifier into a corresponding non-interactive protocol. The beautiful idea underlying the transformation is that one can replace the verifier’s random messages with suitable outputs of the hash function (if sufficiently “secure”), eliminating the need for interaction. This technique, in various incarnations, has numerous and diverse applications across cryptography.

The Fiat–Shamir transformation is often studied in the *random oracle model* (ROM), where the hash function is modeled as a random function (every input is mapped to a random output). This setting is an idealization, as any hash function that one would use in the real world would not be a random function. Nevertheless, this idealization provides an elegant model where security can be precisely quantified and then, in practice, one replaces the random oracle with a “random-looking” hash function such as SHA256, making the heuristic assumption that this replacement does not compromise security. This is an example of the *random oracle methodology* that, despite suffering from notable limitations,¹ has had an immense impact on the construction and analysis of highly-efficient cryptographic primitives for practical use.

In this paper we *analyze a variant of the Fiat–Shamir transformation, aimed at closely modeling deployed variants of the Fiat–Shamir transformation*. We motivate this in the next few paragraphs, highlighting why prior security analyses do not adequately capture features of practical interest.

Different shades of Fiat–Shamir. “Fiat–Shamir transformation” is an umbrella term that refers to a class of *related but distinct* transformations that apply to different classes of interactive protocols. Differences arise due to round complexity (one verifier message or multiple verifier messages), salts for zero knowledge (how many and where they are introduced), and oracle setting (which random oracles are available to parties). We review the main variants in Section 2.

A security gap. The motivation for this paper is a gap between variants analyzed by theoreticians and variants implemented (and deployed) by practitioners. On the one hand, theoreticians assume access to a random oracle (or multiple random oracles) *with sufficiently large input and output size*. For example, one assumes that a prover message (or even all prover messages up to a certain round) can be an input to the random oracle. On the other hand, cryptographic hash functions in practice *support input/output blocks of fixed length*, so practitioners consider (more complex) variants that support sufficiently large inputs/outputs via multiple calls to the fixed-size hash function. These variants follow “modes of operation” that in some cases are known to be *indifferentiable* [MRH04] from the random oracle they emulate. However, indifferentiability is insufficient to establish knowledge soundness and zero knowledge (we discuss in detail why). This security gap should be closed (or at least reduced) in light of the widespread adoption in practice of the Fiat–Shamir transformation (e.g., in zkSNARK constructions), as well as the ongoing community-wide effort to work towards formally verified security analyses of cryptographic proofs [Eth24].

Permutation-based cryptography. Permutation-based cryptography has developed into a viable alternative to traditional block-cipher-based cryptography, leading to elegant constructions used in practice. These include duplex sponges for lightweight authenticated encryption [BDPVA12], eXtensible Output Functions (XOFs) [MF21], deck functions such as Farfalle for high-speed authenticated encryption [BDHPVAVK18], compression modes such as Jive [Bou+23], and more. Most notably, the SHA-3 standard [Sha] based on Keccak is an example of a hash function designed according to this methodology: it is a sponge construction that enables “absorbing” arbitrary-length inputs and “squeeze” arbitrary-length outputs.

¹For example, there are interactive protocols for which the Fiat–Shamir transformation yields a secure protocol if the hash function is a random oracle but not if the hash function is any efficient function [Bar01; GK03; CGH04; BBHMR19; KRS25].

1.1 Our results

We analyze the security of a Fiat–Shamir transformation **based on a duplex sponge**, aimed at closely modeling deployed variants of the Fiat–Shamir transformation. We accompany our analysis with a flexible Rust library that enables practitioners to conveniently and efficiently use our variant. We elaborate on our results below, and then in Section 2 we summarize the ideas for proving our results.

The ideal permutation model. We consider a setting where parties have access to a random permutation $p: \Sigma^{r+c} \rightarrow \Sigma^{r+c}$ where Σ is a finite alphabet, c is the *capacity*, r is the *rate*, and $r + c$ is the *permutation length*.² These parameters are fixed (independent of any specific interactive protocol). In practice, p would be heuristically instantiated by a binary permutation (e.g., Keccak [Sha]) or an “algebraic” permutation (e.g., MiMC [AGRRT16], Poseidon [GKRRS21], Anemoi [Bou+23]). This is the aforementioned setting of permutation-based cryptography. In particular, p directly gives rise to a corresponding duplex sponge construction [BDPVA12], a mode of operation that enables absorbing and squeezing strings over Σ .

The transformation. We transform a given public-coin interactive proof $\text{IP} = (\mathbf{P}, \mathbf{V})$ for a relation $\mathcal{R}$ into a corresponding non-interactive argument $\text{NARG} = (\mathcal{P}, \mathcal{V})$ for $\mathcal{R}$, in the ideal permutation model described above. The transformation additionally depends on a *codec* cdc that bridges the (typically different) representation of IP ’s messages and p ’s alphabet Σ : cdc specifies maps $(\varphi_i)_{i \in [k]}$ to encode prover messages into strings over Σ and maps $(\psi_i)_{i \in [k]}$ to decode strings over Σ into verifier messages. The transformation hardcodes a salt size $\delta \in \mathbb{N}$ used to ensure (preservation of) zero knowledge. Overall, the transformation we consider is a function **DSFS** (“duplex-sponge Fiat–Shamir”) of the following form:

$$\text{NARG} := \text{DSFS}[\text{IP}, \text{cdc}, \delta].$$

We outline the transformation in Section 2, and formally describe it in Section 4. Briefly, the permutation p is used to absorb a random salt in Σ^δ and then to alternately absorb prover messages and squeeze verifier messages (suitably relying on the encoding maps and decoding maps in the codec cdc of the given IP).

The transformation *minimizes the number of calls to the given permutation p* . Specifically, when using p in a duplex sponge, each call to p absorbs r elements of Σ or squeezes r elements of Σ . For every $i \in [k]$, let $\ell_{\mathbf{P}}(i)$ and $\ell_{\mathbf{V}}(i)$ be the length of prover and verifier messages for round i when represented over Σ according to the codec’s maps. Then (up to rounding effects that we ignore here):

- the minimum number of calls to p to absorb all the relevant information is $\frac{1}{r} \cdot (\delta + \sum_{i \in [k]} \ell_{\mathbf{P}}(i))$, and
- the minimum number of calls to p to squeeze all the relevant information is $\frac{1}{r} \cdot \sum_{i \in [k]} \ell_{\mathbf{V}}(i)$.

This is essentially what our transformation achieves.

One can view **DSFS** as a “cleaned up” variant of prior transformation variants (some are deployed). These include: (i) Trail of Bit’s Decree [Wri], internally relying on Merlin [Val] which in turn is based on the STROBE protocol framework [Ham17]; (ii) POKSHO [Sig], the library used in Signal for the generation of credentials in group chats; (iii) the SAFE framework [AKMQ23; KMM23; dus]; and (iv) ad-hoc, non-interoperable implementations used in the wild, such as the ones from arkworks.rs [Ark], Aztec [Azt], Microsoft Research [Set], StarkWare [Sta], and Zcash [BLHG]. Notably, **DSFS** avoids the use of padding or IO patterns, leading to greater simplicity and efficiency.

Our main contribution is a rigorous security analysis of **DSFS**, which we discuss next. See Section 2.3 for why indistinguishability of the underlying duplex sponge construction does not suffice for our purposes.

Soundness and knowledge soundness. The soundness (resp., knowledge soundness) of NARG is, as with other Fiat–Shamir transformations, primarily determined by the state-restoration soundness (resp., knowledge

²In security analyses, adversaries also have access to p^{-1} , as in practice permutations are not designed to be hard to invert.

soundness) of IP; see [CY24] for discussions on why state-restoration security intuitively captures the security of the technique underlying Fiat–Shamir transformations. Here it suffices to note that state-restoration soundness is implied by sufficiently strong notions of soundness such as round-by-round soundness and special soundness (and similarly for state-restoration knowledge soundness); see [CY24] (and [AFK22]).

The informal theorem below puts this in quantitative terms. The take-away is that security against t -query adversaries is determined by: (i) the state-restoration security of IP against t -move adversaries; plus (ii) a term $\frac{25t^2}{|\Sigma|^c}$ that represents the security of a duplex sponge based on the ideal permutation $p: \Sigma^{r+c} \rightarrow \Sigma^{r+c}$. (There is also another additive error term, relevant in practice, that is incurred due to decoding biases. We ignore this term in the informal theorem, and postpone its discussion till further below.)

Theorem 1 (informal). *If IP has state-restoration soundness error $\varepsilon_{\text{IP}}^{\text{sr}}$ then NARG has soundness error $\varepsilon_{\text{NARG}}$ such that*

$$\varepsilon_{\text{NARG}}(t) \leq \varepsilon_{\text{IP}}^{\text{sr}}(t) + \frac{25t^2}{|\Sigma|^c}.$$

Moreover, a similar statement holds for knowledge soundness: if IP has state-restoration knowledge soundness error $\kappa_{\text{IP}}^{\text{sr}}$ then NARG has knowledge soundness error κ_{NARG} such that

$$\kappa_{\text{NARG}}(t) \leq \kappa_{\text{IP}}^{\text{sr}}(t) + \frac{25t^2}{|\Sigma|^c}.$$

We outline the ideas underlying the theorem in Section 2.4, and formally state and prove it in Section 6.

Zero knowledge. The zero-knowledge error of NARG is, as with other Fiat–Shamir transformations, primarily determined by the honest-verifier zero-knowledge error of IP. The main difference compared to prior variants is the fact that we use only *a single random salt* in Σ^δ (independent of the number of rounds). Establishing zero knowledge of NARG with this improvement demands a significantly more delicate analysis.

Specifically, the distinguishing advantage against t -query adversaries is the sum of: (i) the honest-verifier zero-knowledge error of IP; (ii) a term $\frac{t}{|\Sigma|^{\min\{\delta, c\}}}$ representing the probability of guessing the salt; and (iii) a term $\frac{t \cdot \sum_{i \in [k]} \lceil \ell_{\mathbf{V}}(i)/r \rceil}{|\Sigma|^{r+c}}$ representing the probability of guessing intermediate permutation states while squeezing. (Similar to Theorem 1, there is an additional error term due to decoding biases, which we discuss later.)

Theorem 2 (informal). *If IP has honest-verifier zero-knowledge error z_{IP} then NARG has adaptive zero-knowledge error z_{NARG} such that*

$$z_{\text{NARG}}(t) \leq z_{\text{IP}} + \frac{t}{|\Sigma|^{\min\{\delta, c\}}} + \frac{t \cdot \sum_{i \in [k]} \lceil \ell_{\mathbf{V}}(i)/r \rceil}{|\Sigma|^{r+c}}.$$

We outline the ideas underlying the theorem in Section 2.5, and formally state and prove it in Section 7.

Decoding biases. All our analyses explicitly handle an additive error term coming from a mismatch that often arises in practice, and must be accounted for.

A codec for IP abstracts the process of encoding prover messages in IP to strings over Σ (to provide as input to p) and decoding verifier messages in IP from strings over Σ (obtained as outputs from p). For the decoding, *the distribution matters*: since IP is public coin, the verifier message for round $i \in [k]$ is a uniform random element ρ_i sampled from a message space $\mathcal{M}_{\mathbf{V}, i}$; in contrast, in the Fiat–Shamir transformation, the verifier message is obtained as the decoding $\rho_i := \psi_i(\hat{\rho}_i)$ of a uniform random string $\hat{\rho}_i$ in $\Sigma^{\ell_{\mathbf{V}}(i)}$ (up to bad events that are already captured by the other error terms).

However, many decoding maps *do not preserve the uniform distribution*. For example, if $\Sigma = \{0, 1\}$ and $\ell_{\mathbf{v}}(i) = 100$ and $\mathcal{M}_{\mathbf{v},i}$ is a 100-bit prime field then every map from $\Sigma^{\ell_{\mathbf{v}}(i)}$ to $\mathcal{M}_{\mathbf{v},i}$ introduces some bias (i.e., does not map the uniform distribution on $\Sigma^{\ell_{\mathbf{v}}(i)}$ to the uniform distribution on $\mathcal{M}_{\mathbf{v},i}$).

We account for this by explicitly tracking decoding biases. We say that a codec has bias ε_{cdc} if, for every $i \in [k]$, $\psi_i: \Sigma^{\ell_{\mathbf{v}}(i)} \rightarrow \mathcal{M}_{\mathbf{v},i}$ is a $\varepsilon_{\text{cdc},i}$ -biased map (ψ_i maps the uniform distribution on $\Sigma^{\ell_{\mathbf{v}}(i)}$ to a distribution that is $\varepsilon_{\text{cdc},i}$ -close to the uniform distribution on $\mathcal{M}_{\mathbf{v},i}$). Our analyses show that the total bias

$$t \cdot \max_{i \in [k]} \varepsilon_{\text{cdc},i} + \sum_{i \in [k]} \varepsilon_{\text{cdc},i}$$

appears as an additional additive error in Theorem 1, and that

$$\max_{i \in [k]} \varepsilon_{\text{cdc},i}$$

appears as an additional additive error in Theorem 2. The takeaway is that in practice one must ensure that the total bias (for the chosen codec) is also suitably small.

To aid practitioners, we provide basic lemmas for establishing the bias of common decoding maps.

Implementation. We implement **DSFS** in Rust, and make it open-source under a BSD license. Our library, called `spongefish` (duplex sponge Fiat–Shamir), is available at github.com/arkworks-rs/spongefish. Features that stand out compared to other deployed variants of the Fiat–Shamir transformation include:

- **Any permutation.** The library supports arbitrary permutation functions over any alphabet. With the library, we include two permutation functions, `keccak` [Sha] and `poseidon` [GKRRS21], plus byte challenges with grinding (a proof of work integrated within the verifier messages).
- **Built-in codecs.** The library includes common encoding maps and decoding maps. We provide encoding maps for absorbing prover messages from field and elliptic curve points. We support traits defined in `arkworks-rs/algebra` and `zkcrypto/group` (two popular Rust frameworks for argument systems). The non-interactive argument string is interoperable across the two frameworks. We provide decoding maps, with negligible biases, for converting byte hash outputs into field elements, and converting field elements hash outputs into bytes (taking into account Theorem C.1). This avoids manual (and error-prone) conversions that are typically seen in the wild.

We elaborate on our library in Section 8. (Our library does not currently target lattice-based argument systems, which often require rejection sampling and sampling from discrete Gaussian or Bernoulli distributions.)

1.2 Application: a BCS transformation from duplex sponges

The Fiat–Shamir transformation is a building block of the BCS transformation [BCS16], which transforms a public-coin interactive oracle proof (IOP) into a corresponding non-interactive argument in the ROM. The BCS transformation is widely used in practice, enabling the construction of highly-efficient post-quantum zkSNARKs (e.g., zkSTARKs).

DSFS can directly replace the relevant building block in the BCS transformation. This modified BCS transformation inherits the efficiency advantages of our transformation, benefitting from an efficient use of the underlying random permutation.

We elaborate on this application in Section 2.6.

1.3 Open problems

Post-quantum security. Establishing the security of **DSFS** against superposition queries to the permutation function remains a challenging open question. Such a result falls in an exciting line of work that establishes the (unconditional) security of notable non-interactive arguments in the QROM: the Fiat–Shamir transformation for sigma protocols [DFMS19; LZ19], and the BCS transformation (which includes the multi-round Fiat–Shamir transformation [CMS19]). Such results are especially valuable given that, in general, security of a non-interactive argument in the ROM does *not* imply security in the QROM [YZ21].

UC security. In this paper we target (adaptive variants of) soundness, knowledge soundness, and zero-knowledge for single instances; this is for simplicity and should be taken as a valuable starting point. Applications may demand stronger variants, for example: security notions that consider multiple adaptively chosen instances; non-malleability; simulation-secure soundness; simulation-secure knowledge soundness; and so on. The gold standard, which in particular implies all of these security properties of “intermediate” strength, is *universally composable security* (UC security) in an appropriate global model (such as the global random oracle model [CJS14; CDGLN18]). The BCS transformation, which includes as a building block the Fiat–Shamir transformation, does satisfy UC security [CF24]. Establishing the UC security of **DSFS** within an appropriate context remains an important open question.

1.4 Related work

This paper is about the security of **DSFS**, a variant of the Fiat–Shamir transformation, in the ideal permutation model. In Section 2.1 we review prior variants that have been studied by theoreticians for other oracle models (i.e., variants in oracle models with a security analysis). Below we summarize two types of other prior work: (1) research on the Fiat–Shamir transformation in the plain model (no oracles); (2) implementations of the Fiat–Shamir transformation (without security analyses).

(1) Fiat–Shamir in the plain model. An oracle model is merely an idealization. A concrete (efficient) function replaces the oracle in the real world. Doing this securely is delicate, as there are examples of interactive protocols for which no efficient function can securely replace the oracle in the Fiat–Shamir transformation [GK03; CGH04; BBHMR19; KRS25]. On the other hand, there are classes of interactive protocols for which there is a way to securely instantiate a random oracle. Most notably the Fiat–Shamir transformation is secure in the plain model for IPs that are round-by-round sound when the oracle is replaced by *correlation-intractable hash functions* (for a suitable relation) [CCHLRR18]. This has led to a beautiful line of work exploring how to construct such hash functions and to understand for which IPs the Fiat–Shamir transformation “works” [CCR16; KRR17; CCRR18; HL18; CCHLRR18; Can+19; PS19; BKM20; JKKZ21; JJ21; HLR21; KLV23; CGJJZ23]. Separately, the Fiat–Shamir transformation can be extended with a suitable proof-of-work, which rules out many problematic cases [AY25].

(2) Fiat–Shamir in the wild. There are differing variants of the Fiat–Shamir transformation in the wild. The STROBE [Ham17] framework is used by several implementations [Val; Ark; Wri], and internally relies on a fixed (not changeable) duplex binary sponge. Arkworks [ark] supports arbitrary sponges (to be used in duplex mode). Some projects internally rely on compression functions, but expose an “absorb” and “squeeze” interface mimicking duplex sponges: Signal [Sig] internally relies on SHA-2, and Halo2 from Zcash [BLHG] internally relies on BLAKE2. The SAFE API works on any field element and is used by Dusk, a privacy-oriented blockchain protocol [dus]. Some IETF standards [DFHSW; CKGW; Hao; LKWL] perform the Fiat–Shamir transformation, each in a slightly different way, and targeting a specific set of binary hashes; no generalized approach or shared design structure is affirmed in the standards community.

2 Techniques

We outline the main ideas underlying our results. In Section 2.1 we review the basic variants of the Fiat–Shamir transformation and their limitations. In Section 2.2 we describe **DSFS**. In Section 2.4 we outline the proof of Theorem 1 (soundness and knowledge soundness) and Section 2.5 we outline the proof of Theorem 2 (zero knowledge). Finally, in Section 2.6 we discuss how our variant can be applied to the BCS transformation.

Throughout, we let $\text{IP} = (\mathbf{P}, \mathbf{V})$ be a public-coin IP for a relation $\mathcal{R}$ with round complexity k . We denote by $(\mathcal{M}_{\mathbf{P},i})_{i \in [k]}$ and $(\mathcal{M}_{\mathbf{V},i})_{i \in [k]}$ the message spaces of prover messages and of verifier messages.

2.1 Review: starting point and the basic variant

A Fiat–Shamir transformation maps $\text{IP} = (\mathbf{P}, \mathbf{V})$ to a non-interactive argument $\text{NARG} = (\mathcal{P}, \mathcal{V})$, for the same relation $\mathcal{R}$, in a certain oracle model. Different transformation variants share a similar template.

- *Oracle setting.* A variant specifies the distribution $\mathcal{D}$ from which a list of oracles $\mathbf{f}$ is sampled.
- *Argument prover.* On input an instance $\mathbb{x}$ and witness $\mathbb{w}$, the argument prover $\mathcal{P}$ queries the oracles $\mathbf{f}$ and outputs an argument string π that includes IP prover messages $(\alpha_i \in \mathcal{M}_{\mathbf{P},i})_{i \in [k]}$ (as well as some salts for the purpose of zero knowledge as we discuss in Section 2.5). Prover messages are obtained by emulating an interaction between the IP prover $\mathbf{P}(\mathbb{x}, \mathbb{w})$ and an imaginary IP verifier by somehow deriving IP verifier messages $(\rho_i \in \mathcal{M}_{\mathbf{V},i})_{i \in [k]}$ via queries to the oracles $\mathbf{f}$. (In fact, $\mathcal{P}$ does not need to derive ρ_k .)
- *Argument verifier.* On input an instance $\mathbb{x}$ and argument string π , the argument verifier $\mathcal{V}$ queries the oracles $\mathbf{f}$ and outputs a decision bit. The decision corresponds to whether $\mathbf{V}(\mathbb{x}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]})$ accepts, where $(\alpha_i)_{i \in [k]}$ are the IP prover messages in π and $(\rho_i)_{i \in [k]}$ are the IP verifier messages derived, using the oracles $\mathbf{f}$, the same way as the (honest) argument prover $\mathcal{P}$.

Variants of the Fiat–Shamir transformation differ in two main features:

- the choice of oracle distribution $\mathcal{D}$; and
- how IP verifier messages $(\rho_i)_{i \in [k]}$ are derived.

We summarize the main variants of the Fiat–Shamir transformation, and explain their limitations. For simplicity, we ignore the matter of zero knowledge (and salts) until we discuss it explicitly in Section 2.5.

(1) The basic variant. The *basic variant* of the Fiat–Shamir transformation, which we denote as **FS**[IP], derives each IP verifier message by hashing the instance and all prior IP prover messages. Specifically, for every $i \in [k]$, the i -th verifier message ρ_i is derived as

$$\rho_i := f_i(\mathbb{x}, \alpha_1, \dots, \alpha_i)$$

where f_i is a random oracle dedicated to round i . This variant typically appears in theory research papers, and its security is discussed in detail in [CY24]. Briefly, the soundness (and knowledge soundness) of the resulting non-interactive argument is determined by the state-restoration soundness (and knowledge soundness) of IP.

(2) The hash-chain variant. The *hash-chain variant* of the Fiat–Shamir transformation, which we denote as **HCFS**[IP], derives each IP verifier message by hashing the current IP prover message and the prior hash. Specifically, for every $i \in [k]$, the i -th verifier message ρ_i is derived as

$$\rho_i := \begin{cases} f_i(\mathbb{x}, \alpha_1) & \text{if } i = 1 \\ f_i(\rho_{i-1}, \alpha_i) & \text{if } i > 1 \end{cases}.$$

This variant is more common in practice-oriented papers, and its security is also discussed in detail in [CY24]. Informally, this variant incurs, relative to **FS**[IP], an additive error of $O(\frac{t^2}{\min_{i \in [k]} |\mathcal{M}_{V,i}|})$ against t -query adversaries, representing the (small) probability that a t -adversary can “break” the hash chain.³

Limitations in practice. Neither variant described above is used in practice due to the same limitation. Hash functions in practice have fixed input and output sizes; instead both variants rely on *oracles whose input and output domains are not fixed*. In **FS**[IP], for each $i \in [k]$, the oracle f_i receives inputs in $\mathcal{M}_{P,1} \times \cdots \times \mathcal{M}_{P,i}$ (plus the instance $\mathfrak{x}$) and produces outputs in $\mathcal{M}_{V,i}$. In **HCFS**[IP], for each $i \in [k]$, the oracle f_i receives inputs in $\mathcal{M}_{P,i}$ (plus the instance $\mathfrak{x}$ or verifier message ρ_{i-1}) and produces outputs in $\mathcal{M}_{V,i}$. These message spaces depend on the protocol IP and instance $\mathfrak{x}$, so the oracles’ domains and ranges are not fixed.

A separate, and typically ignored, issue is message encoding/decoding: there is often a mismatch between the domains/ranges of oracles (e.g., binary strings) and message spaces (e.g., field elements or group elements).

2.2 DSFS: a Fiat–Shamir transformation based on the ideal duplex sponge

Practitioners consider variants of the Fiat–Shamir transformation that support sufficiently large inputs and outputs via multiple calls to a hash function with fixed-size inputs/outputs. Roughly, one can divide variants based on whether one assumes access to an ideal compression function or an ideal permutation function.

In this paper we consider the case of a permutation function, in line with the trends in symmetric cryptography that we discussed. Let cdc be a codec for IP that specifies encoding maps $(\varphi_i: \mathcal{M}_{P,i} \rightarrow \Sigma^{\ell_P(i)})_{i \in [k]}$ and decoding maps $(\psi_i: \Sigma^{\ell_V(i)} \rightarrow \mathcal{M}_{V,i})_{i \in [k]}$. We denote by **DSFS**[IP, cdc] the transformation variant for the interactive proof IP and codec cdc that we outline below; see Section 4 for the formal description.

Duplex sponge. The *duplex sponge* construction [BDPVA12] is a well-established mode of operation for a given permutation $p: \Sigma^{r+c} \rightarrow \Sigma^{r+c}$, which allows absorbing and squeezing strings over Σ , with each call to p handling at most r elements of Σ at a time. This idea has been foundational to the design of hash functions like SHA-3 and primitives like extensible output functions [MF21].

In Section 3.3 we recall in detail the duplex sponge construction based on an ideal permutation $p: \Sigma^{r+c} \rightarrow \Sigma^{r+c}$ (as well as an additional oracle $h: \{0, 1\}^{\leq n} \rightarrow \Sigma^c$ for offline preprocessing as motivated in Theorem 2.1).⁴ Here it suffices to summarize the main interfaces.

- $\text{st}_0 := \text{DS.Start}^p(x)$. Initialize the sponge state st_0 given a binary string x .
- $\text{st}' := \text{DS.Absorb}^p(\text{st}, \chi)$. Absorb the input string $\chi \in \Sigma^*$, changing the sponge state from st to st' .
- $(\rho, \text{st}') := \text{DS.Squeeze}^p(\text{st}, \ell)$. Squeeze an output $\rho \in \Sigma^\ell$, changing the sponge state from st to st' .

Warmup: the sigma-protocol case. For simplicity first we describe the use of the duplex sponge in the special case of a sigma protocol, which is a particularly simple public-coin IP.

In a sigma protocol, the prover sends a first message $\alpha_1 \in \mathcal{M}_{P,1}$, the verifier sends a random message $\rho_1 \in \mathcal{M}_{V,1}$, and the prover sends a second message $\alpha_2 \in \mathcal{M}_{P,2}$. To bridge the gap between message spaces and the permutation’s alphabet, the codec specifies an encoding function $\varphi_1: \mathcal{M}_{P,1} \rightarrow \Sigma^{\ell_P(1)}$ (that is injective) and a decoding function $\psi_1: \Sigma^{\ell_V(1)} \rightarrow \mathcal{M}_{V,1}$ (that has small bias).⁵

³In particular, one must ensure that each $|\mathcal{M}_{V,i}|$ is super-polynomial in the security parameter λ . If not the case, one can straightforwardly modify IP to have enough additional dummy randomness per round (without affecting IP’s security).

⁴Specifically, we consider the duplex sponge *in overwrite mode* rather than canonical duplex sponges “in XOR mode” as it is simpler. Our analysis can be adapted for related constructions such as XOR mode, and different padding functions.

⁵An example for Schnorr’s protocol can be found in Section B.

The argument prover $\mathcal{P}^p(\mathbb{x}, \mathbb{w})$ initializes the sponge state $\text{st}_0 := \text{DS.Start}^p(\mathbb{x})$, uses $\mathbf{P}(\mathbb{x}, \mathbb{w})$ to compute the first IP prover message $\alpha_1 \in \mathcal{M}_{\mathbf{P},1}$, and then derives the single IP verifier message as follows:

1. $\hat{\alpha}_1 := \varphi_1(\alpha_1) \in \Sigma^{\ell_{\mathbf{P}}(1)}$ (encode the IP prover message);
2. $\text{st}'_1 := \text{DS.Absorb}^p(\text{st}_0, \hat{\alpha}_1)$ (absorb the encoded IP prover message);
3. $(\hat{\rho}_1, \text{st}_1) := \text{DS.Squeeze}^p(\text{st}'_1, \ell_{\mathbf{V}}(1))$ (squeeze the encoded IP verifier message);
4. $\rho_1 := \psi_1(\hat{\rho}_1) \in \mathcal{M}_{\mathbf{V},1}$ (decode to obtain the IP verifier message).

Finally, the argument prover gives ρ_1 to $\mathbf{P}(\mathbb{x}, \mathbb{w})$ in order to obtain the second IP prover message $\alpha_2 \in \mathcal{M}_{\mathbf{P},2}$, and outputs the argument string $\pi := (\alpha_1, \alpha_2)$.

The argument verifier $\mathcal{V}^p(\mathbb{x}, \pi)$ checks that $\mathbf{V}(\mathbb{x}, (\alpha_1, \alpha_2), \rho_1)$ accepts, where ρ_1 is derived from $\mathbb{x}$ and α_1 following the same procedure used by the argument prover.

The multi-round case. The above extends to the general case where $\text{IP} = (\mathbf{P}, \mathbf{V})$ is a k -round public-coin IP: for each round $i \in [k]$, the IP prover sends a message $\alpha_i \in \mathcal{M}_{\mathbf{P},i}$ and then the IP verifier sends a random message $\rho_i \in \mathcal{M}_{\mathbf{V},i}$;⁶ after the interaction, the IP verifier's decision is computed as $\mathbf{V}(\mathbb{x}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]})$. In this case, to bridge the gap between message spaces and the permutation's alphabet, the IP's codec specifies encoding functions $(\varphi_i: \mathcal{M}_{\mathbf{P},i} \rightarrow \Sigma^{\ell_{\mathbf{P}}(i)})_{i \in [k]}$ and decoding functions $(\psi_i: \Sigma^{\ell_{\mathbf{V}}(i)} \rightarrow \mathcal{M}_{\mathbf{V},i})_{i \in [k]}$.

The argument prover $\mathcal{P}^p(\mathbb{x}, \mathbb{w})$ and argument verifier $\mathcal{V}^p(\mathbb{x}, \pi)$ each initialize the sponge state $\text{st}_0 := \text{DS.Start}^p(\mathbb{x})$ and then rely on the codec and duplex sponge to derive the relevant IP verifier messages.⁷ Specifically, for every $i \in [k]$, the IP verifier message $\rho_i \in \mathcal{M}_{\mathbf{V},i}$ is derived according to these steps:

1. $\hat{\alpha}_i := \varphi_i(\alpha_i) \in \Sigma^{\ell_{\mathbf{P}}(i)}$ (encode the IP prover message);
2. $\text{st}'_i := \text{DS.Absorb}^p(\text{st}_{i-1}, \hat{\alpha}_i)$ (absorb the encoded IP prover message);
3. $(\hat{\rho}_i, \text{st}_i) := \text{DS.Squeeze}^p(\text{st}'_i, \ell_{\mathbf{V}}(i))$ (squeeze the encoded IP verifier message);
4. $\rho_i := \psi_i(\hat{\rho}_i) \in \mathcal{M}_{\mathbf{V},i}$ (decode to obtain the IP verifier message).

Efficiency. **DSFS** is essentially optimal in terms of number of calls to the (fixed-size) permutation p . Specifically, putting aside the initialization based on the instance $\mathbb{x}$ (typically an offline computation), the argument prover makes $\sum_{i \in [k-1]} \left(\left\lceil \frac{\ell_{\mathbf{P}}(i)}{r} \right\rceil + \left\lceil \frac{\ell_{\mathbf{V}}(i)}{r} \right\rceil \right)$ queries to p and the argument verifier makes $\sum_{i \in [k]} \left(\left\lceil \frac{\ell_{\mathbf{P}}(i)}{r} \right\rceil + \left\lceil \frac{\ell_{\mathbf{V}}(i)}{r} \right\rceil \right)$ queries to p . This is essentially optimal for the information that must be absorbed/squeezed, given that p can absorb/squeeze at most r elements of Σ per query.

Unlike other Fiat–Shamir libraries used in the wild [Wri; Ark], **DSFS** does not hash lengths/labels of the prover and verifier messages (our results guarantee this is secure). Moreover, differently from **HCFS**, **DSFS** uses only a single oracle and, after squeezing a verifier message, r symbols can be absorbed without calling the permutation function. These efficiency details save precious (and unnecessary) queries to p .

Remark 2.1 (alternative instance hashing). The argument prover $\mathcal{P}$ and verifier $\mathcal{V}$ initialize the sponge state as $\text{st}_0 := \text{DS.Start}^p(\mathbb{x})$, a computation that solely depends on the instance $\mathbb{x}$. The natural implementation of this step is $\text{st}_0 := \text{DS.Absorb}^p(\text{st}_*, \varphi_0(\mathbb{x}))$ where st_* is a fixed sponge state and φ_0 is an encoding for binary strings. However, there are settings where initializing st_0 given $\mathbb{x}$ can be viewed as a distinct offline computation which may favor alternative methods to hash $\mathbb{x}$, rather than $\text{DS.Absorb}^p(\text{st}_*, \varphi_0(\mathbb{x}))$.⁸ This choice is convenient in practice, and appears in implementations [Ark] and the literature [KMM23, Fig. 2].

⁶We adopt the convention that each round consists of a prover message followed by a verifier message (in particular, the prover moves first); other conventions can be supported with straightforward changes.

⁷The argument prover derives $(\rho_i)_{i \in [k-1]}$ in order to compute the IP prover messages $(\alpha_i)_{i \in [k]}$, and the argument verifier derives $(\rho_i)_{i \in [k]}$ in order to compute the decision bit.

⁸For example, in recursive composition of zkSNARKs, the recursive circuit may directly receive as input st_0 (rather than having to compute st_0 from $\mathbb{x}$); this means that a different, possibly convenient, hash function can be used to hash $\mathbb{x}$ without affecting the recursive circuit's efficiency.

In Section 3.3 (and subsequent technical sections) we consider a more flexible oracle setting: for instances $\mathbb{x}$ of size at most n , we consider an additional oracle $h: \{0, 1\}^{\leq n} \rightarrow \Sigma^c$ to be used *only* for the sponge state initialization $\text{st}_0 := \text{DS.Start}^h(\mathbb{x})$ via a computation that stores $h(\mathbb{x})$ in st_0 . Accordingly, our analysis separately keeps track of an adversary's queries to h and to p . The analysis then covers, as a special case, the particular choice where $\text{DS.Start}^h(\mathbb{x}) = \text{DS.Absorb}^p(\text{st}_*, \varphi_0(\mathbb{x}))$ (with $h = p$).

2.3 Limitations of indistinguishability

Indistinguishability is a relaxation of indistinguishability that facilitates security reductions [MRH04], and is a popular security notion proved for modes of operations for hash functions. Several variants of the sponge construction are known to be indistinguishable from a random oracle [BDPV08; BDPVA12; KMM23; AB26], including the flavor of the duplex sponge construction used in DS. Roughly, in our setting, such a guarantee means that there exists a probabilistic stateful algorithm $\mathcal{S}_{\text{DS}}$ such that, for every t -query distinguisher D ,

$$\varepsilon_{\text{ind}}(t) := \left| \Pr \left[D^{\text{DS}^p, p, p^{-1}} = 1 \right] - \Pr \left[D^{(f_i)_{i \in [k]}, \mathcal{S}_{\text{DS}}^{(f_i)_{i \in [k]}}} = 1 \right] \right| = O \left(\frac{t^2}{|\Sigma|^c} \right),$$

where $(f_i)_{i \in [k]}$ are the random oracles for the basic variant **FS** (the oracles we wish to emulate).

However, while one can use indistinguishability to prove that the soundness errors of **FS** and **DSFS** differ by at most $\varepsilon_{\text{ind}}(t)$ (as one may expect), we see no way to prove analogous statements for the knowledge soundness errors or zero knowledge errors of **FS** and **DSFS**. In other words:

Indistinguishability does not suffice to establish either knowledge soundness or zero knowledge.

Below we give high-level intuition for why this is the case, and in Section A we discuss indistinguishability in detail (including how qualitatively stronger flavors of indistinguishability suffice for these security properties).

- *Knowledge soundness.* The knowledge soundness property of a non-interactive argument in an oracle model involves the malicious prover's query-answer trace, as the knowledge extractor crucially uses it to deduce a witness (see Theorem 3.6). Indistinguishability ensures that an adversary cannot tell the difference between query access to the duplex sponge DS^p and oracles $(f_i)_{i \in [k]}$ (while given access, respectively, to (p, p^{-1}) and a stateful emulation of these via $\mathcal{S}_{\text{DS}}^{(f_i)_{i \in [k]}}$). But, in order to design a knowledge extractor for **DSFS** based on one for **FS**, we would also need a way to translate query-answer traces relative to p, p^{-1} into corresponding query-answer traces relative to $(f_i)_{i \in [k]}$. *Indistinguishability provides no such capabilities.*

In Section 2.4 we discuss how, in order to establish Theorem 1, we prove a lemma that can be viewed as a qualitatively stronger variant of indistinguishability; while **DSFS**'s security may appear evident, proving our lemma demands considerable effort. In Section A.2.2 we introduce *extraction-friendly indistinguishability*, a notion capturing the key features exposed by the lemma, and show how this flavor of indistinguishability can generically transfer knowledge soundness from one non-interactive argument to another.

- *Zero knowledge.* The zero knowledge property of a non-interactive argument in an oracle model requires programming the oracle (see Theorem 7.4). However, *indistinguishability offers no guarantees in the presence of programming*. Separately, even if a flavor of indistinguishability were used (by extending it with a programming interface as in simulation-friendly extractability in [CY24]), it would only achieve an additive loss of $\varepsilon_{\text{ind}}(t) = O(\frac{t^2}{|\Sigma|^c})$, whereas Theorem 2 achieves an additive loss of $O(\frac{t}{|\Sigma|^c})$.

In Section 2.5 we discuss how we establish Theorem 2 via a direct analysis. In Section A.2.3 we discuss in more detail how indistinguishability is not the right notion to transfer zero knowledge.

2.4 Soundness and knowledge soundness

We outline the ideas behind Theorem 1, which establishes the soundness and knowledge soundness of **DSFS**[IP, cdc]. The key technical lemma is a reduction that relates the security of **DSFS**[IP, cdc] to the security of **FS**[IP] in a precise sense that we explain below. Intuitively this reduction suffices to prove Theorem 1 because the security of **FS**[IP] is determined by the state-restoration security of IP (see [CY24]).

Below we let $(\mathcal{P}, \mathcal{V})$ be the argument prover and verifier of **DSFS**[IP, cdc], and $(\mathcal{P}_{\text{std}}, \mathcal{V}_{\text{std}})$ be the argument prover and verifier of **FS**[IP]. The oracle model for **DSFS**[IP, cdc] is a random permutation $p: \Sigma^{r+c} \rightarrow \Sigma^{r+c}$ and its inverse p^{-1} (we refer to this oracle distribution as $\mathcal{D}_{\mathfrak{S}}$), and the oracle model for **FS**[IP] is random oracles $(f_i)_{i \in [k]}$ where each f_i receives inputs in $\mathcal{M}_{\mathbf{P},1} \times \cdots \times \mathcal{M}_{\mathbf{P},i}$ (plus the instance $\mathfrak{x}$) and produces outputs in $\mathcal{M}_{\mathbf{V},i}$ (we refer to this oracle distribution as $\mathcal{D}_{\text{IP}}$). We must consider adversaries that have access to the inverse of p because, in practice, permutations are not designed to be hard to invert.

On the security reduction. We seek a procedure **D2SAlgo** that converts a query-efficient malicious argument prover $\tilde{\mathcal{P}}$ for $\mathcal{V}$ into a query-efficient malicious argument prover $\tilde{\mathcal{P}}_{\text{std}}$ for $\mathcal{V}_{\text{std}}$ that “behaves the same” as $\tilde{\mathcal{P}}$ up to a certain additive error η_* (which quantifies the security loss of **DSFS**[IP, cdc] over **FS**[IP]).

The precise meaning of “behaves the same” that suffices for our goals is not obvious. Below we motivate the specific form of our key technical lemma, which provides the desired precise meaning.

First we discuss the goal of reducing the soundness of **DSFS**[IP, cdc] to the soundness of **FS**[IP]. Informally, the (adaptive) soundness property upper bounds the probability, over the choice of oracles, that a query-efficient argument prover outputs a pair $(\mathfrak{x}, \pi)$ such that $\mathfrak{x}$ is not in the language and the argument verifier accepts $(\mathfrak{x}, \pi)$. The key quantities in this experiment are the instance $\mathfrak{x}$, argument string π , and decision bit b . Showing that the following two distributions are η_* -close in statistical distance suffices:

$$\left\{ (\mathfrak{x}, \pi, b) \left| \begin{array}{l} (p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}} \\ (\mathfrak{x}, \pi) \leftarrow \tilde{\mathcal{P}}^{p, p^{-1}} \\ b \leftarrow \mathcal{V}^p(\mathfrak{x}, \pi) \end{array} \right. \right\} \quad \text{and} \quad \left\{ (\mathfrak{x}, \pi, b) \left| \begin{array}{l} (f_i)_{i \in [k]} \leftarrow \mathcal{D}_{\text{IP}} \\ (\mathfrak{x}, \pi) \leftarrow \text{D2SAlgo}^{(f_i)_{i \in [k]}}(\tilde{\mathcal{P}}) \\ b \leftarrow \mathcal{V}_{\text{std}}^{(f_i)_{i \in [k]}}(\mathfrak{x}, \pi) \end{array} \right. \right\}.$$

However, showing that these two distributions are close does *not* suffice for knowledge soundness, as we now explain. Informally, the (adaptive) knowledge soundness property upper bounds the probability, over the choice of oracles, that a query-efficient argument prover outputs a pair $(\mathfrak{x}, \pi)$ such that $(\mathfrak{x}, \mathfrak{w})$ is not in the relation and the argument verifier accepts $(\mathfrak{x}, \pi)$, where $\mathfrak{w}$ is the output of the knowledge extractor. The knowledge extractor receives the instance $\mathfrak{x}$, argument string π , black-box access to the argument prover, and the query-answer trace of the argument prover. The query-answer trace is not part of the above statement, so it does not suffice to reduce the knowledge soundness of **DSFS**[IP, cdc] to the knowledge soundness of **FS**[IP].

We fix this by following the pattern, developed in [CY24], of relying on an additional component: a procedure **D2STrace** that converts a query-answer trace for the permutation function p into a corresponding query-answer trace for the oracles $(f_i)_{i \in [k]}$, in a way that is consistent with the prover conversion by **D2SAlgo**.

This leads to our key technical lemma, which considers two distributions over quadruples $(\mathfrak{x}, \pi, b, \text{tr})$.

Lemma 1. *The following two distributions are η_* -close in statistical distance:*

$$\left\{ (\mathfrak{x}, \pi, b, \text{tr}) \left| \begin{array}{l} (p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}} \\ (\mathfrak{x}, \pi) \xleftarrow{\text{tr}_{\tilde{\mathcal{P}}}} \tilde{\mathcal{P}}^{p, p^{-1}} \\ b \xleftarrow{\text{tr}_{\mathcal{V}}} \mathcal{V}^p(\mathfrak{x}, \pi) \\ \text{tr} := \text{D2STrace}(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}}) \end{array} \right. \right\} \quad \text{and} \quad \left\{ (\mathfrak{x}, \pi, b, \text{tr}) \left| \begin{array}{l} (f_i)_{i \in [k]} \leftarrow \mathcal{D}_{\text{IP}} \\ (\mathfrak{x}, \pi) \xleftarrow{\text{tr}_{\tilde{\mathcal{P}}_*}} \text{D2SAlgo}^{(f_i)_{i \in [k]}}(\tilde{\mathcal{P}}) \\ b \xleftarrow{\text{tr}_{\mathcal{V}_{\text{std}}}} \mathcal{V}_{\text{std}}^{(f_i)_{i \in [k]}}(\mathfrak{x}, \pi) \\ \text{tr} := \text{tr}_{\tilde{\mathcal{P}}_*} \| \text{tr}_{\mathcal{V}_{\text{std}}} \end{array} \right. \right\}. \quad (1)$$

The formal statement is Theorem 5.1, and in Section 6 we prove that the lemma enables us to straightforwardly establish soundness and knowledge soundness of **DSFS**[IP, cdc].

Establishing the key lemma. The technical core of our security reduction is upper bounding the statistical distance between the two distributions in Equation 1: we prove that if $\tilde{\mathcal{P}}$ makes at most t queries then the statistical distance is at most

$$\eta_{\star}(t) := \frac{25t^2}{|\Sigma|^c} + t \cdot \max_{i \in [k]} \varepsilon_{\text{cdc},i} + \sum_{i \in [k]} \varepsilon_{\text{cdc},i}.$$

We outline the salient points of our proof, which works across several hybrids.

- First, we replace access to the random permutation p with access to the random oracles

$$\mathbf{g} = \left(g_i : \Sigma^{\ell_{\mathbf{P}}(1)} \times \dots \times \Sigma^{\ell_{\mathbf{P}}(i)} \rightarrow \Sigma^{\ell_{\mathbf{V}}(i)} \right)_{i \in [k]},$$

where each g_i takes as input encoded prover messages and outputs an encoded verifier message. We seek a reduction R that converts an argument prover $\tilde{\mathcal{P}}$ with oracle access to p into an “equivalent” argument prover $R(\tilde{\mathcal{P}})$ with oracle access to $(g_i)_{i \in [k]}$. Informally, R is tasked with using $\mathbf{g}$ to generate encoded verifier messages, rather than the duplex sponge based on p . This entails that R must do all the necessary bookkeeping: it responds to queries in Σ^{r+c} with random answers in Σ^{r+c} and, whenever a sequence of queries to p can be recognized to be absorbing encoded prover messages in $\Sigma^{\ell_{\mathbf{P}}(1)} \times \dots \times \Sigma^{\ell_{\mathbf{P}}(i)}$, for some $i \in [k]$, it queries g_i to obtain a corresponding output in $\Sigma^{\ell_{\mathbf{V}}(i)}$ to later use when another sequence of queries to p can be recognized as the corresponding squeezing operations. The challenge is that, in the world with p , encoded messages are absorbed/squeezed across multiple queries whereas, in the world with $\mathbf{g}$, a single query to the relevant oracle g_i with encoded prover messages yields the corresponding encoded verifier message.

Intuitively, each query to p has input $(s_{\text{R},\text{in}}, s_{\text{C},\text{in}}) \in \Sigma^{r+c}$ and, by the duplex sponge construction, the capacity segment $s_{\text{C},\text{in}}$ “points” to a previous output in the query-answer trace (it matches a previous output capacity segment). This allows us to recover a sequence of rate segments. We refer to this as *backtracking*. If backtracking yields a list of encoded prover messages $(\hat{\alpha}_1, \dots, \hat{\alpha}_i)$, R can query g_i with it.

However, a malicious argument prover might attempt to make backtracking ambiguous. For instance, if it finds a collision in a capacity segment, backtracking might not find a single sequence of rate segments. We carefully prove that this event happens with probability at most $\frac{25t^2}{|\Sigma|^c}$.

Even if backtracking is successful, there remains another obstacle: if $\ell_{\mathbf{V}}(i) > r$, then R cannot just return g_i ’s output $\hat{\rho}_i$ (else $\tilde{\mathcal{P}}$ would trivially distinguish). The reduction R splits $\hat{\rho}_i$ into blocks of length r , and builds a sequence of queries and answers that mimic the duplex sponge in order to yield the same squeezed output. However, each of these $\lceil \ell_{\mathbf{V}}(i)/r \rceil$ queries may have been already queried in the past and, in the unfortunate case where this happens, R cannot respond with a different answer than before (else $\tilde{\mathcal{P}}$ would trivially distinguish). We carefully measure the probability of this event and show that it does not contribute to the overall error of the reduction, for query bounds $t > \sum_{i \in [k]} \lceil \ell_{\mathbf{V}}(i)/r \rceil$.

- We replace access to the random oracles $(g_i)_{i \in [k]}$ with access to the random oracles

$$(f_i : \mathcal{M}_{\mathbf{P},1} \times \dots \times \mathcal{M}_{\mathbf{P},i} \rightarrow \mathcal{M}_{\mathbf{V},i})_{i \in [k]}$$

incurring a statistical distance that is at most $t \cdot \max_{i \in [k]} \varepsilon_{\text{cdc},i} + \sum_{i \in [k]} \varepsilon_{\text{cdc},i}$.

This entails showing that, for every $i \in [k]$, messages can be converted from $\Sigma^{\ell_P(i)}$ to $\mathcal{M}_{P,i}$ and from $\Sigma^{\ell_V(i)}$ to $\mathcal{M}_{V,i}$. We sketch why, for every $i \in [k]$, f_i is $\varepsilon_{\text{cdc},i}$ -close to $\psi_i \circ g_i \circ \varphi_i^{-1}$.

The inverse encoding map φ_i^{-1} does not “disturb” the oracle: φ_i is injective, so two images under φ_i^{-1} are different if and only if their corresponding inputs are different. We deduce that $g_i \circ \varphi_i^{-1}$ produces random (and consistent) outputs over $\Sigma^{\ell_V(i)}$. (Technical remark: the running time of φ_i^{-1} ultimately affects the running time of the security reduction, due to the need to compute φ_i^{-1} for the above translation.)

The decoding map ψ_i may bias the distribution but we are expecting to account for this: by definition, the statistical distance between $\psi_i(\mathcal{U}(\Sigma^{\ell_V(i)}))$ and $\mathcal{U}(\mathcal{M}_{V,i})$ is at most $\varepsilon_{\text{cdc},i}$.

The malicious argument prover $\tilde{P}$ makes at most t queries, each incurring a bias of up to the maximum $\max_{i \in [k]} \varepsilon_{\text{cdc},i}$; in addition, the argument verifier $\mathcal{V}$ makes the equivalent of one query for each f_i , incurring the bias $\sum_{i \in [k]} \varepsilon_{\text{cdc},i}$.

Throughout, the inverse permutation p^{-1} does not help the adversary; intuitively, this follows from the PRP–PRF switching lemma [BR06], though in our analysis need a slightly stronger statement to account for different query bounds for different oracles. The final bound for $\eta_*(t)$ follows by adding the individual errors.

2.5 Zero knowledge

The desired privacy behavior for a Fiat–Shamir transformation is the following: if the interactive proof IP is honest-verifier zero knowledge then the corresponding non-interactive argument NARG is zero knowledge (in the relevant oracle model). Theorem 2 says that **DSFS**[IP, cdc] has this desired behavior. Here we summarize the efficiency improvement achieved over prior variants, and how this affects the proof of zero knowledge.

Programming for zero knowledge. The zero-knowledge property for a non-interactive argument considers an adversary that chooses an instance-witness pair (in the relation) and then receives an argument string that is either the output of the argument prover or the output of the simulator. As in other settings the simulator must have an edge over the adversary; in this oracle setting, the simulator can *program* oracles (necessarily so [Pas03; Wee09]). Briefly, in addition to a simulated argument string, the simulator outputs a list of query-answer pairs for programming oracles, and the adversary subsequently receives query access to oracles modified this way. (See Theorem 7.4 for the definition of zero-knowledge for a non-interactive argument.)

In particular, the adversary sees oracles before programming (when it chooses the instance-witness pair) and after programming (when it receives a simulated argument string). We deduce that any programmed location must be *unpredictable*, or else an adversary could trivially distinguish programmed oracles from non-programmed ones (and thereby distinguish the real-world and simulated-world distributions).

Programming for Fiat–Shamir. Informally, in a Fiat–Shamir transformation, the zero-knowledge simulator samples a simulated IP transcript (by using the honest-verifier zero-knowledge simulator of the IP) and then programs the oracles so that certain query answers match the verifier messages in the simulated IP transcript. Details of this programming differ across variants due to the differing use of oracles to derive verifier messages; regardless, as noted above, in all cases programmed points must be unpredictable.

In simple variants of the Fiat–Shamir transformation this unpredictability is “for free”. For example, in many sigma protocols the first prover message α_1 is uniformly sampled from a cryptographically-large group and the (single) verifier message is derived as $\rho := f(x, \alpha_1)$, and the simulator for the sigma protocol also outputs a random element α_1 . Hence, the programmed location (x, α_1) is unpredictable.

In general, however, we cannot rely on unpredictability coming from the simulated IP transcript. For example, consider an honest-verifier zero-knowledge IP where the verifier moves first (equivalently, the first

round consists of a fixed prover message followed by a random verifier message). In this case there is no entropy coming from a prover message prior to the verifier message.

Salts, salts, salts. The common solution is to introduce salt strings into random oracle queries in order to make them unpredictable. For example, [CY24] analyzes the variants **FS**[IP] and **HCFS**[IP] of the Fiat–Shamir transformation where a *salt string per IP round is used* (i.e., a total of k rounds).

The same approach of “one salt per oracle query” would work for **DSFS**[IP, cdc] but, unfortunately, would result in *too many salt strings*, specifically $\sum_{i \in [k]} \left(\left\lceil \frac{\ell_{\mathbf{P}}(i)}{r-\delta} \right\rceil + \left\lceil \frac{\ell_{\mathbf{V}}(i)}{r-\delta} \right\rceil \right)$ salt strings of length δ . Indeed, we consider a setting where the oracle has fixed size, which means that the number of oracle queries is proportional to the communication complexity, not round complexity. Including all these salts in the final argument string π would be unacceptable due to the significant increase in argument size.

One salt suffices. We show that, in **DSFS**[IP, cdc], absorbing a single salt at the beginning suffices.

The (honest) argument prover $\mathcal{P}^p(\mathbb{x}, \mathbb{w})$ initializes the sponge state $\text{st}'_0 := \text{DS.Start}^p(\mathbb{x})$ (as before), samples a random salt $\tau \in \Sigma^\delta$, and absorbs it $\text{st}_0 := \text{Absorb}^p(\text{st}'_0, \tau)$; the rest of $\mathcal{P}$ remains unchanged, except the salt is included in the argument string as $\pi := (\tau, (\alpha_i)_{i \in [k]})$. The argument verifier $\mathcal{V}^p(\mathbb{x}, \pi)$ similarly initializes the sponge state and absorbs the salt before proceeding as before.

Establishing zero knowledge entails showing that every absorb and squeeze query after initialization is unpredictable (over a random $\tau \in \Sigma^\delta$). We do so via a direct proof (without reducing zero-knowledge of **DSFS**[IP, cdc] to zero knowledge of **FS**[IP]). We carefully bound the statistical distance between the real-world distribution and simulated-world distribution, by taking into account the effect of the (single) salt and the biases incurred by the decoding maps. The upper bound we prove is

$$z_{\text{IP}} + \frac{t}{|\Sigma|^{\min\{\delta, c\}}} + \frac{t \cdot \sum_{i \in [k]} \lceil \ell_{\mathbf{V}}(i)/r \rceil}{|\Sigma|^{r+c}} + \max_{i \in [k]} \varepsilon_{\text{cdc}, i}.$$

We split our analysis in intermediate hybrids, isolating different distinguishing advantages.

- First, we show that, after absorbing $\tau \leftarrow \mathcal{U}(\Sigma^\delta)$, the sponge state is unpredictable: if $\delta > c$ then the probability that the adversary guesses the capacity segment resulting after absorbing $\lceil \frac{\delta}{r} \rceil$ blocks is at most $\frac{t}{|\Sigma|^c}$; else if $\delta \leq c$ then this probability is (trivially) at most $\frac{t}{|\Sigma|^\delta}$.
- Then, we show that the $\sum_{i \in [k]} \lceil \ell_{\mathbf{V}}(i)/r \rceil$ permutation states associated to the verifier messages are unpredictable. The simulator re-programs these, and the probability that a t -bound adversary queries any one of these is at most $\frac{t}{|\Sigma|^{r+c}}$.
- Finally, we consider the biases arising from decoding maps to the k verifier messages. This involves, for each $i \in [k]$, measuring the statistical distance between

$$\mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)}) \quad \text{and} \quad (\psi_i^{-1} \circ \psi_i \circ \mathcal{U})(\Sigma^{\ell_{\mathbf{V}}(i)}).$$

This is done in two steps: first showing that $\psi_i(\mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)}))$ is $\varepsilon_{\text{cdc}, i}$ -close to $\mathcal{U}(\mathcal{M}_{\mathbf{V}, i})$, and then showing that $\psi_i^{-1}(\mathcal{U}(\mathcal{M}_{\mathbf{P}, i}))$ is identically distributed to $\mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)})$.

Overall, our proof of Theorem 2 is significantly more delicate than corresponding ones in [CY24]. See Section 7 for the formal statement of Theorem 2 and its proof.

Remark 2.2 (one salt in prior variants). Our analysis leads to an efficiency improvement in **FS**[IP] and **HCFS**[IP]. Specifically, our analysis can be adapted to show that one salt would have sufficed for zero knowledge in both of those constructions, rather than k salts as done in [CY24].

2.6 Application: BCS transformation from a duplex sponge

The BCS transformation can be viewed as the composition of two separate transformations: (i) the **iBCS** (interactive BCS) transformation, which maps a public-coin IOP into a corresponding public-coin (succinct) interactive argument, by relying on a Merkle commitment based on a random oracle; (ii) the Fiat–Shamir transformation, which maps the public-coin interactive argument into a corresponding non-interactive argument, by relying on other random oracles. This can be informally summarized via the following “transformation equation” (and is formally discussed in [CY24]):

$$\mathbf{BCS}[\text{IOP}] = \mathbf{HCFS}[\mathbf{iBCS}[\text{IOP}]]. \quad (2)$$

We can replace the second transformation with our variant to obtain a variant of the BCS transformation that internally relies on a (fixed-size) ideal permutation for the purpose of a Fiat–Shamir transformation. (The compression function for the Merkle commitment remains.) This can be informally summarized as follows:

$$\mathbf{BCS}[\text{IOP}] = \mathbf{DSFS}[\mathbf{iBCS}[\text{IOP}], \text{cdc}]. \quad (3)$$

where $\text{cdc} = ((\varphi_i)_{i \in [k]}, (\psi_i)_{i \in [k]})$ is an appropriate codec for **iBCS**[\text{IOP}], i.e., for encoding Merkle commitments (rather than IOP prover messages) and decoding IOP verifier messages for IOP.⁹

Upper bounds on the soundness and knowledge soundness follow from Theorem 1, replacing the error $\frac{t^2}{2^\lambda}$ of **HCFS** with that of **DSFS**; the multi-extraction error κ_{MT} that arises in **iBCS** remains. Similarly, an upper bound on zero knowledge follows from Theorem 2; the hiding error z_{MT} that arises in **iBCS** remains. Below we highlight in blue the contributions from **DSFS** that arise from our results.

Corollary 1 (informal). *Let IOP be a public-coin IOP and let $\text{NARG} := \mathbf{DSFS}[\mathbf{iBCS}[\text{IOP}], \text{cdc}]$.*

- *If IOP has state-restoration soundness error $\varepsilon_{\text{IOP}}^{\text{sr}}$ then NARG has soundness error $\varepsilon_{\text{NARG}}$ such that*

$$\varepsilon_{\text{NARG}}(t) \leq \varepsilon_{\text{IOP}}^{\text{sr}}(t) + \kappa_{\text{MT}}(t, l, (t+1) \cdot k, k) + \frac{25t^2}{|\Sigma|^c} + t \cdot \max_{i \in [k]} \varepsilon_{\text{cdc}, i} + \sum_{i \in [k]} \varepsilon_{\text{cdc}, i}.$$

- *If IOP has state-restoration knowledge soundness error $\kappa_{\text{IOP}}^{\text{sr}}$ then NARG has knowledge soundness error κ_{NARG} such that*

$$\kappa_{\text{NARG}}(t) \leq \kappa_{\text{IOP}}^{\text{sr}}(t) + \kappa_{\text{MT}}(t, l, (t+1) \cdot k, k) + \frac{25t^2}{|\Sigma|^c} + t \cdot \max_{i \in [k]} \varepsilon_{\text{cdc}, i} + \sum_{i \in [k]} \varepsilon_{\text{cdc}, i}.$$

- *If IOP has honest-verifier zero-knowledge error z_{IOP} then NARG has adaptive zero-knowledge error z_{NARG} such that*

$$z_{\text{NARG}}(t) \leq z_{\text{IOP}} + z_{\text{MT}}(l, \delta, q, t) + \frac{t}{|\Sigma|^{\min\{\delta, c\}}} + \frac{t \cdot \sum_{i \in [k]} \lceil \ell_{\mathbf{v}}(i)/r \rceil}{|\Sigma|^{r+c}} + \max_{i \in [k]} \varepsilon_{\text{cdc}, i}.$$

⁹In **iBCS**[\text{IOP}] the prover’s final message in round $k+1$, which contains answers and openings, is not absorbed, so there are no corresponding codec functions for this round.

3 Preliminaries

Several definitions in this section (most notably related to interactive proofs, non-interactive arguments, and the Fiat–Shamir transformation) are direct adaptations of definitions in [CY24].

3.1 Notation

With $[n]$ we denote the range $\{1, \dots, n\}$. An alphabet Σ is a non-empty finite set¹⁰, containing an element $0 \in \Sigma$ denoted *default value*. The Kleene closure Σ^* denotes all finite-length strings over Σ . Let A and B be algorithms. We say that A *uses* B as a *black-box*, denoted $A(\textcolor{brown}{B})$, to emphasize that A relies only on the input-output functionality of B , rather than also using its description. If A invokes B some number of times (on inputs of its choice) and otherwise does not “look” at B ’s description. More precisely, for every two algorithms B_1 and B_2 that represent the same function, $A(\textcolor{brown}{B}_1) = A(\textcolor{brown}{B}_2)$.

3.2 Basics

The preimage of a function $f: X \rightarrow Y$ is a map $f^{-1}: 2^Y \rightarrow 2^X: C \mapsto \{x \in X: f(x) \in C\}$. With a slight abuse of notation, for $x \in X$ we denote by $f^{-1}(x)$ the preimage of the singleton $\{x\}$ under f . With $\text{Im}(f)$ we denote the image of the function f .

A **relation** $\mathcal{R}$ is a set of instance-witness pairs $(\mathbb{x}, \mathbb{w})$. The **language** associated to a relation $\mathcal{R}$ is the set $\mathcal{L}(\mathcal{R})$ of all instances $\mathbb{x}$ for which there exists a witness $\mathbb{w}$ such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$. We denote by $|\mathbb{x}|$ the length of the instance $\mathbb{x}$ (according to some length measure, e.g., the number of bits used to describe $\mathbb{x}$).

We write $a \leftarrow \mathcal{D}$ to denote that the element a is sampled according to the distribution $\mathcal{D}$. We denote by $\mathcal{U}(S)$ the uniform distribution over a given non-empty finite set S . We denote by $X \rightarrow Y$ the set of functions from X to Y , so $\mathcal{U}(X \rightarrow Y)$ refers to the uniform distribution over all such functions. To ease notation, the act of sampling multiple functions $f_i: X_i \rightarrow Y_i$ is denoted as

$$\mathbf{f} = (f_i)_{i \in [k]} \leftarrow \mathcal{U}((X_i \rightarrow Y_i)_{i \in [k]}),$$

indicating that, for each $i \in [k]$, $f_i \leftarrow \mathcal{U}(X_i \rightarrow Y_i)$ is sampled uniformly and independently at random.

We denote $\mathcal{A}^f$ the fact that algorithm $\mathcal{A}$ has query access to the function f , and $\mathcal{A}^{\mathbf{f}}$ the fact that $\mathcal{A}$ has query access to the functions $\mathbf{f} = (f_i)_{i \in [k]}$. We say that $\mathcal{A}$ is t -query if it makes at most t queries (to any oracle), and is $(t_i)_{i \in [k]}$ -query if, for every $i \in [k]$, it makes at most t_i queries to the i -th oracle. We write $a \stackrel{\text{tr}}{\leftarrow} \mathcal{A}^f$ to denote the fact that the query-answer trace of $\mathcal{A}$ with respect to the oracle f is tr ; we also write $a \stackrel{\text{tr}}{\leftarrow} \mathcal{A}^{\mathbf{f}}$ for the case of multiple oracles, in which case tr additionally includes for each query-answer pair the information about which oracle it is for. When sampling from multiple oracles f_i for $i \in [k]$, we denote query-answer trace as tr .

Definition 3.1. We assume the random oracle is implemented via **lazy sampling**, that is, via the following stateful algorithm:

- Set the internal state tr to be the empty mapping.
- Upon receiving a query x , if $x \in \text{tr}$, return $\text{tr}[x]$; otherwise, sample $y \leftarrow \mathcal{U}(Y)$ uniformly at random, set $\text{tr}[x] := y$ and return y .

The list of key-value pairs of tr is called the **trace** of the random oracle.

¹⁰We will generally assume that Σ has total order. This is only needed in order to perform dichotomic search in logarithmic time over a list of Σ -tuples (see Sections 5.2 and 5.3).

We conclude with a simple lemma that we use multiple times.

Lemma 3.2. *Let $\psi: B \rightarrow A$ be a surjective function and let ψ^{-1} be the procedure that samples a preimage of ψ uniformly at random. Then:*

$$\Delta(\mathcal{U}(B), \psi^{-1} \circ \psi \circ \mathcal{U}(B)) = 0.$$

Proof. Let $\bar{x} := \{x' \in B: \psi(x) = \psi(x')\}$ be the set of representatives of x in B/ψ . Recall that $\psi^{-1}(y)$ samples uniformly at random a preimage of y . Then:

$$\begin{aligned} & \Delta(\mathcal{U}(B), (\psi^{-1} \circ \psi \circ \mathcal{U})(B)) \\ &= \sum_{x \in B} \left| \Pr[x' = x \mid x' \leftarrow \mathcal{U}(\Sigma^{\ell_{\mathbf{v}}(i)})] - \Pr \left[x'' = x \mid \begin{array}{l} x' \leftarrow \mathcal{U}(B) \\ y := \psi(x') \\ x'' \leftarrow \psi^{-1}(y) \end{array} \right] \right| \\ &= \sum_{x \in B} \left| \frac{1}{|B|} - \Pr[x' = x \mid x' \leftarrow \mathcal{U}(\bar{x})] \cdot \Pr[\psi(x') = \psi(x) \mid x' \leftarrow \mathcal{U}(\Sigma^{\ell_{\mathbf{v}}(i)})] \right| \\ &= \sum_{x \in B} \left| \frac{1}{|B|} - \frac{1}{|\bar{x}|} \cdot \Pr[\psi(x') = \psi(x) \mid x' \leftarrow \mathcal{U}(B)] \right| \\ &= \sum_{x \in B} \left| \frac{1}{|B|} - \frac{1}{|\bar{x}|} \cdot \frac{|\bar{x}|}{|B|} \right| = 0. \end{aligned}$$

□

3.3 Duplex sponge in overwrite mode

We describe a duplex sponge (in overwrite mode) that is directly inspired from [BDPVA12]. The setting is one where there are two random oracles:

- a random function $h: \{0, 1\}^{\leq n} \rightarrow \Sigma^c$, and
- a random permutation $p: \Sigma^{r+c} \rightarrow \Sigma^{r+c}$ (that we sometimes view as a function $p: \Sigma^r \times \Sigma^c \rightarrow \Sigma^r \times \Sigma^c$).

Here Σ is a finite (non-empty) alphabet, c is the **capacity**, r is the **rate**, and $r + c$ is the **permutation length**.

Informally, the duplex sponge consists of three procedures that initialize and evolve a **sponge state** st , which is a tuple

$$\text{st} = (\mathbf{s}, i_A, i_S) \in \Sigma^{r+c} \times [0, r] \times [0, r]$$

consisting of a permutation state $\mathbf{s} = (\mathbf{s}_R, \mathbf{s}_C) \in \Sigma^{r+c}$ (with a rate segment $\mathbf{s}_R$ and a capacity segment $\mathbf{s}_C$), an absorbing index $i_A \in [0, r]$ for (over)writing parts of the permutation state, and a squeezing index $i_S \in [0, r]$ for reading from parts of the permutation state; reading and writing is always to the rate segment.

- An *initialization phase*, given an input x , produces the initial sponge state $\text{st}_0 = (\mathbf{s}, i_A, i_S) = ((0^r, \mathbf{s}_C), 0, r)$ where the rate segment is initialized to 0^r and the capacity segment is initialized to $\mathbf{s}_C := h(x)$.
- An *absorbing phase*, where an input block $\chi \in \Sigma^*$ is written into rate segment of the sponge state, interleaved with applications of the permutation p . (The overwrite mode refers to the fact that we overwrite the rate segment with the input block, rather than “xoring” the rate segment with the input block.)

- A *squeezing phase* where, for a desired output length $\ell \in \mathbb{N}$, yields an output $\rho \in \Sigma^\ell$ that is obtained from the rate segment of the permutation state, in blocks of r elements interleaved with calls to the permutation p .

In more detail, the duplex sponge is constructed as follows.

Construction 3.3. For $r, c, n \in \mathbb{N}$, let $h: \{0, 1\}^{\leq n} \rightarrow \Sigma^c$ and $p: \Sigma^{r+c} \rightarrow \Sigma^{r+c}$ be oracles. The **duplex sponge (in overwrite mode)** is a list of procedures $\text{DS} := (\text{Start}, \text{Absorb}, \text{Squeeze})$ that have query access to the oracles (h, p, p^{-1}) and work as follows.

- $\text{st}_0 := \text{DS.Start}^h(x)$.

Given as input $x \in \{0, 1\}^{\leq n}$, compute $\mathbf{s}_C := h(x) \in \Sigma^c$, set the initial permutation state $\mathbf{s} := (0^r, \mathbf{s}_C) \in \Sigma^{r+c}$, set the initial absorbing index $i_A := 0$, set the initial squeezing index $i_S := r$, and output the initial sponge state:

$$\text{st}_0 := (\mathbf{s}, i_A, i_S) = ((0^r, \mathbf{s}_C), 0, r) \in \Sigma^{r+c} \times [0, r] \times [0, r].$$

- $\text{st}' := \text{DS.Absorb}^p(\text{st}, \chi)$.

Given as input $\text{st} = (\mathbf{s}, i_A, i_S)$ and $\chi \in \Sigma^*$, output st' computed as follows.

1. Set $i'_S := r$.
2. If χ is the empty string, then return $\text{st}' := (\mathbf{s}, i_A, i'_S)$.
3. Otherwise:
 - (a) If $0 \leq i_A < r$ then:
 - Let χ be the first element of χ , and let χ' be the remaining elements of χ .
 - Let $\mathbf{s}'$ be the permutation state obtained by overwriting the i_A -th element of $\mathbf{s}$ with χ .
 - Set $i'_A := i_A + 1$ (increment the absorbing index).
 - Set $\text{st}' := (\mathbf{s}', i'_A, i'_S)$.
 - Output $\text{DS.Absorb}(\text{st}', \chi')$.
 - (b) If $i_A = r$ then:
 - Set $\mathbf{s}' := p(\mathbf{s})$.
 - Set $i'_A := 0$ (reset the absorbing index).
 - Set $\text{st}' := (\mathbf{s}', i'_A, i'_S)$.
 - Output $\text{DS.Absorb}(\text{st}', \chi)$.

- $(\rho, \text{st}') := \text{DS.Squeeze}^p(\text{st}, \ell)$.

Given as input $\text{st} = (\mathbf{s}, i_A, i_S)$ and $\ell \in \mathbb{N}$, output $\rho \in \Sigma^\ell$ and st' computed as follows.

1. If $\ell = 0$ set ρ to the empty string, and output (ρ, st) .
2. Otherwise:
 - (a) Set $i'_A := 0$ (the absorbing index is not used while squeezing).
 - (b) If $0 \leq i_S < r$ then:
 - Set $i'_S := i_S + 1$ (increment the squeezing index).
 - Set $\text{st}' := (\mathbf{s}, i'_A, i'_S)$.
 - Compute $(\rho', \text{st}') := \text{DS.Squeeze}(\text{st}', \ell - 1)$.
 - Output $(\mathbf{s}_{i_S} \parallel \rho', \text{st}')$.
 - (c) If $i_S = r$ then:
 - Set $\mathbf{s}' := p(\mathbf{s})$.
 - Set $i'_S := 0$ (reset the squeezing index).

- Set $\text{st}' := (s', i'_A, i'_S)$.
- Output $\text{DS.Squeeze}(\text{st}', \ell)$.

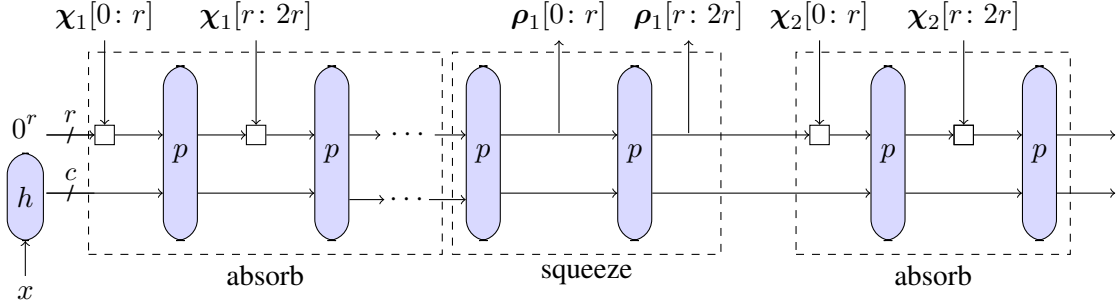


Figure 1: Diagram of the duplex sponge in Theorem 3.3.

3.4 Non-interactive arguments in oracle models

An *oracle distribution* $\mathcal{D}$ receives as input a security parameter $\lambda \in \mathbb{N}$ and an instance size bound $n \in \mathbb{N}$, and samples a list $\mathbf{f}$ of functions. A *non-interactive argument* (NARG) in the $\mathcal{D}$ -oracle model is a tuple $\text{NARG} = (\mathcal{P}, \mathcal{V})$, where $\mathcal{P}$ is an oracle algorithm known as the *argument prover* and $\mathcal{V}$ is an oracle algorithm known as the *argument verifier*, that works as follows. For a given security parameter $\lambda \in \mathbb{N}$ and instance size bound $n \in \mathbb{N}$, a list of functions $\mathbf{f}$ is sampled according to $\mathcal{D}(\lambda, n)$ and is made public; anyone, including the argument prover $\mathcal{P}$ and the argument verifier $\mathcal{V}$, can query any function in $\mathbf{f}$. The argument prover $\mathcal{P}$ receives as input an instance $\mathbf{x}$ and witness $\mathbf{w}$, and outputs an argument string π . The argument verifier $\mathcal{V}$ receives as input the instance $\mathbf{x}$ and argument string π , and outputs a bit denoting whether to accept (the bit is 1) or reject (the bit is 0). Both $\mathcal{P}$ and $\mathcal{V}$ additionally also receive as input λ and n , but we omit them for ease of notation. We consider several properties for a non-interactive argument, stated below. The definitions are from [CY24] (and straightforwardly adapted to any oracle model); we refer the reader to the relevant discussions there.

Definition 3.4. A non-interactive argument $\text{NARG} = (\mathcal{P}, \mathcal{V})$ for relation $\mathcal{R}$ in the $\mathcal{D}$ -oracle model has **(perfect) completeness** if for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, and instance-witness pair $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ such that $|\mathbf{x}| \leq n$,

$$\Pr \left[\mathcal{V}^{\mathbf{f}}(\mathbf{x}, \pi) = 1 \mid \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}(\lambda, n) \\ \pi \leftarrow \mathcal{P}^{\mathbf{f}}(\mathbf{x}, \mathbf{w}) \end{array} \right] = 1.$$

The probability is taken over $\mathbf{f}$ and any randomness of the argument prover $\mathcal{P}$ and verifier $\mathcal{V}$.

Definition 3.5. A non-interactive argument $\text{NARG} = (\mathcal{P}, \mathcal{V})$ for relation $\mathcal{R}$ in the $\mathcal{D}$ -oracle model has **soundness error** $\varepsilon_{\text{NARG}}$ if for every security parameter $\lambda \in \mathbb{N}$, query bound $t \in \mathbb{N}$, t -query malicious argument prover $\tilde{\mathcal{P}}$, and instance size bound $n \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} |\mathbf{x}| \leq n \\ \wedge \mathbf{x} \notin \mathcal{L}(\mathcal{R}) \\ \wedge \mathcal{V}^{\mathbf{f}}(\mathbf{x}, \pi) = 1 \end{array} \mid \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}(\lambda, n) \\ (\mathbf{x}, \pi) \leftarrow \tilde{\mathcal{P}}^{\mathbf{f}} \end{array} \right] \leq \varepsilon_{\text{NARG}}(\lambda, t, n).$$

The probability is taken over $\mathbf{f}$ and any randomness of the argument verifier $\mathcal{V}$.

Definition 3.6. A non-interactive argument $\text{NARG} = (\mathcal{P}, \mathcal{V})$ for a relation $\mathcal{R}$ in the $\mathcal{D}$ -oracle model has **straightline knowledge soundness error** $\kappa_{\text{NARG}}(\lambda, t, n)$ if there exists a polynomial-time deterministic algorithm $\mathcal{E}$ (the extractor) such that for every security parameter $\lambda \in \mathbb{N}$, query bound $t \in \mathbb{N}$, t -query deterministic argument prover $\tilde{\mathcal{P}}$, and instance size bound $n \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge (\mathbb{x}, \mathbb{w}) \notin \mathcal{R} \\ \wedge \mathcal{V}^{\mathbf{f}}(\mathbb{x}, \pi) = 1 \end{array} \middle| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}(\lambda, n) \\ (\mathbb{x}, \pi) \xleftarrow{\text{tr}} \tilde{\mathcal{P}}^{\mathbf{f}} \\ \mathbb{w} \leftarrow \mathcal{E}(\mathbb{x}, \pi, \text{tr}) \end{array} \right] \leq \kappa_{\text{NARG}}(\lambda, t, n).$$

Definition 3.7. Let $\text{NARG} = (\mathcal{P}, \mathcal{V})$ be a non-interactive argument in the $\mathcal{D}$ -oracle model. A deterministic argument prover $\tilde{\mathcal{P}}$ has **failure probability** $\delta_{\tilde{\mathcal{P}}}(\lambda, n)$ if for every security parameter $\lambda \in \mathbb{N}$ and instance size bound $n \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} |\mathbb{x}| > n \\ \vee \mathcal{V}^{\mathbf{f}}(\mathbb{x}, \pi) = 0 \end{array} \middle| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}(\lambda, n) \\ (\mathbb{x}, \pi) \leftarrow \tilde{\mathcal{P}}^{\mathbf{f}} \end{array} \right] \leq \delta_{\tilde{\mathcal{P}}}(\lambda, n).$$

Definition 3.8. A non-interactive argument $\text{NARG} = (\mathcal{P}, \mathcal{V})$ for a relation $\mathcal{R}$ in the $\mathcal{D}$ -oracle model has **rewinding knowledge soundness error** $\kappa_{\text{NARG}}(\lambda, t, n, \delta_{\tilde{\mathcal{P}}}(\lambda, n))$ **with extraction time** et_{NARG} if there exists a probabilistic algorithm $\mathcal{E}$ (the extractor) such that for every security parameter $\lambda \in \mathbb{N}$, query bound $t \in \mathbb{N}$, t -query deterministic argument prover $\tilde{\mathcal{P}}$ with failure probability $\delta_{\tilde{\mathcal{P}}}$ and running time $\tau_{\tilde{\mathcal{P}}}$, and instance size bound $n \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge (\mathbb{x}, \mathbb{w}) \notin \mathcal{R} \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}(\lambda, n) \\ (\mathbb{x}, \pi) \xleftarrow{\text{tr}} \tilde{\mathcal{P}}^{\mathbf{f}} \\ b \xleftarrow{\text{tr}_{\mathcal{V}}} \mathcal{V}^{\mathbf{f}}(\mathbb{x}, \pi) \\ \mathbb{w} \leftarrow \mathcal{E}(\mathbb{x}, \pi, \text{tr}, \text{tr}_{\mathcal{V}}, \tilde{\mathcal{P}}) \end{array} \right] \leq \kappa_{\text{NARG}}(\lambda, t, n, \delta_{\tilde{\mathcal{P}}}(\lambda, n)).$$

Moreover, $\mathcal{E}$ runs in expected time $\text{et}_{\text{NARG}}(\lambda, t, n, \delta_{\tilde{\mathcal{P}}}(\lambda, n), \tau_{\tilde{\mathcal{P}}}(\lambda, n))$ (over the given inputs and internal randomness).

3.5 Interactive proofs

An *interactive proof* (IP) for relation $\mathcal{R}$ is a tuple of interactive algorithms $\text{IP} = (\mathbf{P}, \mathbf{V})$ that works as follows. The IP prover $\mathbf{P}$ receives as input an instance-witness pair $(\mathbb{x}, \mathbb{w})$ and the IP verifier $\mathbf{V}$ receives as input an instance $\mathbb{x}$. They interact across k rounds and, in each round $i \in [k]$, the IP prover sends a message α_i and then the IP verifier sends a message ρ_i . After the interaction, the IP verifier outputs a decision bit $b \in \{0, 1\}$. We denote by $\langle \mathbf{P}, \mathbf{V} \rangle_{\text{IP}}$ the random variable that equals the output of $\mathbf{V}$ after interacting with $\mathbf{P}$ (the probability is taken over the randomness of $\mathbf{P}$ and of $\mathbf{V}$).

Definition 3.9. $\text{IP} = (\mathbf{P}, \mathbf{V})$ for relation $\mathcal{R}$ is (perfectly) **complete** if for every instance-witness pair $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$

$$\Pr[\langle \mathbf{P}(\mathbb{x}, \mathbb{w}), \mathbf{V}(\mathbb{x}) \rangle_{\text{IP}} = 1] = 1.$$

Definition 3.10. $\text{IP} = (\mathbf{P}, \mathbf{V})$ for relation $\mathcal{R}$ has **soundness error** $\varepsilon_{\text{IP}}^{\text{snd}}$ if for every $\mathbb{x} \notin \mathcal{L}(\mathcal{R})$ and malicious IP prover $\tilde{\mathbf{P}}$

$$\Pr[\langle \tilde{\mathbf{P}}, \mathbf{V}(\mathbb{x}) \rangle_{\text{IP}} = 1] \leq \varepsilon_{\text{IP}}^{\text{snd}}(\mathbb{x}).$$

We additionally define $\varepsilon_{\text{IP}}^{\text{snd}}(n) := \max_{\substack{\mathbb{x} \notin \mathcal{L}(\mathcal{R}) \\ |\mathbb{x}| \leq n}} \varepsilon_{\text{IP}}^{\text{snd}}(\mathbb{x})$.

This work focuses on public-coin interactive proofs. An IP is **public coin** if every message sent by $\mathbf{V}$ is a random string in Σ^* , each sampled independently at random, and $\mathbf{V}$ has no other randomness. In this case, we can view the IP verifier as a deterministic algorithm $\mathbf{V}(\mathbb{x}, \alpha, \rho)$ that outputs a decision bit (where α is the sequence of messages sent by the prover and ρ is the sequence of random messages sent by the verifier).

Message spaces. For every $i \in [k]$, we denote by $\mathcal{M}_{\mathbf{P},i}$ and $\mathcal{M}_{\mathbf{V},i}$ the prover message space and verifier message space for the i -th round. (These spaces may be functions of, e.g., the instance size.) The message spaces induce corresponding function spaces that we use later on.

Definition 3.11. For every instance size bound $n \in \mathbb{N}$, round $i \in [k]$, and salt size $\delta_i \in \mathbb{N}$, we define the following function space: $\mathcal{Z}_i(\delta_i, n) := \{0, 1\}^{\leq n} \times \{0, 1\}^{\delta_i} \times \mathcal{M}_{\mathbf{P},1} \times \cdots \times \mathcal{M}_{\mathbf{P},i} \rightarrow \mathcal{M}_{\mathbf{V},i}$.

State restoration. We describe a state-restoration soundness and knowledge soundness, security notions that (informally) allow a malicious prover to obtain (up to a query bound t) verifier messages for the same prover input. Our formulation is more permissive than [CY24, Def. 12.1], as it allows the prover not to pick a salt for each round. This change does not affect the soundness results given in the book.

Definition 3.12. The **IP state-restoration game** for $\text{IP} = (\mathbf{P}, \mathbf{V})$ with salt sizes $\delta := (\delta_1, \dots, \delta_k) \in \mathbb{N}^k$, random oracles $\mathbf{f} = (f_i)_{i \in [k]} \in \mathcal{U}((\mathcal{Z}_i(\delta_i, n))_{i=1}^k)$, and IP state-restoration prover $\tilde{\mathbf{P}}^{\text{sr}}$ is defined below.

$\text{SR}_{\text{IP}, \mathbf{f}, \tilde{\mathbf{P}}^{\text{sr}}}(\delta)$:

1. Repeat the following until $\tilde{\mathbf{P}}^{\text{sr}}$ decides to exit the loop.
 - (a) $\tilde{\mathbf{P}}^{\text{sr}}$ outputs $(\mathbb{x}, (\tau_1, \dots, \tau_i), (\alpha_1, \dots, \alpha_i))$, where $\mathbb{x}$ is an instance, $(\alpha_1, \dots, \alpha_i)$ are IP prover messages, and $(\tau_1, \dots, \tau_i)$ are salt strings in $\{0, 1\}^{\delta_1} \times \cdots \times \{0, 1\}^{\delta_i}$.
 - (b) Set $\rho_i := f_i(\mathbb{x}, (\tau_1, \dots, \tau_i), (\alpha_1, \dots, \alpha_i))$.
 - (c) Send ρ_i to $\tilde{\mathbf{P}}^{\text{sr}}$.
2. $\tilde{\mathbf{P}}^{\text{sr}}$ outputs $(\mathbb{x}, (\tau_1, \dots, \tau_k), (\alpha_1, \dots, \alpha_k))$, where $\mathbb{x}$ is an instance, $(\alpha_1, \dots, \alpha_k)$ are IP prover messages, and $(\tau_1, \dots, \tau_k) \in \{0, 1\}^{\delta_1} \times \cdots \times \{0, 1\}^{\delta_k}$ are salt strings.
3. For every $i \in [k]$, set $\rho_i := f_i(\mathbb{x}, (\tau_1, \dots, \tau_i), (\alpha_1, \dots, \alpha_i))$.
4. Output $(\mathbb{x}, (\tau_i)_{i \in [k]}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]})$.

We denote by tr^{sr} the list of move-response pairs of the form $((\mathbb{x}, (\tau_1, \dots, \tau_i), (\alpha_1, \dots, \alpha_i)), \rho_i)$ performed in the loop. We show tr^{sr} in an execution of the IP state-restoration game using the following notation:

$$(\mathbb{x}, (\tau_i)_{i \in [k]}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) \xleftarrow{\text{tr}^{\text{sr}}} \text{SR}_{\text{IP}, \mathbf{f}, \tilde{\mathbf{P}}^{\text{sr}}}(\delta).$$

$\tilde{\mathbf{P}}^{\text{sr}}$ is **t -move** if $\tilde{\mathbf{P}}^{\text{sr}}$ exits the loop after at most t iterations.

The above game directly leads to the notion of state-restoration soundness error: it is an upper bound on the probability that any IP prover in the IP state-restoration game can find an instance not in the language and an accepting transcript for it.

Definition 3.13. $\text{IP} = (\mathbf{P}, \mathbf{V})$ has **state-restoration soundness error** $\varepsilon_{\text{IP}}^{\text{sr}}$ if for all salt sizes $\delta := (\delta_1, \dots, \delta_k) \in \mathbb{N}^k$, move budget $t \in \mathbb{N}$, t -move malicious IP state-restoration prover $\tilde{\mathbf{P}}^{\text{sr}}$, and instance size bound $n \in \mathbb{N}$:

$$\Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge \mathbb{x} \notin \mathcal{L}(\mathcal{R}) \\ \wedge \mathbf{V}(\mathbb{x}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) = 1 \end{array} \middle| \begin{array}{l} \mathbf{f} = (f_i)_{i \in [k]} \leftarrow \mathcal{U}((\mathcal{Z}_i(\delta_i, n))_{i=1}^k) \\ (\mathbb{x}, (\tau_i)_{i \in [k]}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) \leftarrow \text{SR}_{\text{IP}, \mathbf{f}, \tilde{\mathbf{P}}^{\text{sr}}}(\delta) \end{array} \right] \leq \varepsilon_{\text{IP}}^{\text{sr}}(\delta, t, n).$$

The straightline variant of state-restoration knowledge soundness considers a (deterministic) knowledge extractor $\mathbf{E}^{\text{sr}}$ that is tasked with finding a witness $\mathbf{w}$ while given the final output $(\mathbf{x}, (\tau_1, \dots, \tau_k), (\alpha_1, \dots, \alpha_k))$ of the state-restoration prover $\tilde{\mathbf{P}}^{\text{sr}}$ and its move-response trace tr^{sr} . The knowledge extractor $\mathbf{E}^{\text{sr}}$ does not receive the randomness $(\rho_i)_{i \in [k]}$ used by the IP verifier.

Definition 3.14. $\text{IP} = (\mathbf{P}, \mathbf{V})$ has **straightline state-restoration knowledge soundness error** $\kappa_{\text{IP}}^{\text{sr}}$ if there exists a polynomial-time deterministic algorithm $\mathbf{E}^{\text{sr}}$ (the extractor) such that for all salt sizes $\delta := (\delta_1, \dots, \delta_k) \in \mathbb{N}^k$, move budget $t \in \mathbb{N}$, t -move deterministic IP state-restoration prover $\tilde{\mathbf{P}}^{\text{sr}}$, and instance size bound n :

$$\Pr \left[\begin{array}{l} |\mathbf{x}| \leq n \\ \wedge (\mathbf{x}, \mathbf{w}) \notin \mathcal{R} \\ \wedge \mathbf{V}(\mathbf{x}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) = 1 \end{array} \middle| \begin{array}{l} \mathbf{f} = (f_i)_{i \in [k]} \leftarrow \mathcal{U}((\mathcal{Z}_i(\delta_i, n))_{i=1}^k) \\ (\mathbf{x}, (\tau_i)_{i \in [k]}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) \xleftarrow{\text{tr}^{\text{sr}}} \text{SR}_{\text{IP}, \mathbf{f}, \tilde{\mathbf{P}}^{\text{sr}}}(\delta) \\ \mathbf{w} \leftarrow \mathbf{E}^{\text{sr}}(\mathbf{x}, (\tau_i)_{i \in [k]}, (\alpha_i)_{i \in [k]}, \text{tr}^{\text{sr}}) \end{array} \right] \leq \kappa_{\text{IP}}^{\text{sr}}(\delta, t, n).$$

The rewinding variant of state-restoration knowledge soundness relaxes the prior notion by considering a knowledge extractor that additionally receives the randomness $(\rho_i)_{i \in [k]}$ used by the IP verifier and black-box access to the state-restoration prover $\tilde{\mathbf{P}}^{\text{sr}}$. In this case, the error may additionally depend on the failure probability of $\tilde{\mathbf{P}}^{\text{sr}}$ (an upper bound on the probability that the final output of $\tilde{\mathbf{P}}^{\text{sr}}$ does *not* convince the IP verifier). Intuitively, as the failure probability increases, the error of extraction increases.

Definition 3.15. Let $\text{IP} = (\mathbf{P}, \mathbf{V})$ be an IP. A deterministic IP state-restoration prover $\tilde{\mathbf{P}}^{\text{sr}}$ has **failure probability** $\delta_{\tilde{\mathbf{P}}^{\text{sr}}}$ if for all salt sizes $\delta := (\delta_1, \dots, \delta_k) \in \mathbb{N}^k$ and instance size bound n :

$$\Pr \left[\begin{array}{l} |\mathbf{x}| \leq n \\ \wedge \mathbf{V}(\mathbf{x}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) = 0 \end{array} \middle| \begin{array}{l} \mathbf{f} = (f_i)_{i \in [k]} \leftarrow \mathcal{U}((\mathcal{Z}_i(\delta_i, n))_{i=1}^k) \\ (\mathbf{x}, (\tau_i)_{i \in [k]}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) \leftarrow \text{SR}_{\text{IP}, \mathbf{f}, \tilde{\mathbf{P}}^{\text{sr}}}(\delta) \end{array} \right] \leq \delta_{\tilde{\mathbf{P}}^{\text{sr}}}(\delta, n).$$

Definition 3.16. $\text{IP} = (\mathbf{P}, \mathbf{V})$ has **rewinding state-restoration knowledge soundness error** $\kappa_{\text{IP}}^{\text{sr}}$ **with extraction time** $\text{et}_{\text{IP}}^{\text{sr}}$ if there exists a probabilistic algorithm $\mathbf{E}^{\text{sr}}$ (the extractor) such that for all salt sizes $\delta := (\delta_1, \dots, \delta_k) \in \mathbb{N}^k$, move budget $t \in \mathbb{N}$, t -move deterministic IP state-restoration prover $\tilde{\mathbf{P}}^{\text{sr}}$ with failure probability $\delta_{\tilde{\mathbf{P}}^{\text{sr}}}$ and running time $\tau_{\tilde{\mathbf{P}}^{\text{sr}}}$, and instance size bound n :

$$\Pr \left[\begin{array}{l} |\mathbf{x}| \leq n \\ \wedge (\mathbf{x}, \mathbf{w}) \notin \mathcal{R} \\ \wedge \mathbf{V}(\mathbf{x}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) = 1 \end{array} \middle| \begin{array}{l} \mathbf{f} = (f_i)_{i \in [k]} \leftarrow \mathcal{U}((\mathcal{Z}_i(\delta_i, n))_{i=1}^k) \\ (\mathbf{x}, (\tau_i)_{i \in [k]}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) \xleftarrow{\text{tr}^{\text{sr}}} \text{SR}_{\text{IP}, \mathbf{f}, \tilde{\mathbf{P}}^{\text{sr}}}(\delta) \\ \mathbf{w} \leftarrow \mathbf{E}^{\text{sr}}(\mathbf{x}, (\alpha_i)_{i \in [k]}, (\tau_i)_{i \in [k]}, (\rho_i)_{i \in [k]}, \text{tr}^{\text{sr}}, \tilde{\mathbf{P}}^{\text{sr}}) \end{array} \right] \leq \kappa_{\text{IP}}^{\text{sr}}(\delta, t, n, \delta_{\tilde{\mathbf{P}}^{\text{sr}}}(\delta, n)).$$

Moreover, $\mathbf{E}^{\text{sr}}$ runs in expected time $\text{et}_{\text{IP}}^{\text{sr}}(\delta, t, n, \delta_{\tilde{\mathbf{P}}^{\text{sr}}}(\delta, n), \tau_{\tilde{\mathbf{P}}^{\text{sr}}}(\delta, n))$ (over the given inputs and internal randomness).

3.6 Fiat–Shamir transformation

Let $\text{IP} = (\mathbf{P}, \mathbf{V})$ be a public-coin IP with round complexity k and message spaces $((\mathcal{M}_{\mathbf{P}, i}, \mathcal{M}_{\mathbf{V}, i}))_{i \in [k]}$. Define the oracle distribution $\mathcal{D}_{\text{IP}}$ with salt size δ as follows: letting $\delta_1 := \delta$ and $\delta_2, \dots, \delta_k := 0$, for every security parameter $\lambda \in \mathbb{N}$ and an instance size bound $n \in \mathbb{N}$,

$$\mathcal{D}_{\text{IP}}(\lambda, n) := \mathcal{U}(\mathcal{Z}_1(\delta_1, n), \dots, \mathcal{Z}_k(\delta_k, n)) \quad (4)$$

where $\mathcal{Z}_1(\delta_1, n), \dots, \mathcal{Z}_k(\delta_k, n)$ are the function spaces in Theorem 3.11. We present the canonical Fiat–Shamir transformation for public-coin interactive proofs. The construction here is a simplification of [CY24, Construction 14.1.1] that relies on a single salt rather than k salts. This simplification does not affect the soundness (and knowledge soundness) results given in that book.

Construction 3.17. Let $\delta \in \mathbb{N}$. We define $\text{NARG} := \text{FS}[\text{IP}, \delta]$ to be the non-interactive argument $\text{NARG} = (\mathcal{P}, \mathcal{V})$ in the $\mathcal{D}_{\text{IP}}$ -oracle model constructed as follows. The argument prover $\mathcal{P}$ receives as input an instance $\mathbb{x}$ and witness $\mathbb{w}$, and the argument verifier $\mathcal{V}$ receives as input the instance $\mathbb{x}$ and an argument string π . Both receive query access to oracles f sampled from $\mathcal{D}_{\text{IP}}(\lambda, n)$ defined in Equation 4.

- $\mathcal{P}^f(\mathbb{x}, \mathbb{w})$:

1. Sample a random salt $\tau \in \{0, 1\}^\delta$.
2. For $i = 1, \dots, k$:
 - (a) Compute the i -th message (and auxiliary state) of the IP prover:

$$(\alpha_i, \text{aux}_i) := \begin{cases} \mathbf{P}(\mathbb{x}, \mathbb{w}) & \text{if } i = 1 \\ \mathbf{P}(\text{aux}_{i-1}, \rho_{i-1}) & \text{if } i > 1 \end{cases}.$$

- (b) If $i < k$, derive the i -th random message of the IP verifier:

$$\rho_i := f_i(\mathbb{x}, \tau, \alpha_1, \dots, \alpha_i).$$

3. Output the argument string $\pi := (\tau, (\alpha_i)_{i \in [k]})$.

- $\mathcal{V}^f(\mathbb{x}, \pi)$:

1. Parse the argument string π as a tuple $(\tau, (\alpha_i)_{i \in [k]})$.
2. For $i = 1, \dots, k$, derive the i -th IP verifier message ρ_i (as $\mathcal{P}$ does):

$$\rho_i := f_i(\mathbb{x}, \tau, \alpha_1, \dots, \alpha_i).$$

3. Check that the IP verifier accepts: $\mathbf{V}(\mathbb{x}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) = 1$.

The canonical Fiat–Shamir transformation for public-coin IPs described above satisfies soundness and knowledge soundness, as we now explain. Theorem 3.18 below (adapted from [CY24]) links the soundness error of $\text{NARG} = (\mathcal{P}, \mathcal{V})$ to the state-restoration soundness error of $\text{IP} = (\mathbf{P}, \mathbf{V})$; similarly, Theorem 3.19 (also adapted from [CY24]) does the same for knowledge soundness. (For ease of notation, below we use the salt size $\delta \in \mathbb{N}$ rather than the salt size vector $\boldsymbol{\delta} := (\delta_1, \dots, \delta_k) \in \mathbb{N}^k$ where $\delta_1 = \delta$ and $\delta_2, \dots, \delta_k = 0$.)

Theorem 3.18. If $\text{IP} = (\mathbf{P}, \mathbf{V})$ has state-restoration soundness error $\varepsilon_{\text{IP}}^{\text{sr}}$ (see Theorem 3.14) then $\text{NARG} := \text{FS}[\text{IP}, \delta]$ in Theorem 3.17 has soundness error $\varepsilon_{\text{NARG}}$ (see Theorem 3.5) such that, for every security parameter $\lambda \in \mathbb{N}$, query bound $t \in \mathbb{N}$, and instance size bound $n \in \mathbb{N}$,

$$\varepsilon_{\text{NARG}}(\lambda, t, n) \leq \varepsilon_{\text{IP}}^{\text{sr}}(\delta, t, n).$$

Theorem 3.19. If IP has rewinding state-restoration knowledge soundness error $\kappa_{\text{IP}}^{\text{sr}}$ with extraction time et_{IP} (see Theorem 3.16) then $\text{NARG} := \text{FS}[\text{IP}, \delta]$ in Theorem 3.17 has rewinding knowledge soundness error $\kappa_{\text{NARG}}(\lambda, t, n, \delta_{\tilde{\mathcal{P}}}(\lambda, n))$ with extraction time et_{NARG} (see Theorem 3.8) such that, for every security parameter $\lambda \in \mathbb{N}$, query bound $t \in \mathbb{N}$, and instance size bound $n \in \mathbb{N}$,

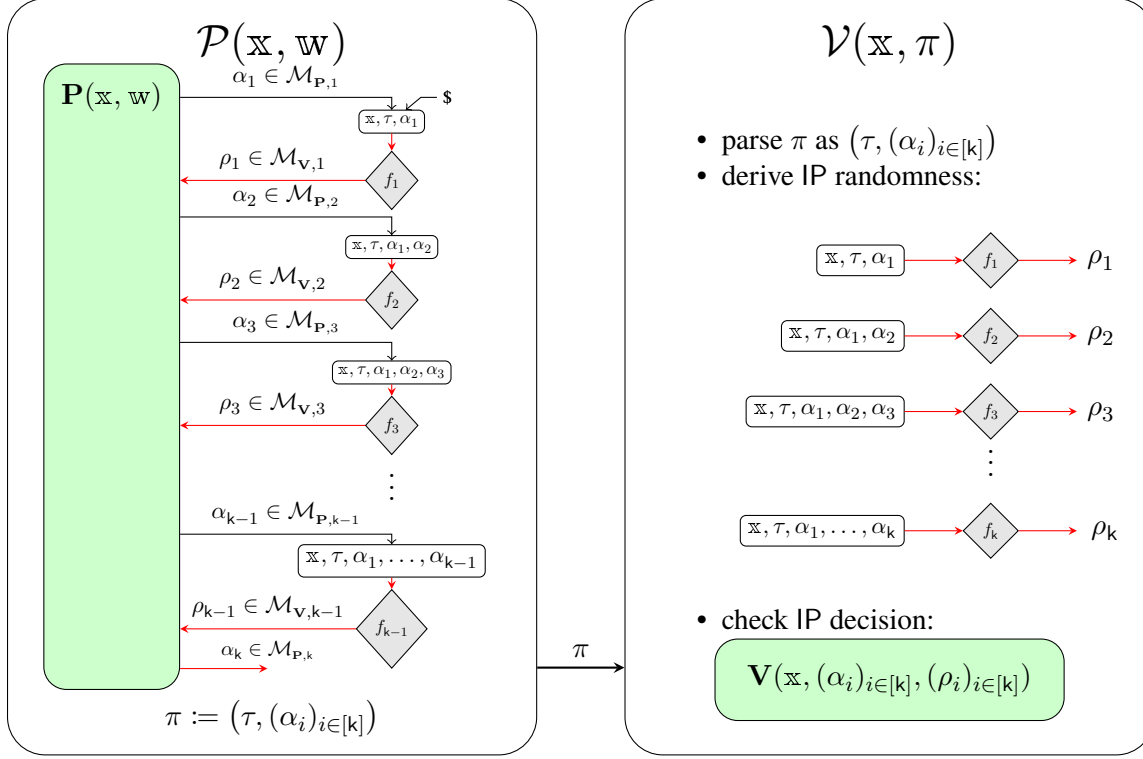


Figure 2: Diagram of $\text{FS}[\text{IP}, \delta]$ as defined in Theorem 3.17.

- $\kappa_{\text{NARG}}(\lambda, t, n, \delta_{\tilde{\mathcal{P}}}(\lambda, n)) \leq \kappa_{\text{IP}}^{\text{SR}}(\delta, t, n, \delta_{\tilde{\mathcal{P}}}(\lambda, n))$, and
 - $\text{et}_{\text{NARG}}(\lambda, t, n, \delta_{\tilde{\mathcal{P}}}(\lambda, n), \tau_{\tilde{\mathcal{P}}}(\lambda, n)) \leq \text{et}_{\text{IP}}^{\text{SR}}(\delta, t, n, \delta_{\tilde{\mathcal{P}}}(\lambda, n), \tau_{\tilde{\mathcal{P}}}(\lambda, n) + O(r_{\max} \cdot t)) + O(r_{\max} \cdot t)$.
- Moreover, if the IP state-restoration extractor is straightline (see Theorem 3.14) then the NARG extractor is also straightline (see Theorem 3.6). In this case:
- the (straightline) knowledge soundness error is $\kappa_{\text{NARG}}(\lambda, t, n) \leq \kappa_{\text{IP}}^{\text{SR}}(\delta, t, n)$; and
 - the (straightline) extraction time is $\text{et}_{\text{NARG}}(\lambda, t, n) \leq \text{et}_{\text{IP}}^{\text{SR}}(\delta, t, n) + O(r_{\max} \cdot t)$.
- Above $r_{\max} := \max_{i \in [k]} \log_2 |\mathcal{M}_{\mathbf{V},i}|$ denotes the maximum verifier randomness length.

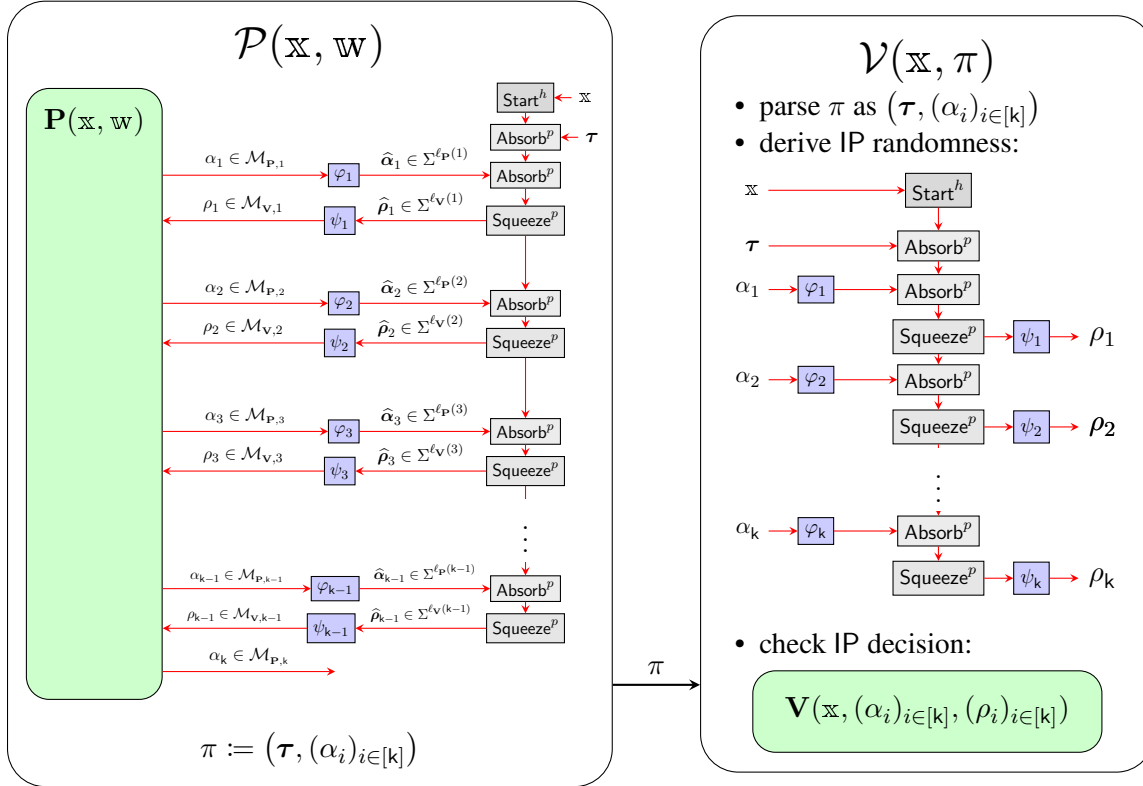


Figure 3: Diagram of **DSFS**[IP, δ] as defined in Theorem 4.3.

4 Duplex-sponge Fiat–Shamir transformation

We describe the Fiat–Shamir transformation that we propose in this paper; the transformation is based on the (ideal) duplex sponge described in Section 3.3. The main security reduction for the transformation is in Section 5, from which in Section 6 we deduce soundness and knowledge soundness. Separately, in Section 7, we establish the zero knowledge property for the transformation.

First we give the definition of a *codec* which is a way to encode prover messages and decode verifier messages relative to a given alphabet Σ .

Definition 4.1. Let Σ be a finite alphabet and $\text{IP} = (\mathbf{P}, \mathbf{V})$ be an IP with round complexity k and message spaces $((\mathcal{M}_{\mathbf{P},i}, \mathcal{M}_{\mathbf{V},i}))_{i \in [k]}$. A **codec** for IP over Σ with bias ε_{cdc} is a function cdc that maps every security parameter $\lambda \in \mathbb{N}$ and instance size bound $n \in \mathbb{N}$ to a tuple

$$\text{cdc}(\lambda, n) = (\ell_{\mathbf{P}}, \ell_{\mathbf{V}}, \varphi, \psi)$$

where

- $\ell_{\mathbf{P}}, \ell_{\mathbf{V}}: \mathbb{N} \rightarrow \mathbb{N}$ are length functions for each round,
- φ is a list of injective maps $(\varphi_i: \mathcal{M}_{\mathbf{P},i} \rightarrow \Sigma^{\ell_{\mathbf{P}}(i)})_{i \in [k]}$, and
- ψ is a list of maps $(\psi_i: \Sigma^{\ell_{\mathbf{V}}(i)} \rightarrow \mathcal{M}_{\mathbf{V},i})_{i \in [k]}$ where, for each $i \in [k]$, ψ_i is $\varepsilon_{\text{cdc},i}(\lambda, n)$ -biased (the distributions $\psi_i(\mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)}))$ and $\mathcal{U}(\mathcal{M}_{\mathbf{V},i})$ are $\varepsilon_{\text{cdc},i}(\lambda, n)$ -close in statistical distance).¹¹

Next we give the definition of the oracle distribution $\mathcal{D}_{\mathfrak{S}}(\lambda, n)$ that we consider.

Definition 4.2. The **ideal permutation oracle distribution** $\mathcal{D}_{\mathfrak{S}}$ over alphabet Σ with capacity $c \in \mathbb{N}$, and rate $r \in \mathbb{N}$ is defined as follows: $\mathcal{D}_{\mathfrak{S}}(\lambda, n)$ outputs (h, p, p^{-1}) where $h: \{0, 1\}^{\leq n} \rightarrow \Sigma^c$ is a random function, $p: \Sigma^{r+c} \rightarrow \Sigma^{r+c}$ is a random permutation, and p^{-1} is the inverse of p .

Finally, we describe our Fiat–Shamir transformation. The inverse permutation p^{-1} will only be used by the adversary, so it does not appear in the construction.

Construction 4.3. Let $\text{IP} = (\mathbf{P}, \mathbf{V})$ be a public-coin IP with round complexity k . Let Σ be a finite alphabet and cdc a codec for IP over Σ (the codec’s bias does not matter for describing the construction). Let $\mathcal{D}_{\mathfrak{S}}$ be the ideal permutation oracle distribution over Σ with capacity $c \in \mathbb{N}$ and rate $r \in \mathbb{N}$. For a salt size $\delta \in \mathbb{N}$, the non-interactive argument $\text{NARG} = \text{DSFS}[\text{IP}, \text{cdc}, \delta]$ in the $\mathcal{D}_{\mathfrak{S}}$ -oracle model is the non-interactive argument $\text{NARG} = (\mathcal{P}, \mathcal{V})$ constructed as follows. The argument prover $\mathcal{P}$ receives as input an instance $\mathfrak{x}$ and witness $\mathfrak{w}$, and the argument verifier $\mathcal{V}$ receives as input the instance $\mathfrak{x}$ and an argument string π . Both receive query access to oracles (h, p) sampled from $\mathcal{D}_{\mathfrak{S}}(\lambda, n)$ defined in Theorem 4.2. Let $(\ell_{\mathbf{P}}, \ell_{\mathbf{V}}, \varphi, \psi) := \text{cdc}(\lambda, n)$.

- $\mathcal{P}^{h,p}(\mathfrak{x}, \mathfrak{w})$:

1. Initialize the sponge state with the instance: $\text{st}'_0 := \text{DS.Start}^h(\mathfrak{x}) \in \Sigma^{r+c} \times [0, r] \times [0, r]$.
2. Sample a random salt $\tau \in \Sigma^{\delta}$.
3. Absorb the salt: $\text{st}_0 := \text{Absorb}^p(\text{st}'_0, \tau) \in \Sigma^{r+c} \times [0, r] \times [0, r]$.
4. For $i = 1, \dots, k$:
 - (a) Compute the i -th message (and auxiliary state) of the IP prover:

$$(\alpha_i, \text{aux}_i) := \begin{cases} \mathbf{P}(\mathfrak{x}, \mathfrak{w}) \in \mathcal{M}_{\mathbf{P},1} & \text{if } i = 1 \\ \mathbf{P}(\text{aux}_{i-1}, \rho_{i-1}) \in \mathcal{M}_{\mathbf{P},i} & \text{if } i > 1 \end{cases}.$$

¹¹I.e., ψ_i maps the uniform distribution on $\Sigma^{\ell_{\mathbf{V}}(i)}$ to a distribution that is $\varepsilon_{\text{cdc},i}(\lambda, n)$ -close to the uniform distribution on $\mathcal{M}_{\mathbf{V},i}$.

(b) If $i < k$, encode the prover message: $\hat{\alpha}_i := \varphi_i(\alpha_i) \in \Sigma^{\ell_{\mathbf{P}}(i)}$.

(c) If $i < k$, absorb the encoded prover message:

$$\text{st}'_i := \text{DS.Absorb}^p(\text{st}_{i-1}, \hat{\alpha}_i) \in \Sigma^{r+c} \times [0, r] \times [0, r].$$

(d) If $i < k$, squeeze the encoded verifier message:

$$(\hat{\rho}_i, \text{st}_i) := \text{DS.Squeeze}^p(\text{st}'_i, \ell_{\mathbf{V}}(i)) \in \Sigma^{\ell_{\mathbf{V}}(i)} \times \Sigma^{r+c} \times [0, r] \times [0, r].$$

(e) If $i < k$, decode the verifier message: $\rho_i := \psi_i(\hat{\rho}_i) \in \mathcal{M}_{\mathbf{V},i}$.

5. Output the argument string $\pi := (\tau, (\alpha_i)_{i \in [k]})$.

• $\mathcal{V}^{h,p}(\mathbb{x}, \pi)$:

1. Parse the argument string π as a salt $\tau \in \Sigma^\delta$ and prover messages $(\alpha_i)_{i \in [k]} \in \mathcal{M}_{\mathbf{P},1} \times \dots \times \mathcal{M}_{\mathbf{P},k}$.
2. Initialize the sponge state with the instance (as $\mathcal{P}$ does): $\text{st}'_0 := \text{DS.Start}^h(\mathbb{x}) \in \Sigma^{r+c} \times [0, r] \times [0, r]$.
3. Absorb the salt (as $\mathcal{P}$ does): $\text{st}_0 := \text{Absorb}^p(\text{st}'_0, \tau) \in \Sigma^{r+c} \times [0, r] \times [0, r]$.
4. For $i = 1, \dots, k$, derive the i -th IP verifier message ρ_i (as $\mathcal{P}$ does):

(a) Encode the prover message: $\hat{\alpha}_i := \varphi_i(\alpha_i) \in \Sigma^{\ell_{\mathbf{P}}(i)}$.

(b) Absorb the encoded prover message:

$$\text{st}'_i := \text{DS.Absorb}^p(\text{st}_{i-1}, \hat{\alpha}_i) \in \Sigma^{r+c} \times [0, r] \times [0, r].$$

(c) Squeeze the encoded verifier message:

$$(\hat{\rho}_i, \text{st}_i) := \text{DS.Squeeze}^p(\text{st}'_i, \ell_{\mathbf{V}}(i)) \in \Sigma^{\ell_{\mathbf{V}}(i)} \times \Sigma^{r+c} \times [0, r] \times [0, r].$$

(d) Decode the verifier message: $\rho_i := \psi_i(\hat{\rho}_i) \in \mathcal{M}_{\mathbf{V},i}$.

5. Check that the IP verifier accepts: $\mathbf{V}(\mathbb{x}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) = 1$.

Efficiency. We discuss efficiency properties of the above constructions.

- *Argument size.* The argument string π contains a salt $\tau \in \Sigma^\delta$ and all IP prover messages (and none of the IP verifier messages). Hence the number of bits in π is

$$\text{len}(\tau) + \sum_{i \in [k]} \text{len}(\alpha_i) = \delta \cdot \log_2 |\Sigma| + \sum_{i \in [k]} \log_2 |\mathcal{M}_{\mathbf{P},i}|.$$

- *Prover complexity.* The cost of the argument prover $\mathcal{P}$ is essentially the same as that of the underlying IP prover $\mathbf{P}$. The difference is that the argument prover $\mathcal{P}$ additionally:
 - makes 1 query of length n to the oracle h , in the call to DS.Start ;
 - makes $\left\lceil \frac{\delta + \ell_{\mathbf{P}}(1)}{r} \right\rceil + \sum_{i=2}^{k-1} \left\lceil \frac{\ell_{\mathbf{P}}(i)}{r} \right\rceil$ queries of length $r + c$ to the oracle p , across calls to DS.Absorb ;
 - makes $\sum_{i=1}^{k-1} \left\lceil \frac{\ell_{\mathbf{V}}(i)}{r} \right\rceil$ queries of length $r + c$ to the oracle p , across calls to DS.Squeeze ;
 - invokes once each of $(\varphi_i)_{i \in [k-1]}$ and $(\psi_i)_{i \in [k-1]}$.
- *Verifier complexity.* The cost of the argument verifier $\mathcal{V}$ is essentially the same as that of the underlying IP verifier $\mathbf{V}$. The difference is that the argument verifier $\mathcal{V}$ additionally:
 - makes the same queries to h and p as $\mathcal{P}$ does;
 - makes $\left\lceil \frac{\ell_{\mathbf{P}}(k)}{r} \right\rceil$ queries of length $r + c$ to the oracle p , due to another call to DS.Absorb ;
 - makes $\left\lceil \frac{\ell_{\mathbf{V}}(k)}{r} \right\rceil$ queries of length $r + c$ to the oracle p , due to another call to DS.Squeeze ;
 - invokes once each of $(\varphi_i)_{i \in [k]}$ and $(\psi_i)_{i \in [k]}$.

5 Security analysis

We provide the main security reduction in this paper: we reduce the security of the duplex-sponge Fiat–Shamir transformation to the security of the basic Fiat–Shamir transformation. In Section 6 we show that this directly implies the soundness and knowledge soundness of the duplex-sponge Fiat–Shamir transformation. This fits the template of *extraction-friendly indifferenciability* that we discuss in Section A.2.2 (see Theorem A.7).

Let $\text{IP} = (\mathbf{P}, \mathbf{V})$ be a public-coin IP for a relation $\mathcal{R}$ with round complexity k . Let Σ be a finite alphabet and cdc a codec for IP over Σ with bias ε_{cdc} (see Theorem 4.1). Let $\mathcal{D}_{\Sigma}$ be an ideal permutation oracle distribution over Σ with capacity $c \in \mathbb{N}$ and rate $r \in \mathbb{N}$ (see Theorem 4.2). Let $\delta \in \mathbb{N}$ be a salt size (in Σ -elements) and $\delta_{\star} := \delta \log_2 |\Sigma|$ be its corresponding bit-size.

Consider the following two non-interactive arguments:

- $(\mathcal{P}, \mathcal{V}) := \mathbf{DSFS}[\text{IP}, \text{cdc}, \delta]$ is the non-interactive argument in the $\mathcal{D}_{\Sigma}$ -oracle model in Theorem 4.3;
- $(\mathcal{P}_{\text{std}}, \mathcal{V}_{\text{std}}) := \mathbf{FS}[\text{IP}, \delta_{\star}]$ is the non-interactive argument in the $\mathcal{D}_{\text{IP}}$ -oracle model in Theorem 3.17.

We reduce the security of $(\mathcal{P}, \mathcal{V})$ to the security of $(\mathcal{P}_{\text{std}}, \mathcal{V}_{\text{std}})$ via Theorem 5.1 below. Informally, Theorem 5.1 shows that any $(t_h, t_p, t_{p^{-1}})$ -query malicious argument prover $\tilde{\mathcal{P}}$ for $\mathcal{V}$ can be transformed, in a black-box way via an auxiliary procedure D2SAlgo defined in Section 5.4, into a $\theta_{\star}(t_h, t_p, t_{p^{-1}})$ -query malicious argument prover $\tilde{\mathcal{P}}_{\text{std}}$ for $\mathcal{V}_{\text{std}}$ that “behaves the same” as $\mathcal{P}$ up to an additive error $\eta_{\star}(\lambda, (t_h, t_p, t_{p^{-1}}))$; both $\theta_{\star}(t_h, t_p, t_{p^{-1}})$ and $\eta_{\star}(\lambda, (t_h, t_p, t_{p^{-1}}))$ are defined below. Specifically, the output instance, argument string, and convincing probability are preserved, as are the query-answer traces, after an appropriate transformation by an auxiliary procedure D2STrace to account for the differing constructions.

Lemma 5.1. *There exist algorithms D2SAlgo and D2STrace such that the following holds: for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, and $(t_h, t_p, t_{p^{-1}})$ -query argument prover $\tilde{\mathcal{P}}$, the two distributions below*

$$\begin{array}{ll}
 (h, p, p^{-1}) \leftarrow \mathcal{D}_{\Sigma}(\lambda, n) & \mathbf{f} \leftarrow \mathcal{D}_{\text{IP}}(\lambda, n) \\
 (\mathbb{X}, \pi) \xleftarrow{\text{tr}_{\tilde{\mathcal{P}}}} \tilde{\mathcal{P}}^{h, p, p^{-1}} & (\mathbb{X}, \pi) \xleftarrow{\text{tr}_{\tilde{\mathcal{P}}_{\star}}} \text{D2SAlgo}^{\mathbf{f}}(\tilde{\mathcal{P}}) \\
 b \xleftarrow{\text{tr}_{\mathcal{V}}} \mathcal{V}^{h, p}(\mathbb{X}, \pi) & b \xleftarrow{\text{tr}_{\mathcal{V}_{\text{std}}}} \mathcal{V}_{\text{std}}^{\mathbf{f}}(\mathbb{X}, \pi) \\
 \text{tr} := \text{D2STrace}(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}}) & \text{tr} := \text{tr}_{\tilde{\mathcal{P}}_{\star}} \| \text{tr}_{\mathcal{V}_{\text{std}}} \\
 \text{return } (b, \mathbb{X}, \pi, \text{tr}) & \text{return } (b, \mathbb{X}, \pi, \text{tr})
 \end{array}$$

have statistical distance at most:

$$\begin{aligned}
 \eta_{\star}(\lambda, (t_h, t_p, t_{p^{-1}})) := & \\
 & \frac{7(t_h + t_p + t_{p^{-1}})^2 + 28(L+1)(t_h + t_p + t_{p^{-1}}) + 14(L+1)^2 - 3(t_h + t_p + t_{p^{-1}}) - 13(L+1)}{2|\Sigma|^c} + \quad (5) \\
 & \theta_{\star}(t_h, t_p, t_{p^{-1}}) \cdot \max_{i \in [k]} \varepsilon_{\text{cdc}, i}(\lambda, n) + \sum_{i \in [k]} \varepsilon_{\text{cdc}, i}(\lambda, n).
 \end{aligned}$$

where

$$\theta_{\star}(t_h, t_p, t_{p^{-1}}) := t_p \quad \text{and} \quad t_p \geq \max \left\{ \sum_{i \in [k]} \left\lceil \frac{\ell_{\mathbf{P}}(i)}{r} \right\rceil, \sum_{i \in [k]} \left\lceil \frac{\ell_{\mathbf{V}}(i)}{r} \right\rceil \right\}. \quad (6)$$

Moreover, $\text{D2SAlgo}(\tilde{\mathcal{P}})$ is a $\theta_{\star}(t_h, t_p, t_{p^{-1}})$ -query algorithm.

We prove the lemma in Section 5.8. Before that, we define the two algorithms D2SAlgo and D2STrace in Sections 5.4 and 5.5. In turn, these rely on the procedures BackTrack and LookAhead defined in Sections 5.2 and 5.3. We study probabilities for bad events and for aborts in Sections 5.6 and 5.7 respectively.

Throughout, we use the following notation (for various block numbers):

$$L_\delta := \lceil \delta/r \rceil \quad , \quad L_P(i) := \lceil \ell_P(i)/r \rceil \quad , \quad L_V(i) := \lceil \ell_V(i)/r \rceil \quad , \quad (7)$$

and

$$L_P := \sum_{i \in [k]} L_P(i) \quad , \quad L_V := \sum_{i \in [k]} L_V(i) \quad , \quad L := L_P + L_V \quad . \quad (8)$$

Multiple times we use the fact that the query-answer trace tr_V of an execution of the argument verifier $\mathcal{V}^{h,p}$ contains exactly 1 query to h , L_P queries to p for absorbing all the IP prover messages, and L_V queries to p for squeezing all the IP verifier messages (for a total of L queries to p).

5.1 Data structure for query-answer trace

Definition 5.2. Let $h: \{0, 1\}^{\leq n} \rightarrow \Sigma^c$, $p: \Sigma^{r+c} \rightarrow \Sigma^{r+c}$, and $p^{-1}: \Sigma^{r+c} \rightarrow \Sigma^{r+c}$ be functions. A (h, p, p^{-1}) -trace tr is a list of tuples of the following form:

- $(h', \mathbb{x}, s_C)$ to denote a query $\mathbb{x} \in \{0, 1\}^{\leq n}$ to h with answer $s_C \in \Sigma^c$,
- $(p', s_{\text{in}}, s_{\text{out}})$ to denote a query $s_{\text{in}} \in \Sigma^{r+c}$ to p with answer $s_{\text{out}} \in \Sigma^{r+c}$,
- $(p^{-1}', s_{\text{out}}, s_{\text{in}})$ to denote a query $s_{\text{out}} \in \Sigma^{r+c}$ to p^{-1} with answer $s_{\text{in}} \in \Sigma^{r+c}$.

We use “.” as a placeholder for any variable matching the above patterns. For a (h, p, p^{-1}) -trace tr , we let $\text{tr}_h^{<j}$ be the first $j - 1$ entries in tr of the form $(h', \cdot, \cdot)$, $\text{tr}_p^{<j}$ be the first $j - 1$ entries in tr of the form $(p', \cdot, \cdot)$, and $\text{tr}_{p^{-1}}^{<j}$ be the first $j - 1$ entries in tr of the form $(p^{-1}', \cdot, \cdot)$.

Sorted query-answer trace. We describe a data structure tr_∇ that (efficiently) supports searching for queries or answers in (h, p, p^{-1}) -traces. We use this data structure in D2SAlgo and D2STrace.

- $\text{tr}_\nabla.h$ supports efficient insertions and lookups of query-answer pairs for h .
 - $\text{tr}_\nabla.h.\text{add}(\mathbb{x}, s_C)$ adds the query-answer pair $(\mathbb{x}, s_C) \in \{0, 1\}^{\leq n} \times \Sigma^c$ to the data structure.
 - $\text{tr}_\nabla.h.\text{inlu}(\mathbb{x})$ returns the answer stored for the query $\mathbb{x}$ (or $\perp$ if there are zero or multiple matches).
 - $\text{tr}_\nabla.h.\text{outlu}(s_C)$ returns the query stored for the answer s_C (or $\perp$ if there are zero or multiple matches).

We write $\text{tr}_\nabla.h$ to denote the entries sorted by insertion time, and by $|\text{tr}_\nabla.h|$ the number of entries. These methods can be efficiently performed by maintaining three lists, sorted (respectively) by insertion time, by input query, and by answer. An insertion or lookup takes time $O(\log |\text{tr}_\nabla.h| \cdot c \cdot \log |\Sigma| + n)$.

- $\text{tr}_\nabla.p$ supports efficient insertions and lookups of query-answer pairs for p (equivalently, p^{-1}).
 - $\text{tr}_\nabla.p.\text{add}(s_{\text{in}}, s_{\text{out}})$ adds the query-answer pair $(s_{\text{in}}, s_{\text{out}}) \in \Sigma^{r+c} \times \Sigma^{r+c}$ to the data structure.
 - $\text{tr}_\nabla.p.\text{inlu}(s_{\text{in}})$ returns the answer stored for the query s_{in} (or $\perp$ if there are zero or multiple matches).
 - $\text{tr}_\nabla.p.\text{outlu}(s_{\text{out}})$ returns the query stored for the answer s_{out} (or $\perp$ if there are zero or multiple matches).

We write $\text{tr}_\nabla.p$ to denote the entries sorted by insertion time, and by $|\text{tr}_\nabla.p|$ the number of entries. These methods can be efficiently performed by maintaining three lists, sorted (respectively) by insertion time, by input query (capacity segment, then rate segment), and by answer (capacity segment, then rate segment). An insertion or lookup takes time $O(\log |\text{tr}_\nabla.p| \cdot (r + c) \cdot \log |\Sigma| + (r + c) \cdot \log |\Sigma|)$.

Sorting implies, in particular, that the permutation alphabet Σ has an order over its elements.

5.2 Backtracking procedure

We define the sequences of queries in a given query-answer trace tr that led to the capacity segment $\mathbf{s}_C$ in a given permutation state $\mathbf{s} = (\mathbf{s}_R, \mathbf{s}_C) \in \Sigma^{r+c}$, and then formally describe BackTrack.

Definition 5.3. Let $\mathcal{S}_{\text{BT}}(\text{tr}, \mathbf{s}) = \left\{ S^{(k)} \right\}_k$ be the set of sequences where each sequence

$$S^{(k)} := (\mathbb{x}^{(k)}, \mathbf{s}_{\text{in},0}^{(k)}, \mathbf{s}_{\text{out},0}^{(k)}, \mathbf{s}_{\text{in},1}^{(k)}, \mathbf{s}_{\text{out},1}^{(k)}, \dots, \mathbf{s}_{\text{in},m_k-1}^{(k)}, \mathbf{s}_{\text{out},m_k-1}^{(k)}, \mathbf{s}_{\text{in},m_k}^{(k)}) \quad (9)$$

is such that:

- $\mathbf{s}_{\text{in},m_k}^{(k)} = \mathbf{s}$ (the last permutation state is the given one);
- $(\text{'h'}, \mathbb{x}^{(k)}, \mathbf{s}_{\text{C},\text{in},0}^{(k)}) \in \text{tr}$;
- $\forall \iota \in [0, m_k-1]$, $(\text{'p'}, \mathbf{s}_{\text{in},\iota}^{(k)}, \mathbf{s}_{\text{out},\iota}^{(k)}) \in \text{tr} \vee (\text{'p'}^{-1}, \mathbf{s}_{\text{out},\iota}^{(k)}, \mathbf{s}_{\text{in},\iota}^{(k)}) \in \text{tr}$ (input-output states agree with p);
- $\forall \iota \in [0, m_k-1]$, $\mathbf{s}_{\text{C},\text{out},\iota}^{(k)} = \mathbf{s}_{\text{C},\text{in},\iota+1}^{(k)}$ (the capacity segment is shared across queries);
- $\forall \iota \in [0, m_k-1]$, $\mathbf{s}_{\text{C},\text{in},\iota}^{(k)} \neq \mathbf{s}_{\text{C},\text{out},\iota}^{(k)}$ (no “loops” across query and answer capacity segments);
- the length of the sequence is maximal (no other sequence in $\mathcal{S}_{\text{BT}}(\text{tr}, \mathbf{s})$ is strictly contained in $S^{(k)}$).

Definition 5.4. Let $\mathcal{J}_{\text{BT}}(\text{tr}, \mathbf{s}) = \left\{ J^{(k)} \right\}_k$ be the set of index lists associated to $\mathcal{S}_{\text{BT}}(\text{tr}, \mathbf{s}) = \left\{ S^{(k)} \right\}_k$, i.e.,

$$J^{(k)} := (j_h^{(k)}, j_0^{(k)}, j_1^{(k)}, \dots, j_{m_k}^{(k)}) \quad (10)$$

where for every k :

- $j_h^{(k)}$ is the index in tr of the first occurrence of $(\text{'h'}, \mathbb{x}^{(k)}, \mathbf{s}_{\text{C},\text{in},0}^{(k)})$;
- $\forall \iota \in [0, m_k-1]$, $j_\iota^{(k)}$ is the index in tr of the first occurrence of $(\text{'p'}, \mathbf{s}_{\text{in},\iota}^{(k)}, \mathbf{s}_{\text{out},\iota}^{(k)})$ or $(\text{'p'}^{-1}, \mathbf{s}_{\text{out},\iota}^{(k)}, \mathbf{s}_{\text{in},\iota}^{(k)})$;
- $j_{m_k}^{(k)} = |\text{tr}| + 1$ (the last index is a convenience value associated to an upcoming query with input $\mathbf{s}$).

The backtracking procedure $\text{BackTrack}(\text{tr}, \text{tr}_\nabla, \mathbf{s})$

- takes as input a (h, p, p^{-1}) -trace tr , associated data structure tr_∇ (see Section 5.1), and permutation state $\mathbf{s}$, and
- returns a special symbol (err or none) or a tuple $(i, \mathbb{x}, \tau, (\hat{\alpha}_1, \dots, \hat{\alpha}_i))$ with a round index $i \in [k]$, an instance $\mathbb{x} \in \{0, 1\}^n$, a salt string $\tau \in \Sigma^\delta$, and a tuple $\hat{\alpha} = (\hat{\alpha}_1, \dots, \hat{\alpha}_i) \in \Sigma^{\ell_P(1)} \times \dots \times \Sigma^{\ell_P(i)}$ of encoded prover messages.

Informally, BackTrack uses the trace to recover from the permutation state $\mathbf{s}$ the list of elements absorbed so far, returning the current round index i , absorbed instance $\mathbb{x}$, salt string τ , and encoded prover messages $\hat{\alpha}$.

$\text{BackTrack}(\text{tr}, \text{tr}_\nabla, \mathbf{s})$:

1. Initialize an empty list $\text{Outs} := []$.
2. Compute the set $\mathcal{S}_{\text{BT}} := \mathcal{S}_{\text{BT}}(\text{tr}, \mathbf{s})$ according to Theorem 5.3:

$$\mathcal{S}_{\text{BT}} := \left\{ S^{(k)} \right\}_k = \left\{ (\mathbb{x}^{(k)}, \mathbf{s}_{\text{in},0}^{(k)}, \mathbf{s}_{\text{out},0}^{(k)}, \mathbf{s}_{\text{in},1}^{(k)}, \mathbf{s}_{\text{out},1}^{(k)}, \dots, \mathbf{s}_{\text{in},m_k-1}^{(k)}, \mathbf{s}_{\text{out},m_k-1}^{(k)}, \mathbf{s}_{\text{in},m_k}^{(k)}) \right\}_k. \quad (11)$$

3. For every $k \in [|\mathcal{S}_{\text{BT}}|]$, assemble a candidate salt

$$\tau^{(k)} := (\mathbf{s}_{\text{R},\text{in},0}^{(k)} \parallel \dots \parallel \mathbf{s}_{\text{R},\text{in},L_\delta-1}^{(k)})[0 : \delta] \in \Sigma^\delta.$$

If $\delta > r$ (which implies $L_\delta > 1$), check that $\mathbf{s}_{\text{R},\text{in},L_\delta-1}^{(k)}[\delta \bmod r : r] = \mathbf{s}_{\text{R},\text{out},L_\delta-2}^{(k)}[\delta \bmod r : r]$.

4. For every $k \in [|\mathcal{S}_{\text{BT}}|]$, assemble a candidate list of encoded prover messages.

(a) For every $i \in [k]$:

- i. Let $L_{\text{ptr}}(i) := L_\delta + \sum_{\iota < i} (L_{\text{P}}(\iota) + L_{\text{V}}(\iota))$ be the current “block offset”.
- ii. If $L_{\text{ptr}}(i) + L_{\text{P}}(i) > m_k + 1$, then the k -th candidate is invalid.
Remove the k -th element from $\mathcal{S}_{\text{BT}}$ and continue to the next element in $\mathcal{S}_{\text{BT}}$ (if any).
- iii. Else (if $L_{\text{ptr}}(i) + L_{\text{P}}(i) \leq m_k + 1$):
A. Use the next $L_{\text{P}}(i)$ rate segments to define

$$\hat{\alpha}_i^{(k)} := (\mathbf{s}_{\text{R},\text{in},L_{\text{ptr}}(i)}^{(k)} \parallel \mathbf{s}_{\text{R},\text{in},L_{\text{ptr}}(i)+1}^{(k)} \parallel \cdots \parallel \mathbf{s}_{\text{R},\text{in},L_{\text{ptr}}(i)+L_{\text{P}}(i)-1}^{(k)})[0 : \ell_{\text{P}}(i)] \in \Sigma^{\ell_{\text{P}}(i)}. \quad (12)$$

B. Define $\mathbf{z}_i^{(k)} := \mathbf{s}_{\text{R},\text{in},L_{\text{ptr}}(i)+L_{\text{P}}(i)}^{(k)}[\ell_{\text{P}}(i) \bmod r : r]$.

C. Check that the remainder of the message is equal to the previous output:

$$\mathbf{z}_i^{(k)} = \mathbf{s}_{\text{R},\text{out},L_{\text{ptr}}(i)+L_{\text{P}}(i)-1}^{(k)}[\ell_{\text{P}}(i) \bmod r : r].$$

If the above does not hold, remove the k -th element from $\mathcal{S}_{\text{BT}}$ and continue to the next element in $\mathcal{S}_{\text{BT}}$ (if any).

D. If $L_{\text{ptr}}(i) + L_{\text{P}}(i) = m_k + 1$, store $(i, \mathbb{x}^{(k)}, \boldsymbol{\tau}^{(k)}, (\hat{\alpha}_1^{(k)}, \dots, \hat{\alpha}_i^{(k)}))$ into Outs and continue to the next element in $\mathcal{S}_{\text{BT}}$.

E. If $L_{\text{ptr}}(i) + L_{\text{P}}(i) + L_{\text{V}}(i) < m_k + 1$, check that the verifier message was squeezed correctly by checking that the rate part is preserved across invocations of the permutation function (to match the overwrite mode of the sponge):

$$\begin{aligned} & \mathbf{s}_{\text{R},\text{in},L_{\text{ptr}}(i)+L_{\text{P}}(i)+1}^{(k)} \parallel \cdots \parallel \mathbf{s}_{\text{R},\text{in},L_{\text{ptr}}(i)+L_{\text{P}}(i)+L_{\text{V}}(i)}^{(k)} \\ &= \mathbf{s}_{\text{R},\text{out},L_{\text{ptr}}(i)+L_{\text{P}}(i)}^{(k)} \parallel \cdots \parallel \mathbf{s}_{\text{R},\text{out},L_{\text{ptr}}(i)+L_{\text{P}}(i)+L_{\text{V}}(i)-1}^{(k)}. \end{aligned}$$

If the above does not hold, remove the k -th element from $\mathcal{S}_{\text{BT}}$ and continue to the next element in $\mathcal{S}_{\text{BT}}$ (if any). Else, continue to the next index $i + 1$ (if any).

F. If $L_{\text{ptr}}(i) + L_{\text{P}}(i) + L_{\text{V}}(i) \geq m_k + 1$, then continue to the next element in $\mathcal{S}_{\text{BT}}$ (if any).

5. Final output:

- If Outs contains more than one element, return err.
- If Outs contains no element, return none.
- Else, return the only element in Outs, $(i, \mathbb{x}^{(1)}, \boldsymbol{\tau}^{(1)}, (\hat{\alpha}_1^{(1)}, \dots, \hat{\alpha}_i^{(1)}))$.

Time complexity. The time complexity of BackTrack is dominated by the time to construct $\mathcal{S}_{\text{BT}}$ in Equation 11. If an element in $\mathcal{S}_{\text{BT}}$ matches the duplex sponge construction, then it is stored in Outs. By the return condition in Item 5, BackTrack can stop when Outs has two elements, so BackTrack can be implemented so to look for at most one element. An element in $\mathcal{S}_{\text{BT}}$ contains a statement and a tuple of permutation states of length at most $|\text{tr}_{\nabla}.\text{p}|$. The sequence of permutation states can be computed by backtracking. This process involves the following operations.

- Looking up the last capacity segment in $\text{tr}_{\nabla}.\text{p}$ (with the initial segment being the input to backtrack).

Each lookup in $\text{tr}_{\nabla}.\text{p}$ is for elements of length $(r + c) \cdot \log |\Sigma|$ and takes time $\log |\text{tr}_{\nabla}.\text{p}|$ via dichotomic search. If an element is found, the associated permutation state (of length $(r + c) \cdot \log |\Sigma|$) is added to $\mathcal{S}_{\text{BT}}$.

- Continuing the search until reaching a permutation state that is not in $\text{tr}_\nabla.p$ but is in $\text{tr}_\nabla.h$.

Since all elements looked up are different (by Theorem 5.3, each permutation state in a sequence $S^{(k)} \in \mathcal{S}_{\text{BT}}(\text{tr}, s)$ has different input capacity segments), at most $|\text{tr}_\nabla.p|$ lookups are needed to find the last capacity segment. The last lookup is done over the trace $\text{tr}_\nabla.h$ and takes at most time $\log |\text{tr}_\nabla.h| \cdot (c \cdot \log |\Sigma|) + n$. If the lookup returns an entry, the instance $\mathfrak{x}$ (of length n) is copied into $\mathcal{S}_{\text{BT}}$.

Overall, the total running time is

$$O(|\text{tr}_\nabla.p| \cdot \log |\text{tr}_\nabla.p| \cdot (r + c) \cdot \log |\Sigma| + \log |\text{tr}_\nabla.h| \cdot (c \cdot \log |\Sigma|) + n). \quad (13)$$

5.3 Lookahead procedure

The lookahead procedure $\text{LookAhead}(\text{tr}_\nabla.p, s, i)$

- takes as input the data structure $\text{tr}_\nabla.p$ as defined in Section 5.1, a permutation state $s = (s_R, s_C) \in \Sigma^{r+c}$, and a round index $i \in [k]$, and
- returns a special symbol (err or none) or a vector $\hat{\rho}_i \in \Sigma^{\ell_V(i)}$.

Informally, LookAhead recovers an encoded verifier message $\hat{\rho}_i$ obtained from a hash chain starting at s .

$\text{LookAhead}(\text{tr}_\nabla.p, s, i)$:

1. Parse the query-answer list of $\text{tr}_\nabla.p$ into a list

$$\mathcal{S}_{\text{LA}} := \left(S_{\text{LA}}^{(k)} \right)_k = \left(s_{\text{in},0}^{(k)}, s_{\text{out},0}^{(k)}, s_{\text{in},1}^{(k)}, s_{\text{out},1}^{(k)}, \dots, s_{\text{in},m_k-1}^{(k)}, s_{\text{out},m_k-1}^{(k)} \right)_k \quad (14)$$

where, for every $k \in [|\mathcal{S}_{\text{LA}}|]$:

- (a) $m_k \leq L_V(i)$ (i.e., the length of the chain is at most $L_V(i)$);
 - (b) $s_{\text{in},0}^{(k)} = s$ (i.e., the first input state equals the permutation state received as input);
 - (c) $\forall \iota \in [m_k - 1], s_{\text{out},\iota-1}^{(k)} = s_{\text{in},\iota}^{(k)}$ (i.e., the $(\iota - 1)$ -th output state equals the ι -th input state);
 - (d) $\forall \iota \in [0, m_k - 1], s_{\text{C,in},\iota}^{(k)} \neq s_{\text{C,out},\iota}^{(k)}$ (no “loops” across query and answer capacity segments);
 - (e) the length of the sequence is maximal (among sequences satisfying the above properties).
2. Final output:
 - (a) If $\mathcal{S}_{\text{LA}}$ contains more than one element, then return err.
 - (b) If $\mathcal{S}_{\text{LA}}$ is empty, then return none.
 - (c) Else, $\mathcal{S}_{\text{LA}}$ contains a single element $S_{\text{LA}}^{(1)}$. Sample $L_V(i) - m_1$ additional random permutation states $s_{\text{R,out},m_1}^{(1)}, \dots, s_{\text{R,out},L_V(i)-1}^{(1)} \leftarrow \mathcal{U}(\Sigma^r)$. Return

$$\hat{\rho}_i := (s_{\text{R,out},0}^{(1)} \| s_{\text{R,out},1}^{(1)} \| \dots \| s_{\text{R,out},L_V(i)-1}^{(1)})[0: \ell_V(i)] \in \Sigma^{\ell_V(i)}.$$

Time complexity. The time of LookAhead is dominated by Item 1c. Producing an element in $\mathcal{S}_{\text{LA}}$ involves, at worst, $L_V(i)$ lookups by input in $\text{tr}_\nabla.p$ (via its method $\text{tr}_\nabla.p.\text{inlu}$). The search stops if it encounters two conflicting chains. Since there are no loops (the query and answer capacity segments are different), at most $|\text{tr}_\nabla.p|$ lookups by input lead to the last capacity segment. Therefore, the overall time complexity is

$$O(L_V(i) \cdot \log |\text{tr}_\nabla.p| \cdot (r + c) \cdot \log |\Sigma|). \quad (15)$$

5.4 Prover transformation

The auxiliary procedure D2SAlgo translates queries of a malicious prover $\tilde{\mathcal{P}}$ for $\mathcal{V}$ (intended for oracles sampled from $\mathcal{D}_{\mathbb{E}}(\lambda, n)$) into corresponding oracle queries of a malicious prover $\tilde{\mathcal{P}}_{\text{std}}$ for $\mathcal{V}_{\text{std}}$ (intended for oracles sampled from $\mathcal{D}_{\text{IP}}(\lambda, n)$). To reduce the complexity of this step, we first define D2SQuery to deal with most of the complexity of translating the oracle queries, and then define D2SAlgo as a thin wrapper around it, dealing with the encoding and decoding of messages.

Oracle wrapper D2SQuery. The oracle D2SQuery relies on an “intermediate” oracle distribution $\mathcal{D}_{\Sigma}(\lambda, n)$ to answer oracle queries for (h, p, p^{-1}) . It is defined as follows

$$\mathcal{D}_{\Sigma}(\lambda, n) := \mathcal{U} \left((\{0, 1\}^{\leq n} \times \Sigma^{\delta} \times \Sigma^{\ell_{\mathbf{P}}(1)} \times \dots \times \Sigma^{\ell_{\mathbf{P}}(i)} \rightarrow \Sigma^{\ell_{\mathbf{V}}(i)})_{i \in [k]} \right). \quad (16)$$

Oracles sampled from $\mathcal{D}_{\Sigma}(\lambda, n)$ are denoted as g_i for $i \in [k]$. The oracle D2SQuery works as follows.

1. Initialize the following.
 - A list tr that will store query-answer pairs for h and p , stored as tuples $(h', \mathbb{x}, s_C)$ for h and tuples $(p', s_{\text{in}}, s_{\text{out}})$ for p , ordered by query time of the adversary.
 - A list Cache_p that will store query-answer pairs $(s_{\text{in}}, s_{\text{out}})$, sorted lexicographically by input.
 - A data structure tr_{∇} that will maintain entries from tr , allowing for efficient lookups (see Section 5.1).
2. For every query $\mathbb{x} \in \{0, 1\}^{\leq n}$ to the oracle h :
 - (a) Lookup $s_{C, \text{out}} := \text{tr}_{\nabla}.h.\text{inlu}(\mathbb{x}) \in \Sigma^c$.
 - (b) If the lookup returned $\perp$, sample $s_{C, \text{out}} \leftarrow \mathcal{U}(\Sigma^c)$ and run $\text{tr}_{\nabla}.h.\text{add}(\mathbb{x}, s_{C, \text{out}})$.
 - (c) Add $(h', \mathbb{x}, s_{C, \text{out}})$ to tr .
 - (d) Return $s_{C, \text{out}}$.
3. For every query $s_{\text{out}} \in \Sigma^{r+c}$ to the oracle p^{-1} :
 - (a) Lookup $s_{\text{in}} := \text{tr}_{\nabla}.p.\text{outlu}(s_{\text{out}}) \in \Sigma^{r+c}$.
 - (b) If the lookup returned $\perp$, sample $s_{\text{in}} \leftarrow \mathcal{U}(\Sigma^{r+c})$ and run $\text{tr}_{\nabla}.p.\text{add}(s_{\text{in}}, s_{\text{out}})$.
 - (c) Add $(p', s_{\text{in}}, s_{\text{out}})$ to tr .
 - (d) Return s_{in} .
4. For every query $s_{\text{in}} = (s_{R, \text{in}}, s_{C, \text{in}}) \in \Sigma^{r+c}$ to the oracle p :
 - (a) Run $\text{BackTrack}(\text{tr}, \text{tr}_{\nabla}, s_{\text{in}})$, and proceed as below according to its output.
 - (b) If BackTrack 's output is err then abort.
 - (c) If BackTrack 's output is none , then:
 - i. If $\exists s_{\text{out}} \in \Sigma^{r+c} : (s_{\text{in}}, s_{\text{out}}) \in \text{Cache}_p$, remove the first occurrence of $(s_{\text{in}}, s_{\text{out}})$ from Cache_p and let the first such match be denoted s_{out} .
 - ii. Else, let $s_{\text{out}} := \text{tr}_{\nabla}.p.\text{inlu}(s_{\text{in}}) \in \Sigma^{r+c}$, if any.
 - iii. Else, sample $s_{\text{out}} \leftarrow \mathcal{U}(\Sigma^{r+c})$ and run $\text{tr}_{\nabla}.p.\text{add}(s_{\text{in}}, s_{\text{out}})$.
 - (d) If BackTrack 's output is a tuple $(i, \mathbb{x}, \tau, (\hat{\alpha}_1, \dots, \hat{\alpha}_i))$ such that $\exists \iota \in [i], \hat{\alpha}_{\iota} \notin \text{Im}(\varphi_{\iota})$ then:
 - i. Let $s_{\text{out}} := \text{tr}_{\nabla}.p.\text{inlu}(s_{\text{in}}) \in \Sigma^{r+c}$, if any.
 - ii. Else, sample $s_{\text{out}} \leftarrow \mathcal{U}(\Sigma^{r+c})$ and run $\text{tr}_{\nabla}.p.\text{add}(s_{\text{in}}, s_{\text{out}})$.
 - (e) If BackTrack 's output is a tuple $(i, \mathbb{x}, \tau, (\hat{\alpha}_1, \dots, \hat{\alpha}_i))$ such that $\forall \iota \in [i], \hat{\alpha}_{\iota} \in \text{Im}(\varphi_{\iota})$ then:
 - i. Set $\hat{\rho}_i := g_i(\mathbb{x}, \tau, \hat{\alpha}_1, \dots, \hat{\alpha}_i)$.
 - ii. Let $s_{\text{out}} \in \Sigma^{r+c} = \text{tr}_{\nabla}.p.\text{inlu}(s_{\text{in}})$, if any.
 - iii. Else
 - A. Set $\ell_z := r - (\ell_{\mathbf{V}}(i) \bmod r)$ and sample a random remainder $\mathbf{z} \leftarrow \mathcal{U}(\Sigma^{\ell_z})$.

- B. Parse $\hat{\rho}_i \| \mathbf{z}$ as rate segments $\mathbf{s}_R^{(0)}, \dots, \mathbf{s}_R^{(L_V(i)-1)} \in \Sigma^r$.
- C. Sample random capacities $\mathbf{s}_C^{(0)}, \dots, \mathbf{s}_C^{(L_V(i)-1)} \leftarrow \mathcal{U}(\Sigma^c)$.
- D. Append to Cache_p the following pairs in $\Sigma^{r+c} \times \Sigma^{r+c}$:

$$\left((\mathbf{s}_R^{(0)}, \mathbf{s}_C^{(0)}), (\mathbf{s}_R^{(1)}, \mathbf{s}_C^{(1)}) \right), \dots, \left((\mathbf{s}_R^{(L_V(i)-2)}, \mathbf{s}_C^{(L_V(i)-2)}), (\mathbf{s}_R^{(L_V(i)-1)}, \mathbf{s}_C^{(L_V(i)-1)}) \right).$$

- E. Set $\mathbf{s}_{\text{out}} := (\mathbf{s}_R^{(0)}, \mathbf{s}_C^{(0)})$ and run $\text{tr}_\nabla.\text{p.add}(\mathbf{s}_{\text{in}}, \mathbf{s}_{\text{out}})$.
- (f) Add $(p, \mathbf{s}_{\text{in}}, \mathbf{s}_{\text{out}})$ to tr .
- (g) Return $\mathbf{s}_{\text{out}}$.

Time complexity of D2SQuery. We discuss the time of D2SQuery depending on the type of query.

- *Query to h.* D2SQuery does a lookup for a previously-answered query in $\text{tr}_\nabla.h$ and stores the query in tr . Hence

$$t_{\text{D2SQuery},h} = O((c \cdot \log |\Sigma|) \cdot \log |\text{tr}_\nabla.h| + n).$$

- *Query to p.* The runtime of D2SQuery for a query to p is dominated by the time to compute BackTrack with inputs tr_∇ , as well as the time to insert at most $\max_{i \in [k]} L_V(i)$ pairs in $\Sigma^{r+c} \times \Sigma^{r+c}$ in Cache_p . Hence

$$\begin{aligned} t_{\text{D2SQuery},p} &= O(t_{\text{BackTrack}} + (r+c) \cdot \log |\Sigma| \cdot \max_{i \in [k]} L_V(i)) \\ &= O \left(|\text{tr}_\nabla.p| \cdot \log |\text{tr}_\nabla.p| \cdot (r+c) \cdot \log |\Sigma| + \log |\text{tr}_\nabla.h| \cdot (c \cdot \log |\Sigma|) + n + (r+c) \cdot \log |\Sigma| \cdot \max_{i \in [k]} L_V(i) \right). \end{aligned}$$

(via the expression for $t_{\text{BackTrack}}$ in Equation 13)

- *Query to p^{-1} .* The time of D2SQuery is dominated by the lookup for a previously-answered query in $\text{tr}_\nabla.p$:

$$t_{\text{D2SQuery},p^{-1}} = O((r+c) \cdot \log |\text{tr}_\nabla.p|).$$

Auxiliary procedure D2SAlgo. The algorithm $\text{D2SAlgo}^f(\mathcal{A})$ is defined as follows.

$\text{D2SAlgo}^f(\mathcal{A})$:

1. For every $i \in [k]$, initialize an empty list tr_i .
2. Run $\mathcal{A}^{\text{D2SQuery}}$, answering its queries as follows.
3. For every query $(\mathbf{x}, \boldsymbol{\tau}, \hat{\alpha}_1, \dots, \hat{\alpha}_i)$ to oracle g_i (with $i \in [k]$, $\mathbf{x} \in \{0,1\}^{\leq n}$, $\boldsymbol{\tau} \in \Sigma^\delta$, and $\hat{\alpha}_\iota \in \Sigma^{\ell_P(\iota)}$):
 - (a) If the query is already present in tr_i , respond with the corresponding answer. Otherwise, proceed as follows.
 - (b) For every $\iota \in [i]$, set $\alpha_\iota := \varphi_\iota^{-1}(\hat{\alpha}_\iota)$ (compute the unique preimage of α_ι under φ_ι). (By Item 4e, each α_ι contained in a query from Item 4(e)i is in the image space of φ_ι .)
 - (c) Let $\tilde{\tau} := \text{bin}(\boldsymbol{\tau}) \in \{0,1\}^{\delta_\star}$ be the binary representation of $\boldsymbol{\tau} \in \Sigma^\delta$, where $\delta_\star := \log |\Sigma| \cdot \delta$.
 - (d) Set $\rho_i := f_i(\mathbf{x}, \tilde{\tau}, \alpha_1, \dots, \alpha_i)$.
 - (e) Sample $\hat{\rho}_i \leftarrow \psi_i^{-1}(\rho_i)$ (sample a preimage of ρ_i under ψ_i).
 - (f) Add the query-answer pair $((\mathbf{x}, \tilde{\tau}, \hat{\alpha}_1, \dots, \hat{\alpha}_i), \hat{\rho}_i)$ to tr_i .
 - (g) Respond to $\mathcal{A}$ with $\hat{\rho}_i$.

4. Let $\mathcal{A}$'s output be $(\mathbb{x}, \pi_{\mathcal{A}})$, and parse $\pi_{\mathcal{A}}$ as $(\tau, \alpha_1, \dots, \alpha_k)$.
5. Set $\tilde{\pi} := (\tilde{\tau}, \alpha_1, \dots, \alpha_k)$ where $\tilde{\tau} := \text{bin}(\tau)$.
6. Output $(\mathbb{x}, \tilde{\pi})$.

To simplify notation in the analysis, we use the following shorthand when referring to D2SAlgo:

$$\text{D2SAlgo}^f(\mathcal{A}) := \mathcal{A}^{\text{D2SQuery}^{\psi^{-1} \circ f \circ \varphi^{-1}}}. \quad (17)$$

Query complexity of D2SAlgo. Intuitively, D2SAlgo groups queries to p into a single query to f (if any). If D2SQuery does not abort, $\mathcal{A}$ must perform at least $L_p(i)$ queries to p in order to trigger a query to f_i , but can later trigger additional queries to f_i via additional queries to p by keeping the same message prefix. Separately, queries to h or p^{-1} do not trigger queries to f . Hence the number of queries by $\text{D2SAlgo}(\mathcal{A})$ to f is at most

$$\theta_*(t_h, t_p, t_{p^{-1}}) := t_p.$$

Time complexity of D2SAlgo. D2SAlgo incurs an additive cost for each oracle query made by $\mathcal{A}$. The cost is given by the runtime of D2SQuery, and the time to compute (for each $i \in [k]$ queried by D2SQuery) ψ_i^{-1} and φ_i^{-1} for all $i \leq i$. Let:

- $t_{\varphi^{-1}}$ be a bound on the time to compute each φ_i^{-1} ,
- $t_{\psi^{-1}}$ be a bound on the time to compute each ψ_i^{-1} ,
- t_{bin} be the time to compute the binary encoding of the input salt (note that $t_{\text{bin}} = \delta \log |\Sigma|$ as the salt consists of δ symbols in Σ),
- $t_{\text{D2SQuery}, h}, t_{\text{D2SQuery}, p}, t_{\text{D2SQuery}, p^{-1}}$ be bounds for D2SQuery to respond to queries to h, p, p^{-1} .

The time complexity of D2SAlgo is:

$$\begin{aligned} & O\left(t_h \cdot t_{\text{D2SQuery}, h} + t_p \cdot (t_{\text{D2SQuery}, p} + t_{\text{bin}} + t_{\psi^{-1}} + t_{\varphi^{-1}}) + t_{p^{-1}} \cdot t_{\text{D2SQuery}, p^{-1}}\right) \\ & \leq O\left((t_h + t_p + t_{p^{-1}}) \cdot (t_p + t_{p^{-1}}) \cdot \log(t_h(t_p + t_{p^{-1}})) \cdot (r + c) \cdot \log |\Sigma| + n \right. \\ & \quad \left. + (t_p + t_{p^{-1}})(t_{\psi^{-1}} + t_{\varphi^{-1}} + \delta \cdot \log |\Sigma|)\right), \end{aligned} \quad (18)$$

provided $t_p + t_{p^{-1}} \geq \max_{i \in [k]} L_V(i)$ (used in the bound for $t_{\text{D2SQuery}, p}$).

5.5 Trace transformation

Before defining D2STrace we introduce StdTrace, re-mapping query-answer traces into input-output traces from $\mathcal{D}_{\Sigma}(\lambda, n)$ (see Equation 16). Then, we define D2STrace as a thin wrapper around it dealing with encoding and decoding of messages.

5.5.1 Auxiliary procedure StdTrace

The procedure StdTrace works as follows.

StdTrace(tr):

1. Initialize a list tr_{std} , that will maintain query-answer pairs for g , sorted by query time.

2. Initialize a list $\text{tr}_{\text{std}}^{\text{LA}}$ that will maintain query-answer pairs for g , sorted lexicographically by input and then by output.
3. Initialize the data structure tr_{∇} for tr as defined in Section 5.1:
 - (a) For each entry in tr of the form $(h', \mathbb{x}, s_{\text{C,out}})$, run $\text{tr}_{\nabla}.\text{h.add}(\mathbb{x}, s_{\text{C,out}})$.
 - (b) For each entry in tr of the form $(p', s_{\text{in}}, s_{\text{out}})$, run $\text{tr}_{\nabla}.\text{p.add}(s_{\text{in}}, s_{\text{out}})$.
 - (c) For each entry in tr of the form $(p^{-1}, s_{\text{out}}, s_{\text{in}})$, ignore the entry and continue.
4. For each $j \in [|\text{tr}|]$:
 - (a) If tr_j is of the form $(p', s_{\text{in}}, s_{\text{out}})$:
 - i. Run $\text{BackTrack}(\text{tr}, \text{tr}_{\nabla}, s_{\text{in}})$, and do the following based on its output.
 - ii. If the output is err, then abort.
 - iii. If the output is none, or a tuple $(i, \mathbb{x}, \tau, (\hat{\alpha}_1, \dots, \hat{\alpha}_i))$ such that $\exists \iota \in [i], \hat{\alpha}_{\iota} \notin \text{Im}(\varphi_{\iota})$, then continue to the next element in tr .
 - iv. Else the output is a tuple $(i, \mathbb{x}, \tau, (\hat{\alpha}_1, \dots, \hat{\alpha}_i))$ such that $\forall \iota \in [i], \hat{\alpha}_{\iota} \in \text{Im}(\varphi_{\iota})$, so then:
 - A. If $\text{tr}_{\text{std}}^{\text{LA}}$ contains a query-answer pair of the form $((i, \mathbb{x}, \tau, (\hat{\alpha}_1, \dots, \hat{\alpha}_i)), \cdot)$ then add such a pair to tr_{std} .
 - B. Else, run $\text{LookAhead}(\text{tr}_{\nabla}.\text{p}, s_{\text{in}}, i)$, and do the following based on its output.
 - C. If LookAhead outputs err, then abort.
 - D. Else, denote LookAhead 's output as $\hat{\rho}_i \in \Sigma^{\ell_{\mathbf{v}}(i)}$ and add the query-answer pair $((i, \mathbb{x}, \tau, (\hat{\alpha}_1, \dots, \hat{\alpha}_i)), \hat{\rho}_i)$ to $\text{tr}_{\text{std}}^{\text{LA}}$ and to tr_{std} . (The output is not none as $(p', s_{\text{in}}, s_{\text{out}}) \in \text{tr}$.)
 - (b) If tr_j is of the form $(p^{-1}, s_{\text{out}}, s_{\text{in}})$ or $(h', \mathbb{x}, s_{\text{C,out}})$, continue.
5. Return tr_{std} .

Time complexity. The time complexity of StdTrace is dominated by the following items.

- The time t_{sort} to build tr_{∇} (see Item 3b), of size at most $|\text{tr}|$. (Nit: this procedure does not rely on input lookups for $\text{tr}_{\nabla}.\text{h}$.)
- The time $t_{\text{BackTrack}}$ for BackTrack , executed for every query to p in tr (see Item 4(a)i). A bound for the running time of BackTrack is given in Equation 13.
- The time $t_{\text{LookAhead}}$ of LookAhead , executed every time BackTrack returns a tuple (see Item 4(a)iv). A bound for the running time of LookAhead is given in Equation 15.
- The time t_{std} to look up a tuple in $\text{tr}_{\text{std}}^{\text{LA}}$. Since its entries are elements $((i, \mathbb{x}, \tau, (\hat{\alpha}_1, \dots, \hat{\alpha}_i)), \hat{\rho}_i) \in [k] \times \{0, 1\}^{\leq n} \times \Sigma^{\delta} \times \mathcal{M}_{\mathbf{P},1} \times \dots \times \mathcal{M}_{\mathbf{P},i} \times \mathcal{M}_{\mathbf{V},i}$,

$$t_{\text{std}} \leq \log |\text{tr}| \cdot \left(\log k + n + \left(\delta + \sum_{\iota \in [i]} L_{\mathbf{P}}(\iota) + L_{\mathbf{V}}(i) \right) \log |\Sigma| \right). \quad (19)$$

Therefore:

$$\begin{aligned} t_{\text{StdTrace}} &= O(t_{\text{sort}} + |\text{tr}| \cdot (t_{\text{BackTrack}} + t_{\text{LookAhead}} + t_{\text{std}})) \\ &= O(|\text{tr}| \cdot (t_{\text{BackTrack}} + t_{\text{LookAhead}} + t_{\text{std}})) \quad (\text{by Equations 13, 15 and 19}) \end{aligned} \quad (20)$$

5.5.2 Auxiliary procedure D2STrace

The algorithm D2STrace is defined as follows.

D2STrace(tr):

1. Initialize an empty list tr_{std} .
2. Set $\widehat{\text{tr}}_{\text{std}} := \text{StdTrace}(\text{tr})$.
3. For each tuple $((i, \mathbb{x}, \tau, (\hat{\alpha}_1, \dots, \hat{\alpha}_i), \hat{\rho}_i))$ in $\widehat{\text{tr}}_{\text{std}}$, where $i \in [k]$, $\mathbb{x} \in \{0, 1\}^{\leq n}$, $\tau \in \Sigma^\delta$, $\hat{\alpha}_1 \in \Sigma^{\ell_P(1)}, \dots, \hat{\alpha}_i \in \Sigma^{\ell_P(i)}, \hat{\rho}_i \in \Sigma^{\ell_V(i)}$:
 - Compute the (unique) preimage $(\alpha_1, \dots, \alpha_i)$ of $(\hat{\alpha}_1, \dots, \hat{\alpha}_i)$ under φ :

$$\alpha_1 := \varphi_1^{-1}(\hat{\alpha}_1) \in \mathcal{M}_{P,1}, \dots, \alpha_i := \varphi_i^{-1}(\hat{\alpha}_i) \in \mathcal{M}_{P,i}.$$
 - Decode the encoded verifier message: $\rho_i := \psi_i(\hat{\rho}_i) \in \mathcal{M}_{V,i}$.
 - Map the salt string to a binary string: $\tilde{\tau} := \text{bin}(\tau) \in \{0, 1\}^{\delta^*}$.
 - Append $(i, (\mathbb{x}, \tilde{\tau}, (\alpha_1, \dots, \alpha_i)), \rho_i)$ to tr_{std} .
4. Return tr_{std} .

With a slight abuse of notation, we indicate the above procedure with

$$\text{D2STrace} := (\varphi^{-1}, \psi) \circ \text{StdTrace}.$$

Time complexity. D2STrace is a thin wrapper over StdTrace that re-maps the prover and verifier messages using from the alphabet Σ to the prover and verifier messages spaces, via the codec. Let:

- $t_{\varphi^{-1}}$ be the total time to compute each φ_i^{-1} for $i \in [k]$,
- $t_{\psi^{-1}}$ be the total time to compute each ψ_i^{-1} for $i \in [k]$,
- $t_{\text{bin}} = \delta \log |\Sigma|$ be the time to compute the binary encoding of the input salt, consisting of δ symbols in Σ .

Let $t_{\text{BackTrack}}$ and $t_{\text{LookAhead}}$ be the times to compute BackTrack and LookAhead. Then

$$\begin{aligned} t_{\text{D2STrace}} &= O \left(t_{\text{StdTrace}} + |\widehat{\text{tr}}_{\text{std}}| \cdot (t_{\psi^{-1}} + t_{\varphi^{-1}} + t_{\text{bin}}) \right) \\ &= O \left(|\text{tr}| \cdot (t_{\text{BackTrack}} + t_{\text{LookAhead}} + t_{\text{std}}) + |\widehat{\text{tr}}_{\text{std}}| \cdot (t_{\psi^{-1}} + t_{\varphi^{-1}} + t_{\text{bin}}) \right) \quad (\text{by Equation 20}) \\ &= O \left(|\text{tr}| \cdot (t_{\text{BackTrack}} + t_{\text{LookAhead}} + t_{\text{std}}) + t_{\psi^{-1}} + t_{\varphi^{-1}} + \delta \log |\Sigma| \right). \quad (\text{since } |\widehat{\text{tr}}_{\text{std}}| \leq |\text{tr}|) \end{aligned}$$

5.6 Bad events

Definition 5.5. An entry tr_j in tr is **redundant** if one of the following holds:

$$\text{tr}_j = (h', \mathbb{x}, s_C) \implies \exists j' < j: \text{tr}_{j'} = (h', \mathbb{x}, s_C), \quad (21)$$

$$\text{tr}_j = (p', s_{\text{in}}, s_{\text{out}}) \implies \exists j' < j: \text{tr}_{j'} = (p', s_{\text{in}}, s_{\text{out}}) \vee \text{tr}_{j'} = (p^{-1}, s_{\text{out}}, s_{\text{in}}), \quad (22)$$

$$\text{tr}_j = (p^{-1}, s_{\text{out}}, s_{\text{in}}) \implies \exists j' < j: \text{tr}_{j'} = (p^{-1}, s_{\text{out}}, s_{\text{in}}) \vee \text{tr}_{j'} = (p', s_{\text{in}}, s_{\text{out}}). \quad (23)$$

Definition 5.6. The **base trace** $\bar{\text{tr}}$ of tr is obtained by removing all redundant entries.

Definition 5.7. The predicate $E(\text{tr})$ checks if one of the following holds for the query-answer trace tr .

- *There exists an output capacity segment in the base trace $\bar{\text{tr}}$ that appears as a prior input/output capacity segment. Formally, one of the following predicates holds:*

$$E_h(\text{tr}) := \exists j > 0, \mathbf{s}_C \in \Sigma^c : \bar{\text{tr}}_j = ('h', \cdot, \mathbf{s}_C) \text{ and } \exists j' < j : \begin{aligned} &\bar{\text{tr}}_{j'} = ('h', \cdot, \mathbf{s}_C) \\ &\vee \bar{\text{tr}}_{j'} = ('p', \cdot, (\cdot, \mathbf{s}_C)) \\ &\vee \bar{\text{tr}}_{j'} = ('p^{-1}', \cdot, (\cdot, \mathbf{s}_C)) \\ &\vee \bar{\text{tr}}_{j'} = ('p', (\cdot, \mathbf{s}_C), \cdot) \\ &\vee \bar{\text{tr}}_{j'} = ('p^{-1}', (\cdot, \mathbf{s}_C), \cdot) \end{aligned} \quad (24)$$

$$E_p(\text{tr}) := \exists j > 0, \mathbf{s}_C \in \Sigma^c : \bar{\text{tr}}_j = ('p', \cdot, (\cdot, \mathbf{s}_C)) \text{ and } \begin{aligned} &\exists j' < j : \bar{\text{tr}}_{j'} = ('h', \cdot, \mathbf{s}_C) \\ &\vee \exists j' < j : \bar{\text{tr}}_{j'} = ('p', \cdot, (\cdot, \mathbf{s}_C)) \\ &\vee \exists j' < j : \bar{\text{tr}}_{j'} = ('p^{-1}', \cdot, (\cdot, \mathbf{s}_C)) \\ &\vee \exists j' \leq j : \bar{\text{tr}}_{j'} = ('p', (\cdot, \mathbf{s}_C), \cdot) \\ &\vee \exists j' \leq j : \bar{\text{tr}}_{j'} = ('p^{-1}', (\cdot, \mathbf{s}_C), \cdot) \end{aligned} \quad (25)$$

$$E_{p^{-1}}(\text{tr}) := \exists j > 0, \mathbf{s}_C \in \Sigma^c : \bar{\text{tr}}_j = ('p^{-1}', \cdot, (\cdot, \mathbf{s}_C)) \text{ and } \begin{aligned} &\exists j' < j : \bar{\text{tr}}_{j'} = ('h', \cdot, \mathbf{s}_C) \\ &\vee \exists j' < j : \bar{\text{tr}}_{j'} = ('p', \cdot, (\cdot, \mathbf{s}_C)) \\ &\vee \exists j' < j : \bar{\text{tr}}_{j'} = ('p^{-1}', \cdot, (\cdot, \mathbf{s}_C)) \\ &\vee \exists j' \leq j : \bar{\text{tr}}_{j'} = ('p', (\cdot, \mathbf{s}_C), \cdot) \\ &\vee \exists j' \leq j : \bar{\text{tr}}_{j'} = ('p^{-1}', (\cdot, \mathbf{s}_C), \cdot) \end{aligned} \quad (26)$$

- *The same query to p leads to different answers, or there are inconsistent queries across p and p^{-1} :*

$$E_{\text{func}}(\text{tr}) := \exists j > 0, \mathbf{s}_{\text{in}} \in \Sigma^{r+c} : \bar{\text{tr}}_j = ('p', \mathbf{s}_{\text{in}}, \cdot) \text{ and } \exists j' < j : \begin{aligned} &\bar{\text{tr}}_{j'} = ('p', \mathbf{s}_{\text{in}}, \cdot) \\ &\vee \bar{\text{tr}}_{j'} = ('p^{-1}', \cdot, \mathbf{s}_{\text{in}}) \end{aligned} \quad (27)$$

$E_{\text{func}}(\text{tr})$ never holds for a permutation p and its inverse p^{-1} , however in the analyses certain probabilistic stateful algorithms may, with small probability, cause such events.

We now bound the probability that this predicate happens, when considering oracles sampled from different distributions.

Lemma 5.8. *For every $(t_h, t_p, t_{p^{-1}})$ -query algorithm $\tilde{\mathcal{P}}$ such that $t_p \geq L$,*

$$\begin{aligned} &\max \left\{ \Pr \left[E(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}}) \mid \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\Sigma}(\lambda, n) \\ (\mathbb{X}, \pi) \xleftarrow{\text{tr}_{\tilde{\mathcal{P}}}} \tilde{\mathcal{P}}^{h,p,p^{-1}} \\ b \xleftarrow{\text{tr}_{\mathcal{V}}} \mathcal{V}^{h,p}(\mathbb{X}, \pi) \end{array} \right], \Pr \left[E(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}}) \mid \begin{array}{l} \mathbf{g} \leftarrow \mathcal{D}_{\Sigma}(\lambda, n) \\ (\mathbb{X}, \pi) \xleftarrow{\text{tr}_{\tilde{\mathcal{P}}}} \tilde{\mathcal{P}}^{\text{D2SQuery}^{\mathbf{g}}} \\ b \xleftarrow{\text{tr}_{\mathcal{V}}} \mathcal{V}^{\text{D2SQuery}^{\mathbf{g}}}(\mathbb{X}, \pi) \end{array} \right] \right\} \\ &\leq \frac{7(t_h + 1 + t_p + L + t_{p^{-1}})^2 - 3(t_h + 1 + t_p + L + t_{p^{-1}})}{2|\Sigma|^c}. \end{aligned}$$

Proof. $\tilde{\mathcal{P}}$ is a $(t_h, t_p, t_{p^{-1}})$ -query algorithm, while $\mathcal{V}$ is a $(1, L, 0)$ -query algorithm. Therefore, the base trace at the end of the execution of $\tilde{\mathcal{P}}$ and $\mathcal{V}$ has size (in both experiments) at most:

$$|\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}}| \leq t_h + 1 + t_p + L + t_{p^{-1}}.$$

For each $j \in [t_h + 1 + t_p + L + t_{p^{-1}}]$, let $E^{(j)}$ be the predicate that checks E for the (fixed) j -th query-answer pair in the base trace $\bar{\text{tr}}$ of $\text{tr}_{\bar{p}} \parallel \text{tr}_{\mathcal{V}}$. Therefore, we can write:

$$E(\text{tr}_{\bar{p}} \parallel \text{tr}_{\mathcal{V}}) = \bigvee_{j \in [t_h + 1 + t_p + L + t_{p^{-1}}]} \left(E_h^{(j)}(\text{tr}_{\bar{p}} \parallel \text{tr}_{\mathcal{V}}) \vee E_p^{(j)}(\text{tr}_{\bar{p}} \parallel \text{tr}_{\mathcal{V}}) \vee E_{p^{-1}}^{(j)}(\text{tr}_{\bar{p}} \parallel \text{tr}_{\mathcal{V}}) \vee E_{\text{func}}^{(j)}(\text{tr}_{\bar{p}} \parallel \text{tr}_{\mathcal{V}}) \right).$$

Denote with $E_{\mathfrak{S}}$ the left-hand side experiment of which we are bounding the probability, and with E_{Σ} the right-hand side experiment. Note that, in E_{Σ} , if D2SQuery aborts then execution stops and the query-answer trace is stopped at that point. By Theorem 5.18, E will over the (partial) query-answer trace.

We bound the probability of the two events using sub-additivity of the function max and the probability measure, studying each sub-predicate below. We have:

$$\begin{aligned} & \max\{\Pr[E_{\mathfrak{S}}], \Pr[E_{\Sigma}]\} \\ & \leq \sum_{j \in [t_h + 1 + t_p + L + t_{p^{-1}}]} \max\left\{\Pr[E_{\mathfrak{S}}^{(j)}], \Pr[E_{\Sigma}^{(j)}]\right\}, \end{aligned} \quad (28)$$

where each term is defined as follows:

$$\begin{aligned} \Pr[E_{\mathfrak{S},h}^{(j)}] &:= \Pr \left[E_h^{(j)}(\text{tr}_{\bar{p}} \parallel \text{tr}_{\mathcal{V}}) \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n) \\ (\mathbb{X}, \pi) \xleftarrow{\text{tr}_{\bar{p}}} \tilde{\mathcal{P}}^{h,p,p^{-1}} \\ b \xleftarrow{\text{tr}_{\mathcal{V}}} \mathcal{V}^{h,p}(\mathbb{X}, \pi) \end{array} \right. \right], \\ \Pr[E_{\Sigma,h}^{(j)}] &:= \Pr \left[E_h^{(j)}(\text{tr}_{\bar{p}} \parallel \text{tr}_{\mathcal{V}}) \left| \begin{array}{l} \mathbf{g} \leftarrow \mathcal{D}_{\Sigma}(\lambda, n) \\ (\mathbb{X}, \pi) \xleftarrow{\text{tr}_{\bar{p}}} \tilde{\mathcal{P}}^{\text{D2SQuery}^{\mathbf{g}}} \\ b \xleftarrow{\text{tr}_{\mathcal{V}}} \mathcal{V}^{\text{D2SQuery}^{\mathbf{g}}}(\mathbb{X}, \pi) \end{array} \right. \right], \end{aligned}$$

and similarly for the remaining terms. Each sub-predicate is studied separately.

Predicate E_h (Equation 24). This predicate checks that an answer from h previously appears in the base trace $\bar{\text{tr}}$ as an answer of h , an answer of p (or p^{-1}), or a query to p (or p^{-1}).

In both events, h 's answers are sampled uniformly from Σ^c :

- In $E_{\mathfrak{S}}$, the oracle h is sampled uniformly from $\mathcal{U}(\{0, 1\}^{\leq n} \rightarrow \Sigma^c)$, and so its output distribution is uniform;
- In E_{Σ} , D2SQuery responds to queries to h performing lazy sampling in Item 2, and its outputs are uniform in Σ^c .

Before the j -th query, at most $(j - 1)$ output capacity segments and $(j - 1)$ input capacity segments appear in the base trace $\bar{\text{tr}}$. Therefore:

$$\max\left\{\Pr[E_{\mathfrak{S},h}^{(j)}], \Pr[E_{\Sigma,h}^{(j)}]\right\} \leq \frac{2(j-1)}{|\Sigma|^c}. \quad (29)$$

Predicate E_p (Equation 25). This predicate checks that an answer from p previously appears in the base trace $\bar{\text{tr}}$ as an answer from h , an answer from p (or p^{-1}), or a query to p (or p^{-1}). In both events p 's answers are uniformly distributed:

- In $E_{\mathfrak{S}}$, p is a random permutation over Σ^{r+c} and therefore its output is uniformly distributed.
- In E_{Σ} , D2SQuery responds to queries to p as follows:
 - In Items 4(c)iii and 4(d)ii they are sampled uniformly at random;

- In Items 4(c)i and 4(e)iiiE, the answer capacity segment is uniformly distributed, and the rate segment is an r -chunk of an answer by g , and thus uniformly distributed as well.
- In Items 4(c)ii, 4(d)i and 4(e)ii the output is chosen to be consistent with a previous query to p or p^{-1} . Fresh queries to p^{-1} are distributed uniformly in Σ^{r+c} (cf. Item 3b).

Before the j -th query index at most $(j-1)$ output capacity segments appear in the base trace $\bar{\text{tr}}$, and at most j input capacity segments (including the j -th input segment) in the base trace $\bar{\text{tr}}$. Therefore:

$$\max\left\{\Pr\left[E_{\mathfrak{S},p}^{(j)}\right], \Pr\left[E_{\Sigma,p}^{(j)}\right]\right\} \leq \frac{2j-1}{|\Sigma|^c}. \quad (30)$$

Predicate $E_{p^{-1}}$ (Equation 26). A similar reasoning as above applies to p^{-1} and $E_{p^{-1}}$, yielding

$$\max\left\{\Pr\left[E_{\mathfrak{S},p^{-1}}^{(j)}\right], \Pr\left[E_{\Sigma,p^{-1}}^{(j)}\right]\right\} \leq \frac{2j-1}{|\Sigma|^c}. \quad (31)$$

Predicate E_{func} (Equation 27). In $E_{\mathfrak{S}}$, the oracles p, p^{-1} are functions so the event E_{func} never holds for query-answer traces generated by $\tilde{\mathcal{P}}\|\mathcal{V}$. In E_{Σ} , instead, while all queries to p^{-1} are answered consistently, it may happen that, if the j -th query is to p with input s_{in} , in Item 4(c)i we match an entry in Cache_p and never hit Item 4(c)ii, thus responding inconsistently to the j -th query. In this case, the resulting base trace $\bar{\text{tr}}$ there exist $s_{\text{out}} \neq s'_{\text{out}}$ in Σ^{r+c} such that

$$\bar{\text{tr}}_j = (p', s_{\text{in}}, s_{\text{out}}) \text{ and } \exists j' < j : \begin{cases} \bar{\text{tr}}_{j'} = (p', s_{\text{in}}, s'_{\text{out}}) \\ \vee \bar{\text{tr}}_{j'} = (p^{-1}, s'_{\text{out}}, s_{\text{in}}) \end{cases}, \quad (32)$$

making E_{func} hold. In particular, this means that after the j -th query is made, one of the following holds:

$$\begin{aligned} (s_{\text{in}}, s_{\text{out}}) \in \text{Cache}_p^{<j} \wedge (p', s_{\text{in}}, s'_{\text{out}}) \in \text{tr}^{<j}, \text{ or} \\ (s_{\text{in}}, s_{\text{out}}) \in \text{Cache}_p^{<j} \wedge (p^{-1}, s'_{\text{out}}, s_{\text{in}}) \in \text{tr}^{<j}, \end{aligned} \quad (33)$$

where $\text{tr}^{<j}, \text{Cache}_p^{<j}$ are the states of the variables tr and Cache_p before the j -th query.

All pairs in Cache_p are sampled at random from $\Sigma^{r+c} \times \Sigma^{r+c}$. For every query to p for which BackTrack returns a tuple of the form $(i, \mathbb{x}, \tau, (\hat{\alpha}_1, \dots, \hat{\alpha}_i))$ (see Item 4e), $L_{\mathbb{V}}(i) - 1$ elements are added to Cache_p (see Item 4(e)iiiD). Recall that $L := \sum_{i \in [k]} L_{\mathbb{P}}(i) + L_{\mathbb{V}}(i)$ and $L_{\mathbb{V}} := \max_{i \in [k]} L_{\mathbb{V}}(i)$. Then:

$$\begin{aligned} & \max\left\{\Pr\left[E_{\mathfrak{S},\text{func}}^{(j)}\right], \Pr\left[E_{\Sigma,\text{func}}^{(j)}\right]\right\} \\ &= \Pr\left[E_{\Sigma,\text{func}}^{(j)}\right] \\ &\leq \Pr\left[\begin{array}{l} \exists s_{\text{in}}, s_{\text{out}}, s'_{\text{out}} \in \Sigma^{r+c} : \\ (s_{\text{in}}, s_{\text{out}}) \in \text{Cache}_p^{<j} \\ \wedge (p', s_{\text{in}}, s'_{\text{out}}) \in \text{tr}^{<j} \\ \wedge s_{\text{out}} \neq s'_{\text{out}} \end{array} \left| \begin{array}{l} g \leftarrow \mathcal{D}_{\Sigma}(\lambda, n) \\ (\mathbb{x}, \pi) \xleftarrow{\text{tr}_{\tilde{\mathcal{P}}}} \tilde{\mathcal{P}}^{\text{D2SQuery}^g} \\ b \xleftarrow{\text{tr}_{\mathcal{V}}} \mathcal{V}^{\text{D2SQuery}^g}(\mathbb{x}, \pi) \end{array} \right. \right] \\ &+ \Pr\left[\begin{array}{l} \exists s_{\text{in}}, s_{\text{out}}, s'_{\text{out}} \in \Sigma^{r+c} : \\ (s_{\text{in}}, s_{\text{out}}) \in \text{Cache}_p^{<j} \\ \wedge (p^{-1}, s'_{\text{out}}, s_{\text{in}}) \in \text{tr}^{<j} \\ \wedge s_{\text{out}} \neq s'_{\text{out}} \end{array} \left| \begin{array}{l} g \leftarrow \mathcal{D}_{\Sigma}(\lambda, n) \\ (\mathbb{x}, \pi) \xleftarrow{\text{tr}_{\tilde{\mathcal{P}}}} \tilde{\mathcal{P}}^{\text{D2SQuery}^g} \\ b \xleftarrow{\text{tr}_{\mathcal{V}}} \mathcal{V}^{\text{D2SQuery}^g}(\mathbb{x}, \pi) \end{array} \right. \right] \end{aligned} \quad (34)$$

$$\begin{aligned}
&\leq \frac{|\text{Cache}_p^{<j} \cap \text{tr}^{<j}|}{|\Sigma|^{r+c}} + \frac{|\text{Cache}_p^{<j} \cap \text{tr}^{<j}|}{|\Sigma|^{r+c}} \\
&\leq 2 \cdot \frac{\min\{(j-1)L_v, (j-1)\}}{|\Sigma|^{r+c}} \\
&= \frac{2(j-1)}{|\Sigma|^{r+c}} \quad (\text{since } L_v \geq 1) \\
&\leq \frac{j-1}{|\Sigma|^c},
\end{aligned}$$

since $|\Sigma^{r+c}| \geq 2$.

Conclusion. Wrapping up, we have:

$$\begin{aligned}
&\max \left\{ \Pr \left[E(\text{tr}_{\tilde{p}} \| \text{tr}_v) \mid \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{E}}(\lambda, n) \\ (\mathfrak{x}, \pi) \xleftarrow{\text{tr}_{\tilde{p}}} \tilde{\mathcal{P}}^{h,p,p^{-1}} \\ b \xleftarrow{\text{tr}_v} \mathcal{V}^{h,p}(\mathfrak{x}, \pi) \end{array} \right], \Pr \left[E(\text{tr}_{\tilde{p}} \| \text{tr}_v) \mid \begin{array}{l} g \leftarrow \mathcal{D}_{\Sigma}(\lambda, n) \\ (\mathfrak{x}, \pi) \xleftarrow{\text{tr}_{\tilde{p}}} \tilde{\mathcal{P}}^{\text{D2SQuery}^g} \\ b \xleftarrow{\text{tr}_v} \mathcal{V}^{\text{D2SQuery}^g}(\mathfrak{x}, \pi) \end{array} \right] \right\} \\
&\leq \sum_{j \in [t_h+1+t_p+L+t_{p-1}]} \left(\max \left\{ \Pr[E_{\mathfrak{E},h}^{(j)}], \Pr[E_{\Sigma,h}^{(j)}] \right\} + \max \left\{ \Pr[E_{\mathfrak{E},p}^{(j)}], \Pr[E_{\Sigma,p}^{(j)}] \right\} \right. \\
&\quad \left. + \max \left\{ \Pr[E_{\mathfrak{E},p^{-1}}^{(j)}], \Pr[E_{\Sigma,p^{-1}}^{(j)}] \right\} + \max \left\{ \Pr[E_{\mathfrak{E},\text{func}}^{(j)}], \Pr[E_{\Sigma,\text{func}}^{(j)}] \right\} \right) \quad (\text{by Equation 28}) \\
&\leq \sum_{j \in [t_h+1+t_p+L+t_{p-1}]} \left(\frac{2(j-1)}{|\Sigma|^c} + \frac{2j-1}{|\Sigma|^c} + \frac{2j-1}{|\Sigma|^c} + \frac{j-1}{|\Sigma|^c} \right) \quad (\text{by Eqs. 29 to 34}) \\
&= \frac{7(t_h+1+t_p+L+t_{p-1})^2 - 3(t_h+1+t_p+L+t_{p-1})}{2|\Sigma|^c}. \quad \square
\end{aligned}$$

Definition 5.9. Given a (h, p, p^{-1}) -trace tr , we define $E_{\text{prp}}(\text{tr})$ be the disjunction of the following predicates.

1. $E_{\text{col},p}(\text{tr})$: there exist $(p', s_{\text{in}}, s_{\text{out}}), (p', s'_{\text{in}}, s'_{\text{out}}) \in \bar{\text{tr}}$ such that $s_{\text{in}} \neq s'_{\text{in}}$ and $s_{\text{out}} = s'_{\text{out}}$.
2. $E_{\text{col},p^{-1}}(\text{tr})$: there exist $(p^{-1}, s_{\text{out}}, s_{\text{in}}), (p^{-1}, s'_{\text{out}}, s'_{\text{in}}) \in \bar{\text{tr}}$ with $s_{\text{out}} \neq s'_{\text{out}}$ and $s_{\text{in}} = s'_{\text{in}}$.
3. $E_{\text{col},p,p^{-1}}(\text{tr})$: there exist $(p', s_{\text{in}}, s_{\text{out}}), (p^{-1}, s'_{\text{out}}, s'_{\text{in}}) \in \bar{\text{tr}}$ with $s_{\text{out}} = s'_{\text{out}}$ and $s_{\text{in}} \neq s'_{\text{in}}$.
4. $E_{\text{col},p^{-1},p}(\text{tr})$: there exist $(p^{-1}, s_{\text{out}}, s_{\text{in}}), (p', s'_{\text{in}}, s'_{\text{out}}) \in \bar{\text{tr}}$ with $s_{\text{out}} \neq s'_{\text{out}}$ and $s_{\text{in}} = s'_{\text{in}}$.

Informally, if Item 1 or Item 3 hold, then p is onto and p^{-1} is not a function; if Item 2 or Item 4 hold, then p^{-1} is onto and p is not a function. We rely on the base trace to avoid ambiguity on what query triggered the collision in redundant queries.

Lemma 5.10. Let tr be a (h, p, p^{-1}) -trace. If $E(\text{tr}) = 0$, then $E_{\text{prp}}(\text{tr}) = 0$.

Definition 5.11. Let tr be a (h, p, p^{-1}) -trace and $s \in \Sigma^{r+c}$ be a permutation state. Let $E_{\text{inv}}(\text{tr}, s)$ be the predicate that checks if there is a sequence of permutation states $S^{(k)} \in \mathcal{S}_{\text{BT}}(\text{tr}, s)$ constructed using p^{-1} :

$$\exists J^{(k)} = (j_h^{(k)}, j_0^{(k)}, j_1^{(k)}, \dots, j_{m_k}^{(k)}) \in \mathcal{J}_{\text{BT}}(\text{tr}, s) : \exists \ell \in [0, m_k-1] \text{ s.t. } \text{tr}_{j_\ell^{(k)}} = (p^{-1}, \cdot, \cdot). \quad (36)$$

In particular the above condition has the following two sub-cases (which we refer to later on).

- An inversion after a query to h :

$$\exists J^{(k)} = (j_h^{(k)}, j_0^{(k)}, j_1^{(k)}, \dots, j_{m_k}^{(k)}) \in \mathcal{J}_{\text{BT}}(\text{tr}, \mathbf{s}) : \text{tr}_{j_0^{(k)}} = (p^{-1}, \cdot, \cdot). \quad (37)$$

- An inversion after a query to p :

$$\exists J^{(k)} = (j_h^{(k)}, j_0^{(k)}, j_1^{(k)}, \dots, j_{m_k}^{(k)}) \in \mathcal{J}_{\text{BT}}(\text{tr}, \mathbf{s}) : \exists \iota \in [m_k - 1] \text{ s.t. } \text{tr}_{j_\iota^{(k)}} = (p^{-1}, \cdot, \cdot). \quad (38)$$

Lemma 5.12. Let tr be a (h, p, p^{-1}) -trace and $\mathbf{s} \in \Sigma^{r+c}$ be a permutation state. If $E(\text{tr}) = 0$, then $E_{\text{inv}}(\text{tr}, \mathbf{s}) = 0$.

Definition 5.13. Let tr be a (h, p, p^{-1}) -trace and $\mathbf{s} \in \Sigma^{r+c}$ be a permutation state. Let $E_{\text{fork}}(\text{tr}, \mathbf{s})$ be the predicate that checks if there is a (capacity segment) collision for h or p : $|\mathcal{S}_{\text{BT}}(\text{tr}, \mathbf{s})| > 1$. We distinguish several (exhaustive) special cases of this predicate.

- Collisions of two outputs of h :

$$E_{\text{fork},h}(\text{tr}, \mathbf{s}) := \exists S^{(1)}, S^{(2)} \in \mathcal{S}_{\text{BT}}(\text{tr}, \mathbf{s}) : \mathbf{x}^{(1)} \neq \mathbf{x}^{(2)} \wedge \mathbf{s}_{\text{C},\text{in},0}^{(1)} = \mathbf{s}_{\text{C},\text{in},0}^{(2)}. \quad (39)$$

- Capacity segment collisions of two outputs of p :

$$E_{\text{fork},p}(\text{tr}, \mathbf{s}) := \begin{array}{l} \exists S^{(1)}, S^{(2)} \in \mathcal{S}_{\text{BT}}(\text{tr}, \mathbf{s}), \iota_1 \in [0, m_1 - 1], \iota_2 \in [0, m_2 - 1] : \\ \mathbf{s}_{\text{in},\iota_1}^{(1)} \neq \mathbf{s}_{\text{in},\iota_2}^{(2)} \wedge \mathbf{s}_{\text{C},\text{out},\iota_1}^{(1)} = \mathbf{s}_{\text{C},\text{out},\iota_2}^{(2)} \end{array}. \quad (40)$$

- Collisions of h with the output capacity segment of a query to p :

$$E_{\text{fork},h,p}(\text{tr}, \mathbf{s}) := \exists S^{(1)}, S^{(2)} \in \mathcal{S}_{\text{BT}}(\text{tr}, \mathbf{s}), \iota \in [m_2 - 1] : \mathbf{s}_{\text{C},\text{in},0}^{(1)} = \mathbf{s}_{\text{C},\text{out},\iota}^{(2)}. \quad (41)$$

Lemma 5.14. Let tr be a (h, p, p^{-1}) -trace and $\mathbf{s} \in \Sigma^{r+c}$ be a permutation state. If $E(\text{tr}) = 0$, then $E_{\text{fork}}(\text{tr}, \mathbf{s}) = 0$.

Definition 5.15. $E_{\text{time}}(\text{tr}, \mathbf{s}) := E_{\text{time},h}(\text{tr}, \mathbf{s}) \vee E_{\text{time},p}(\text{tr}, \mathbf{s})$ checks if there is an index list $J^{(k)} \in \mathcal{J}_{\text{BT}}(\text{tr}, \mathbf{s})$ out of order.

- $E_{\text{time},h}(\text{tr}, \mathbf{s})$: the query to h is out of order, i.e.,

$$\exists J^{(k)} = (j_h^{(k)}, j_0^{(k)}, j_1^{(k)}, \dots, j_{m_k}^{(k)}) \in \mathcal{J}_{\text{BT}}(\text{tr}, \mathbf{s}) : j_h^{(k)} > j_0^{(k)}. \quad (42)$$

- $E_{\text{time},p}(\text{tr}, \mathbf{s})$: a query to p is out of order, i.e.,

$$\exists J^{(k)} = (j_h^{(k)}, j_0^{(k)}, j_1^{(k)}, \dots, j_{m_k}^{(k)}) \in \mathcal{J}_{\text{BT}}(\text{tr}, \mathbf{s}) : \exists \iota \in [m_k - 1] \text{ s.t. } j_{\iota-1}^{(k)} > j_\iota^{(k)}. \quad (43)$$

Lemma 5.16. Let tr be a (h, p, p^{-1}) -query answer trace, and $\mathbf{s} \in \Sigma^{r+c}$ be a permutation state. If $E(\text{tr}) = 0$, then $E_{\text{time}}(\text{tr}, \mathbf{s}) = 0$.

5.7 Analysis of aborts

We prove (the contrapositive of) two implications that we use in Section 5.8.

- Theorem 5.17: if StdTrace aborts, then $E(\text{tr})$ holds.
- Theorem 5.18: if $(\tilde{\mathcal{P}} \parallel \mathcal{V})^{\text{D2SQuery}}$ aborts then $E(\text{tr})$ holds.

Lemma 5.17. *For every (h, p, p^{-1}) -trace tr , if $E(\text{tr}) = 0$ then $\text{StdTrace}(\text{tr})$ does not abort.*

Proof. StdTrace aborts in one of the following (mutually-exclusive) cases.

- In Item 4(a)ii, which checks that $\text{BackTrack}(\text{tr}, \text{tr}_\nabla, \mathbf{s}_{\text{in}})$ returns err for an entry $(p', \mathbf{s}_{\text{in}}, \mathbf{s}_{\text{out}})$ in tr . Since $E(\text{tr}) = 0$, then by Theorems 5.10, 5.12 and 5.14 we have $E_{\text{inv}}(\text{tr}, \mathbf{s}) = 0$, $E_{\text{prp}}(\text{tr}) = 0$, and $E_{\text{fork}}(\text{tr}, \mathbf{s}) = 0$, and then by Theorem 5.19 BackTrack does not return err.
- In Item 4(a)ivC, which checks that $\text{LookAhead}(\text{tr}_p^{\text{LA}}, \mathbf{s}_{\text{in}}, i)$ returns err. Since $E(\text{tr}) = 0$, then by Theorem 5.10, $E_{\text{prp}}(\text{tr}) = 0$, and then by Theorem 5.20 LookAhead does not return err. $\square$

Lemma 5.18. *For every $(t_h, t_p, t_{p^{-1}})$ -query algorithm $\mathcal{A}$, let $\text{tr}_{\mathcal{A}}$ be the query-answer trace resulting from the execution of $\mathcal{A}$ with D2SQuery oracle access (cf. Section 5.4). If $E(\text{tr}_{\mathcal{A}}) = 0$ then $\mathcal{A}^{\text{D2SQuery}}$ does not abort.*

Proof. The procedure D2SQuery aborts in Item 4b (cf. Section 5.4), which checks that $\text{BackTrack}(\text{tr}, \text{tr}_\nabla, \mathbf{s})$ returns err. Since $E(\text{tr}) = 0$, then by Theorems 5.10, 5.12 and 5.14 we have $E_{\text{inv}}(\text{tr}, \mathbf{s}) = 0$, $E_{\text{prp}}(\text{tr}) = 0$, and $E_{\text{fork}}(\text{tr}, \mathbf{s}) = 0$, and then by Theorem 5.19 BackTrack does not return err. $\square$

Claim 5.19. *Let tr be a query-answer trace for oracles h, p, p^{-1} , and let tr_∇ be its associated data structure (see Section 5.1). Let $\mathbf{s} \in \Sigma^{r+c}$ be a permutation state. If $E_{\text{inv}}(\text{tr}, \mathbf{s}) = 0$, $E_{\text{prp}}(\text{tr}) = 0$, and $E_{\text{fork}}(\text{tr}, \mathbf{s}) = 0$, then $\text{BackTrack}(\text{tr}, \text{tr}_\nabla, \mathbf{s}) \neq \text{err}$.*

Proof. If $\text{BackTrack}(\text{tr}, \text{tr}_\nabla, \mathbf{s})$ returns err in Item 5, then Outs contains more than one element (see Section 5.2). The size of Outs is bounded by the size of $\mathcal{S}_{\text{BT}}$ (Outs is initialized in Item 1, and new elements are added in Item 4(a)iiiD). We argue that $\mathcal{S}_{\text{BT}}$ contains at most one element by showing that $S^{(1)} = S^{(2)}$ for every $S^{(1)}, S^{(2)} \in \mathcal{S}_{\text{BT}}$.

Consider two elements in $\mathcal{S}_{\text{BT}}$:

$$\begin{aligned} S^{(1)} &= (\mathbf{x}^{(1)}, \mathbf{s}_{\text{in},0}^{(1)}, \mathbf{s}_{\text{out},0}^{(1)}, \mathbf{s}_{\text{in},1}^{(1)}, \mathbf{s}_{\text{out},1}^{(1)}, \dots, \mathbf{s}_{\text{in},m_1-1}^{(1)}, \mathbf{s}_{\text{out},m_1-1}^{(1)}, \mathbf{s}_{\text{in},m_1}^{(1)}) \\ S^{(2)} &= (\mathbf{x}^{(2)}, \mathbf{s}_{\text{in},0}^{(2)}, \mathbf{s}_{\text{out},0}^{(2)}, \mathbf{s}_{\text{in},1}^{(2)}, \mathbf{s}_{\text{out},1}^{(2)}, \dots, \mathbf{s}_{\text{in},m_2-1}^{(2)}, \mathbf{s}_{\text{out},m_2-1}^{(2)}, \mathbf{s}_{\text{in},m_2}^{(2)}) \end{aligned}$$

We note the following:

- $S^{(1)}, S^{(2)}$ have the same end element. $\mathbf{s}_{\text{in},m_1}^{(1)} = \mathbf{s}_{\text{in},m_2}^{(2)} = \mathbf{s}$. This is by construction of $\mathcal{S}_{\text{BT}}(\text{tr}, \mathbf{s})$.
- The two sequences share the last $\min\{m_1, m_2\}$ permutations states.

Let $\iota \in [0, \min\{m_1, m_2\} - 1]$. We show that if $\mathbf{s}_{\text{in},m_1-\iota+1}^{(1)} = \mathbf{s}_{\text{in},m_2-\iota+1}^{(2)}$, then

$$\mathbf{s}_{\text{out},m_1-\iota}^{(1)} = \mathbf{s}_{\text{out},m_2-\iota}^{(2)} \text{ and } \mathbf{s}_{\text{in},m_1-\iota}^{(1)} = \mathbf{s}_{\text{in},m_2-\iota}^{(2)}. \quad (44)$$

By definition of $\mathcal{S}_{\text{BT}}(\text{tr}, \mathbf{s})$, we have that the output capacity segment matches the next input capacity segment, that is, $\mathbf{s}_{\text{C,out},m_1-\iota}^{(1)} = \mathbf{s}_{\text{C,in},m_1-\iota+1}^{(1)} = \mathbf{s}_{\text{C,in},m_2-\iota+1}^{(2)} = \mathbf{s}_{\text{C,out},m_2-\iota}^{(2)}$.

$E(\text{tr}, \mathbf{s}) = 0$ implies that $E_{\text{fork},p}(\text{tr}, \mathbf{s}) = 0$ (that is, there are no collisions in the capacity segment, cf. Equation 40). This implies that the inputs must have been identical, i.e. $\mathbf{s}_{\text{in},m_1-\iota}^{(1)} = \mathbf{s}_{\text{in},m_2-\iota}^{(2)}$. If the inputs are identical, since $E_{\text{prp}}(\text{tr}) = 0$, then the outputs must have been identical, i.e. $\mathbf{s}_{\text{out},m_1-\iota}^{(1)} = \mathbf{s}_{\text{out},m_2-\iota}^{(2)}$.

- The two sequences have the same length, i.e. $m_1 = m_2$.

In fact, since $E_{\text{fork},h,p}(\text{tr}) = 0$ (Equation 41), $\mathbf{s}_{\text{C},\text{in},0}^{(1)} \neq \mathbf{s}_{\text{C},\text{out},\iota}^{(2)}$ for any $\iota > 0$. In particular, we conclude that

$$\mathbf{s}_{\text{in},0}^{(1)} = \mathbf{s}_{\text{in},0}^{(2)}. \quad (45)$$

- $\mathbb{x}^{(1)} = \mathbb{x}^{(2)}$.

Since $\mathbf{s}_{\text{in},0}^{(1)} = \mathbf{s}_{\text{in},0}^{(2)}$, we have that $(\langle h, \mathbb{x}^{(1)}, \mathbf{s}_{\text{C},\text{in},0}^{(1)} \rangle, (\langle h, \mathbb{x}^{(2)}, \mathbf{s}_{\text{C},\text{in},0}^{(1)} \rangle) \in \text{tr}$, which by Equation 39 implies that $\mathbb{x}^{(1)} = \mathbb{x}^{(2)}$.

□

Note that the proof of Theorem 5.19 does not rely on E_{time} (one of the events in E). Later on we use E_{time} to argue that if $\text{BackTrack}(\text{tr}, \text{tr}_{\nabla}, \mathbf{s})$ outputs a tuple $(i, \mathbb{x}, \boldsymbol{\tau}, (\hat{\alpha}_1, \dots, \hat{\alpha}_i))$ then this output remains unchanged even if additional elements are added to tr_h and tr_p .

Claim 5.20. *Let tr be a (h, p, p^{-1}) -trace, and tr_{∇} its corresponding data structure (see Section 5.1). If $E_{\text{prp}}(\text{tr}) = 0$ then, for every $i \in [k]$ and $\mathbf{s} \in \Sigma^{r+c}$, $\text{LookAhead}(\text{tr}_{\nabla} \cdot \mathbf{p}, \mathbf{s}, i) \neq \text{err}$.*

Proof. We prove that, for any $S_{\text{LA}}^{(1)}, S_{\text{LA}}^{(2)} \in \mathcal{S}_{\text{LA}}$ (cf. Equation 14), $S_{\text{LA}}^{(1)} = S_{\text{LA}}^{(2)}$. We have that:

$$\begin{aligned} S_{\text{LA}}^{(1)} &= \left(\mathbf{s}_{\text{in},0}^{(1)}, \mathbf{s}_{\text{out},0}^{(1)}, \mathbf{s}_{\text{in},1}^{(1)}, \mathbf{s}_{\text{out},1}^{(1)}, \dots, \mathbf{s}_{\text{in},m_1-1}^{(1)}, \mathbf{s}_{\text{out},m_1-1}^{(1)} \right), \\ S_{\text{LA}}^{(2)} &= \left(\mathbf{s}_{\text{in},0}^{(2)}, \mathbf{s}_{\text{out},0}^{(2)}, \mathbf{s}_{\text{in},1}^{(2)}, \mathbf{s}_{\text{out},1}^{(2)}, \dots, \mathbf{s}_{\text{in},m_2-1}^{(2)}, \mathbf{s}_{\text{out},m_2-1}^{(2)} \right). \end{aligned}$$

Then:

- $\mathbf{s}_{\text{in},0}^{(1)} = \mathbf{s}_{\text{in},0}^{(2)} = \mathbf{s}$. That is, the two sequences share the same prefix. This is by construction of $\mathcal{S}_{\text{LA}}$.
- For every $\iota \in [0, \min\{m_1, m_2\} - 1]$, if $\mathbf{s}_{\text{in},\iota}^{(1)} = \mathbf{s}_{\text{in},\iota}^{(2)}$ then $\mathbf{s}_{\text{out},\iota}^{(1)} = \mathbf{s}_{\text{out},\iota}^{(2)}$.

Since $E_{\text{prp}}(\text{tr}) = 0$, then for all elements $(\mathbf{s}_{\text{in}}, \mathbf{s}_{\text{out}}), (\mathbf{s}'_{\text{in}}, \mathbf{s}'_{\text{out}}) \in \text{tr}_p$, if $\mathbf{s}_{\text{in}} = \mathbf{s}'_{\text{in}}$, then $\mathbf{s}_{\text{out}} = \mathbf{s}'_{\text{out}}$. In fact:

- For every $(\langle p, \mathbf{s}_{\text{in}}, \mathbf{s}_{\text{out}} \rangle, (\langle p, \mathbf{s}'_{\text{in}}, \mathbf{s}'_{\text{out}} \rangle) \in \text{tr}$, if $\mathbf{s}_{\text{in}} = \mathbf{s}'_{\text{in}}$ then $\mathbf{s}_{\text{out}} = \mathbf{s}'_{\text{out}}$ (cf. Item 1).
- For every $(\langle p^{-1}, \mathbf{s}_{\text{out}}, \mathbf{s}_{\text{in}} \rangle, (\langle p^{-1}, \mathbf{s}'_{\text{out}}, \mathbf{s}_{\text{in}} \rangle) \in \text{tr}$, if $\mathbf{s}_{\text{in}} = \mathbf{s}'_{\text{in}}$ then $\mathbf{s}_{\text{out}} = \mathbf{s}'_{\text{out}}$ (cf. Item 2).
- For every $(\langle p^{-1}, \mathbf{s}_{\text{out}}, \mathbf{s}_{\text{in}} \rangle, (\langle p, \mathbf{s}_{\text{in}}, \mathbf{s}'_{\text{out}} \rangle) \in \text{tr}$, if $\mathbf{s}_{\text{in}} = \mathbf{s}'_{\text{in}}$ then $\mathbf{s}_{\text{out}} = \mathbf{s}'_{\text{out}}$ (cf. Item 4).¹²

Since, by construction of $\mathcal{S}_{\text{LA}}$, for all $\iota > 0$, $\mathbf{s}_{\text{out},\iota-1}^{(1)} = \mathbf{s}_{\text{in},\iota}^{(1)}$ and $\mathbf{s}_{\text{out},\iota-1}^{(2)} = \mathbf{s}_{\text{in},\iota}^{(2)}$, it must be that $S_{\text{LA}}^{(1)} \subseteq S_{\text{LA}}^{(2)}$ or $S_{\text{LA}}^{(2)} \subseteq S_{\text{LA}}^{(1)}$. Since the sequences have maximal length (Item 1e), then $S_{\text{LA}}^{(1)} = S_{\text{LA}}^{(2)}$. Therefore, if $\mathcal{S}_{\text{LA}}$ is not empty, it contains at most one element. In particular, it means that LookAhead does not output err. □

¹²Note that Item 3 is concerned with different input states, which are not relevant here.

5.8 Proof of Theorem 5.1

We finally prove the lemma via a hybrid argument; hybrids are summarized in Figure 4.

Hyb₀. The first hybrid is the left-side experiment in the lemma statement, where the malicious argument prover $\tilde{\mathcal{P}}$ queries the oracles $h: \{0, 1\}^{\leq n} \rightarrow \Sigma^c$ and $p, p^{-1}: \Sigma^{r+c} \rightarrow \Sigma^{r+c}$ sampled from $\mathcal{D}_{\mathfrak{S}}$ (see Theorem 4.2).

Hyb₁. In this hybrid we make the following changes:

(A) in line 1 we sample oracles as

$$\mathbf{g} := (g_i)_{i \in [k]} \leftarrow \mathcal{D}_{\Sigma}(\lambda, n) \quad (46)$$

instead of the oracles $(h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n)$ from Hyb₀;

(B) in line 2 the argument prover is $\tilde{\mathcal{P}}^{\text{D2SQuery}^{\mathbf{g}}}$ instead of $\tilde{\mathcal{P}}^{h,p,p^{-1}}$ from Hyb₀;

(C) in line 3 the argument verifier is $\mathcal{V}^{\text{D2SQuery}^{\mathbf{g}}}$ instead of $\mathcal{V}^{h,p}$ from Hyb₀;

(D) in line 4 the query-answer trace is set to $(\varphi^{-1}, \psi)(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}})$ instead of $\text{D2STrace}(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}})$ from Hyb₀.

Claim 5.21. *The statistical distance between Hyb₀ and Hyb₁ is at most*

$$\frac{7(t_h + 1 + t_p + L + t_{p^{-1}})^2 - 3(t_h + 1 + t_p + L + t_{p^{-1}})}{2|\Sigma|^c}.$$

Proof. We consider the predicate E in Theorem 5.7 in both hybrids:

- E_0 denotes the event that $E(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}})$ holds at the end of Hyb₀;
- E_1 denotes the event that $E(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}})$ holds at the end of Hyb₁.

We argue that

$$\Delta(\text{Hyb}_0 \mid \overline{E_0}, \text{Hyb}_1 \mid \overline{E_1}) = 0. \quad (47)$$

We discuss the hybrid changes in Items (A) to (C), and then Item (D).

- (i) In Hyb₀, the oracle $h: \{0, 1\}^{\leq n} \rightarrow \Sigma^c$ is a random function, $p: \Sigma^{r+c} \rightarrow \Sigma^{r+c}$ a random permutation, and p^{-1} the inverse of p . We argue that, in Hyb₁, $\text{D2SQuery}^{\mathbf{g}}$ answers identically (if E does not hold).

$\text{D2SQuery}^{\mathbf{g}}$ answers queries to h randomly and consistently, as in Hyb₀. The crux of the argument is to show that $\text{D2SQuery}^{\mathbf{g}}$ answers queries to p, p^{-1} as in Hyb₀. For this it suffices to prove the following.

- (a) The (emulated) oracles p, p^{-1} behave as functions (the same inputs yield the same outputs):

$$\forall ('p', s_{\text{in}}, s_{\text{out}}), ('p', s'_{\text{in}}, s'_{\text{out}}) \in \text{tr} : s_{\text{in}} = s'_{\text{in}} \implies s_{\text{out}} = s'_{\text{out}}, \quad (48)$$

$$\forall ('p^{-1}', s_{\text{out}}, s_{\text{in}}), ('p^{-1}', s'_{\text{out}}, s'_{\text{in}}) \in \text{tr} : s_{\text{out}} = s'_{\text{out}} \implies s_{\text{in}} = s'_{\text{in}}. \quad (49)$$

Equation 48 holds by inspection of D2SQuery : new queries to p are stored in $\text{tr}_{\nabla}.p$; duplicate queries are answered consistently via $\text{tr}_{\nabla}.p.\text{inlu}$ (cf. Items 3a, 4(c)ii, 4(d)i and 4(e)ii).

Equation 49 holds by inspection of D2SQuery : new queries to p^{-1} are stored in $\text{tr}_{\nabla}.p$ (cf. Item 3b); duplicate queries are answered consistently via $\text{tr}_{\nabla}.p.\text{outlu}$ (cf. Item 3). Since E does not hold, answers in Cache_p are not in conflict with answers stored in $\text{tr}_{\nabla}.p$ (cf. Equation 27).

- (b) The (emulated) oracles p, p^{-1} are injective functions:

$$\forall ('p', s_{\text{in}}, s_{\text{out}}), ('p', s'_{\text{in}}, s'_{\text{out}}) \in \text{tr} : s_{\text{out}} = s_{\text{out}'} \implies s_{\text{in}} = s'_{\text{in}}, \quad (50)$$

$$\forall ('p^{-1}', s_{\text{out}}, s_{\text{in}}), ('p^{-1}', s'_{\text{out}}, s'_{\text{in}}) \in \text{tr} : s_{\text{in}} = s_{\text{in}'} \implies s_{\text{out}} = s'_{\text{out}}. \quad (51)$$

$E(\text{tr})$ does not hold so $E_{\text{prp}}(\text{tr})$ does not hold (Theorem 5.10), so:

- $E_{\text{col},p}(\text{tr}) = 0$ (cf. Item 1), which implies Equation 50; and
- $E_{\text{col},p^{-1}}(\text{tr}) = 0$ (cf. Item 2), which implies Equation 51.

(c) The (emulated) oracle p^{-1} behaves as the inverse of the oracle p :

$$\forall (s'_p, s_{\text{in}}, s_{\text{out}}), (s'^{-1}_p, s'_{\text{out}}, s'_{\text{in}}) \in \text{tr}: s_{\text{out}} = s'_{\text{out}} \iff s_{\text{in}} = s'_{\text{in}}. \quad (52)$$

$E(\text{tr})$ does not hold so $E_{\text{prp}}(\text{tr})$ does not hold (Theorem 5.10), so $E_{\text{col},p,p^{-1}}(\text{tr}) = 0$ (cf. Item 3) and $E_{\text{col},p^{-1},p}(\text{tr}) = 0$ (cf. Item 4), which implies Equation 52.

(d) The (emulated) oracles p and p^{-1} answer random values (given the above constraints).

- Answers for p^{-1} are sampled in Item 3b uniformly at random.
- Answers for p are sampled in the following cases.
 - If BackTrack outputs a tuple in the image of φ (Item 4e), then the answer comes from the output of g (Item 4(e)i) and possibly extra padding (Item 4(e)iiiA).
 - If BackTrack outputs a tuple not in the image of φ (Item 4d), then the answer is sampled at random (Item 4(d)ii).
 - If BackTrack outputs none and the query is in Cache_p (Item 4(c)i), then the answer was sampled at random in Item 4(e)iiiD for a previous query.
 - If BackTrack outputs none (Item 4(c)ii) and the query is not in Cache_p , then the answer is sampled at random (Item 4(c)iii).
 - Since E does not happen, then BackTrack does not return err (Theorem 5.19).

(ii) We address the change in Item (D) by arguing that $\text{D2STrace}(\text{tr}_{\tilde{p}} \parallel \text{tr}_{\mathcal{V}})$ in Hyb_0 is identically distributed to $(\varphi^{-1}, \psi)(\text{tr}_{\tilde{p}} \parallel \text{tr}_{\mathcal{V}})$ in Hyb_1 . Since $\text{D2STrace} = (\varphi^{-1}, \psi) \circ \text{StdTrace}$, it suffices to argue that these two traces are identically distributed:

- (a) $\text{tr}_{\text{std},0}$, the output of StdTrace invoked on $\text{tr}_{\tilde{p}} \parallel \text{tr}_{\mathcal{V}}$ in Hyb_0 ; and
- (b) $\text{tr}_{\text{std},1}$, the query-answer trace of $(\tilde{\mathcal{P}} \parallel \mathcal{V})^{\text{D2SQuery}}$ given oracle access to g in Hyb_1 .

We argue this by induction on the queries to (h, p, p^{-1}) made by $\tilde{\mathcal{P}}$ and $\mathcal{V}$. The base case, where no oracle query is made, is trivial. For the inductive case, we assume that $\text{tr}_{\text{std},0}$ and $\text{tr}_{\text{std},1}$ are identically distributed in the prefix resulting from the first $j - 1$ oracle queries to (h, p, p^{-1}) , and argue that $\text{tr}_{\text{std},0}$ and $\text{tr}_{\text{std},1}$ are identically distributed in the prefix resulting from an additional query to (h, p, p^{-1}) .

If the j -th query is to h or p^{-1} , then no entries are added to $\text{tr}_{\text{std},0}$ and $\text{tr}_{\text{std},1}$:

- StdTrace does not add any entries to its output tr_{std} (cf. Item 4b); and
- D2SQuery does not perform any query to g (cf. Items 2 and 3).

In this case, the claim follows by the inductive assumption.

If the j -th query is to p then a new entry may be added to $\text{tr}_{\text{std},0}$ and $\text{tr}_{\text{std},1}$, and we have to carefully establish that the distribution of this entry is identical in the two hybrids.

Denote by $s_{\text{in}} \in \Sigma^{r+c}$ the j -th query (which is to p). Both hybrids invoke the deterministic procedure BackTrack (defined in Section 5.2), as follows.

- In Hyb_0 , StdTrace runs BackTrack in Item 4(a)i on (i) a query-answer trace (and its corresponding data structure) resulting from the (entire) execution of $\tilde{\mathcal{P}} \parallel \mathcal{V}$, and (ii) the permutation state s_{in} (the j -th query in this trace).

- In Hyb_1 , D2SQuery runs BackTrack in Item 4a on (i) a query-answer trace (and its corresponding data structure) that contains the first $j - 1$ query-answer pairs made by $\tilde{\mathcal{P}} \parallel \mathcal{V}$, and (ii) the permutation state s_{in} .

Since previous oracle queries and answers are identically distributed, s_{in} is identically distributed.

Despite the different traces provided given as input to BackTrack in the two hybrids, the output of BackTrack is identically distributed: neither E_0 nor E_1 happens, so $E(\text{tr}_{\tilde{\mathcal{P}}} \parallel \text{tr}_{\mathcal{V}}) = 0$ and, in particular, $E_{\text{time}}(\text{tr}_{\tilde{\mathcal{P}}} \parallel \text{tr}_{\mathcal{V}}, s_{\text{in}}) = 0$ by Theorem 5.16. Hence, by Equations 42 and 43, each index list $J^{(k)} \in \mathcal{J}_{\text{BT}}(\text{tr}, s)$ associated to the sequence of permutation states $S^{(k)} \in \mathcal{S}_{\text{BT}}$ computed by BackTrack in Equation 11 is monotonically increasing, with the last index being j . This means that all permutation states used to compute $\mathcal{S}_{\text{BT}}$ appear in the first j -th entries of the query-answer trace $\text{tr}_{\tilde{\mathcal{P}}} \parallel \text{tr}_{\mathcal{V}}$.

The traces $\text{tr}_{\text{std},0}, \text{tr}_{\text{std},1}$ are left unchanged unless BackTrack outputs a tuple $(i, \mathbb{x}, \tau, (\hat{\alpha}_1, \dots, \hat{\alpha}_i))$ such that $\forall \iota \in [i], \hat{\alpha}_\iota \in \text{Im}(\varphi_\iota)$ (see Item 4(a)iv in StdTrace and Item 4e in D2SQuery). In this case, the tuple $(\mathbb{x}, \tau, \hat{\alpha}_1, \dots, \hat{\alpha}_i)$ is appended to $\text{tr}_{\text{std},0}, \text{tr}_{\text{std},1}$ as a “query” for the i -th oracle in $\text{Hyb}_0, \text{Hyb}_1$ respectively. We have argued that the output of BackTrack is identically distributed, so we are left to argue that the answer added to $\text{tr}_{\text{std},0}$ is identically distributed to the answer added to $\text{tr}_{\text{std},1}$.

If there is a query of the form $(i, (\mathbb{x}, \tau, \hat{\alpha}_1, \dots, \hat{\alpha}_i), \cdot)$ already present in $\text{tr}_{\text{std},0}, \text{tr}_{\text{std},1}$, the claim follows by inductive hypothesis:

- in Hyb_0 , StdTrace appends to $\text{tr}_{\text{std},0}$ the tuple $(i, (\mathbb{x}, \tau, \hat{\alpha}_1, \dots, \hat{\alpha}_i), \hat{\rho}_i)$, where $\hat{\rho}_i$ is the same answer as in a previous query (see Item 4(a)ivA);
- in Hyb_1 , D2SQuery appends to $\text{tr}_{\text{std},1}$ the tuple $(i, (\mathbb{x}, \tau, \hat{\alpha}_1, \dots, \hat{\alpha}_i), \hat{\rho}_i)$, where $\hat{\rho}_i$ is the output of g_i , which responds consistently to queries with the same input (see Item 4(e)i).

If the query is fresh:

- StdTrace appends to $\text{tr}_{\text{std},0}$ the tuple $(i, (\mathbb{x}, \tau, \hat{\alpha}_1, \dots, \hat{\alpha}_i), \hat{\rho}_i)$, where $\hat{\rho}_i$ is the output of LookAhead;
- D2SQuery appends to $\text{tr}_{\text{std},1}$ the tuple $(i, (\mathbb{x}, \tau, \hat{\alpha}_1, \dots, \hat{\alpha}_i), \hat{\rho}_i)$ where $\hat{\rho}_i = g_i(\mathbb{x}, \tau, \hat{\alpha}_1, \dots, \hat{\alpha}_i)$.

We claim that $\hat{\rho}_i \in \Sigma^{\ell_{\mathbf{V}}(i)}$ is identically distributed in the two hybrids, by arguing, in each case, that $\hat{\rho}_i$ is sampled at random and that $\hat{\rho}_i$ agrees with the query-answer trace.

- (a) $\hat{\rho}_i$ is sampled at random in Hyb_0 and Hyb_1 .

In Hyb_1 , g is sampled from $\mathcal{D}_{\Sigma}(\lambda, n)$. In Hyb_0 , $\hat{\rho}_i$ is the output of LookAhead (cf. Item 2). Note that, since E does not hold, the output is not err (Theorem 5.20), and, since $(s_{\text{in}}, s_{\text{out}}) \in \text{tr}_p, S_{\text{LA}}^{(k)}$ is not empty and thus the output is not none. Hence, LookAhead’s output is indeed a tuple of elements in $\Sigma^{\ell_{\mathbf{V}}(i)}$ which is uniformly distributed (by construction of LookAhead in Item 2c, and because p is a random permutation sampled from $\mathcal{D}_{\Sigma}(\lambda, n)$).

- (b) $\hat{\rho}_i$ agrees with the query-answer trace (queries to p that follow DS.Squeeze are r -“chunks” of $\hat{\rho}_i$).

Consider all queries to p whose input is s_{in} and output is of the form $s_{\text{out}}^* = (\hat{\rho}_i[0:r], s_{\text{C,out}}^{(0)})$ (for some $s_{\text{C,out}}^{(0)} \in \Sigma^c$) made during the execution of $\tilde{\mathcal{P}} \parallel \mathcal{V}$. There is at least one such query: the j -th query with input s_{in} has associated answer rate segment the first r elements of $\hat{\rho}_i$ in both hybrids (see Item 2c in Hyb_0 and Item 4(e)iiiB in Hyb_1). Since E does not hold in either hybrid, then E_{prp} does not hold either (Theorem 5.10), and all queries with input s_{in} have output s_{out}^* .

The remaining $L_v(i) - 1$ chunks may be present in the query-answer trace. In Hyb_0 , $\hat{\rho}_i$ is constructed to agree with them (cf. Item 1c in LookAhead). In Hyb_1 , D2SQuery stores all chunks in a list Cache_p in Item 4(e)iiiD, and answers them throughout the execution of $\tilde{\mathcal{P}} \parallel \mathcal{V}$ in Item 4(c)i. Since E does not hold, then by Theorem 5.10 E_{prp} does not hold either (in particular, this means that Equation 33 does not hold), and all query-answer pairs stored in Cache_p consistent with the query-answer trace.

Hence, after the j -th query, the element (if any) added to $\text{tr}_{\text{std},0}$ and $\text{tr}_{\text{std},1}$ is identically distributed.

We conclude that

$$\begin{aligned}
& \Delta(\text{Hyb}_0, \text{Hyb}_1) \\
& \leq \Delta(\text{Hyb}_0 \mid \overline{E_0}, \text{Hyb}_1 \mid \overline{E_1}) + \max\{\Pr[E_0], \Pr[E_1]\} \quad (\text{by [CY24, Claim 1.2.10]}) \\
& \leq 0 + \max\{\Pr[E_0], \Pr[E_1]\} \quad (\text{by Equation 47}) \\
& \leq \frac{7(t_h + 1 + t_p + L + t_{p-1})^2 - 3(t_h + 1 + t_p + L + t_{p-1})}{2|\Sigma|^c} \quad (\text{by Theorem 5.8}).
\end{aligned}$$

□

Hyb₂. In this hybrid we make the following changes:

- in line 1 we sample oracles as

$$e := (e_i)_{i \in [k]} \leftarrow \mathcal{U}\left(\{0, 1\}^{\leq n} \times \Sigma^\delta \times \Sigma^{\ell_{\mathbf{P}}(1)} \times \dots \times \Sigma^{\ell_{\mathbf{P}}(i)} \rightarrow \mathcal{M}_{\mathbf{V},i}\right)_{i \in [k]} \quad (53)$$

instead of the oracles g from Hyb_1 (see Equation 46);

- in line 2 the argument prover is $\tilde{\mathcal{P}}^{\text{D2SQuery}^{\psi^{-1} \circ e}}$ instead of $\tilde{\mathcal{P}}^{\text{D2SQuery}^g}$ from Hyb_1 ;
- in line 3 the argument verifier is $\mathcal{V}^{\text{D2SQuery}^{\psi^{-1} \circ e}}$ instead of $\mathcal{V}^{\text{D2SQuery}^g}$ from Hyb_1 ;
- in line 4 the query-answer trace is set to $\varphi^{-1}(\text{tr}_{\tilde{\mathcal{P}}} \parallel \text{tr}_{\mathcal{V}})$ instead of $(\varphi^{-1}, \psi)(\text{tr}_{\tilde{\mathcal{P}}} \parallel \text{tr}_{\mathcal{V}})$ from Hyb_1 .

Claim 5.22. *The statistical distance between Hyb_1 and Hyb_2 is at most*

$$\theta_*(t_h, t_p, t_{p-1}) \cdot \max_{i \in [k]} \varepsilon_{\text{cdc},i}(\lambda, n) + \sum_{i \in [k]} \varepsilon_{\text{cdc},i}(\lambda, n).$$

Proof. The hybrids differ in that in Hyb_1 the oracle g_i outputs an element in $\Sigma^{\ell_{\mathbf{V}}(i)}$, whereas in Hyb_2 the oracle e_i outputs an element in $\mathcal{M}_{\mathbf{V},i}$. The input domain is the same across hybrids, since both g_i and e_i take inputs in $\{0, 1\}^{\leq n} \times \Sigma^\delta \times \Sigma^{\ell_{\mathbf{P}}(1)} \times \dots \times \Sigma^{\ell_{\mathbf{P}}(i)}$. The output domain difference is reconciled in two ways.

- In lines 2 and 3, D2SQuery receives oracle access to $\psi^{-1} \circ e$, which means that, for each $i \in [k]$, outputs of e_i are mapped by the pre-image sampling procedure ψ_i^{-1} from $\mathcal{M}_{\mathbf{V},i}$ to $\Sigma^{\ell_{\mathbf{V}}(i)}$.

Let $i \in [k]$ be a round index. For $x \in \Sigma^{\ell_{\mathbf{V}}(i)}$, let $\bar{x} := \{y \in \Sigma^{\ell_{\mathbf{V}}(i)} \mid \psi(y) = \psi(x)\}$ be the corresponding elements in the quotient space in $\Sigma^{\ell_{\mathbf{V}}(i)}$. The statistical distance between the distributions of the answer to a query to the i -th oracle in Hyb_1 and Hyb_2 is the following:

$$\Delta_{\psi_i^{-1}} := \Delta\left(\mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)}), \psi_i^{-1} \circ \mathcal{U}(\mathcal{M}_{\mathbf{V},i})\right)$$

$$\begin{aligned}
&= \frac{1}{2} \sum_{\mathbf{x} \in \Sigma^{\ell_{\mathbf{V}}(i)}} \left| \frac{1}{|\Sigma|^{\ell_{\mathbf{V}}(i)}} - \Pr[\mathbf{x} = \psi_i^{-1}(\mathbf{y}) \mid \mathbf{y} \leftarrow \mathcal{U}(\mathcal{M}_{\mathbf{V},i})] \right| \\
&= \frac{1}{2} \sum_{\mathbf{x} \in \Sigma^{\ell_{\mathbf{V}}(i)}} \left| \frac{1}{|\Sigma|^{\ell_{\mathbf{V}}(i)}} - \Pr[\psi_i(\mathbf{x}) = \mathbf{y} \mid \mathbf{y} \leftarrow \mathcal{U}(\mathcal{M}_{\mathbf{V},i})] \cdot \Pr[\mathbf{x} = \mathbf{x}' \mid \mathbf{x}' \leftarrow \mathcal{U}(\bar{\mathbf{x}})] \right| \\
&= \frac{1}{2} \sum_{\mathbf{x} \in \Sigma^{\ell_{\mathbf{V}}(i)}} \left| \frac{1}{|\Sigma|^{\ell_{\mathbf{V}}(i)}} - \frac{|\bar{\mathbf{x}}|}{|\Sigma|^{\ell_{\mathbf{V}}(i)}} \cdot \frac{1}{|\bar{\mathbf{x}}|} \right| = 0.
\end{aligned}$$

- In line 4, answers in the query-answer trace $\text{tr}_{\tilde{\mathcal{P}}} \parallel \text{tr}_{\mathcal{V}}$ are not translated: they are left as the answers of the oracles $\mathbf{e} = (e_i)_{i \in [k]}$. In contrast, in Hyb_1 , answers of the oracles $\mathbf{g} = (g_i)_{i \in [k]}$ in the query-answer trace $\text{tr}_{\tilde{\mathcal{P}}} \parallel \text{tr}_{\mathcal{V}}$ are mapped via $\psi = (\psi_i: \Sigma^{\ell_{\mathbf{V}}(i)} \rightarrow \mathcal{M}_{\mathbf{V},i})_{i \in [k]}$.

Fix a round index $i \in [k]$. By Theorem 4.1, for every $i \in [k]$, ψ_i is $\varepsilon_{\text{cdc},i}(\lambda, n)$ -biased (the distributions $\psi_i(\mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)}))$ and $\mathcal{U}(\mathcal{M}_{\mathbf{V},i})$ are $\varepsilon_{\text{cdc},i}(\lambda, n)$ -close in statistical distance). Hence the statistical distance between the distributions of an answer corresponding to the i -th oracle appearing in the query-answer trace in Hyb_1 and Hyb_2 is the following:

$$\Delta_{\psi_i} := \Delta(\psi_i \circ \mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)}), \mathcal{U}(\mathcal{M}_{\mathbf{V},i})) = \varepsilon_{\text{cdc},i}(\lambda, n).$$

Each of the $\theta_{\star}(t_h, t_p, t_{p-1})$ queries made by $\tilde{\mathcal{P}}^{\text{D2SQuery}}$ incurs a statistical error that is at most $\max_{i \in [k]} \{\Delta_{\psi_i} + \Delta_{\psi_i^{-1}}\}$ (a query can be to any of the k oracles). Moreover, the argument verifier $\mathcal{V}$ makes one query to each oracle, incurring a statistical error that is at most $\sum_{i \in [k]} \max\{\Delta_{\psi_i} + \Delta_{\psi_i^{-1}}\}$. We conclude that the statistical distance between Hyb_1 and Hyb_2 is at most

$$\begin{aligned}
&\Delta(\text{Hyb}_1, \text{Hyb}_2) \\
&\leq \theta_{\star}(t_h, t_p, t_{p-1}) \cdot \max_{i \in [k]} \{\Delta_{\psi_i} + \Delta_{\psi_i^{-1}}\} + \sum_{i \in [k]} \max\{\Delta_{\psi_i} + \Delta_{\psi_i^{-1}}\} \\
&= \theta_{\star}(t_h, t_p, t_{p-1}) \cdot \max_{i \in [k]} \varepsilon_{\text{cdc},i}(\lambda, n) + \sum_{i \in [k]} \varepsilon_{\text{cdc},i}(\lambda, n).
\end{aligned} \tag{54}$$

□

Hyb₃. In this hybrid we make the following changes:

- in line 1 we sample oracles as

$$\mathbf{f} := (f_i)_{i \in [k]} \leftarrow \mathcal{U}\left(\left(\{0, 1\}^{\leq n} \times \{0, 1\}^{\delta_{\star}} \times \mathcal{M}_{\mathbf{P},1} \times \cdots \times \mathcal{M}_{\mathbf{P},i} \rightarrow \mathcal{M}_{\mathbf{V},i}\right)_{i \in [k]}\right) \tag{55}$$

instead of the oracles $\mathbf{e}$ from Hyb_2 (see Equation 53);

- in line 2 the argument prover is $\tilde{\mathcal{P}}^{\text{D2SQuery}^{\psi^{-1} \circ f \circ \varphi^{-1}}}$ instead of $\tilde{\mathcal{P}}^{\text{D2SQuery}^{\psi^{-1} \circ \mathbf{e}}}$ from Hyb_2 ;
- in line 3 the argument verifier is $\mathcal{V}^{\text{D2SQuery}^{\psi^{-1} \circ f \circ \varphi^{-1}}}$ instead of $\mathcal{V}^{\text{D2SQuery}^{\psi^{-1} \circ \mathbf{e}}}$ from Hyb_2 ;
- in line 4 the query-answer trace is set to $\text{tr}_{\tilde{\mathcal{P}}} \parallel \text{tr}_{\mathcal{V}}$ instead of $\varphi^{-1}(\text{tr}_{\tilde{\mathcal{P}}} \parallel \text{tr}_{\mathcal{V}})$ from Hyb_2 .

Claim 5.23. *Hyb₂ and Hyb₃ are identically distributed.*

Proof. The hybrids differ in that in Hyb_2 the prover messages remain encoded (the oracle e_i takes inputs in $\{0, 1\}^{\leq n} \times \Sigma^\delta \times \Sigma^{\ell_{\mathbf{P}(1)}} \times \dots \times \Sigma^{\ell_{\mathbf{P}(i)}}$) whereas in Hyb_3 the prover messages are not encoded (the oracle f_i takes inputs in $\{0, 1\}^{\leq n} \times \{0, 1\}^{\delta^*} \times \mathcal{M}_{\mathbf{P},1} \times \dots \times \mathcal{M}_{\mathbf{P},i}$). Note that verifier messages are not encoded in both hybrids (the oracles e_i and f_i both output elements in $\mathcal{M}_{\mathbf{V},i}$). The difference is reconciled in two ways.

- In lines 2 and 3, D2SQuery receives oracle access to $\psi^{-1} \circ f \circ \varphi^{-1}$ (compared to $\psi^{-1} \circ e$ in Hyb_2), which means that, for each $i \in [k]$, inputs of f_i are mapped by the inverse φ_i^{-1} of the encoding map $\varphi_i: \mathcal{M}_{\mathbf{P},i} \rightarrow \Sigma^{\ell_{\mathbf{P}(i)}}$ to elements in $\mathcal{M}_{\mathbf{P},i}$. (Moreover, salt strings in Σ^δ are mapped to their binary representation in $\{0, 1\}^{\delta^*}$.)
- In line 4, the query-answer trace $\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}}$ is left unchanged. In contrast, in Hyb_2 , queries to the oracles $e = (e_i)_{i \in [k]}$ in the query-answer trace $\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}}$ are mapped via the inverse(s) φ^{-1} of $\varphi = (\varphi_i: \mathcal{M}_{\mathbf{P},i} \rightarrow \Sigma^{\ell_{\mathbf{P}(i)}})_{i \in [k]}$, and to (unencoded) prover messages (and salt strings in Σ^δ are mapped to their binary representation in $\{0, 1\}^{\delta^*}$).

The maps $\varphi = (\varphi_i: \mathcal{M}_{\mathbf{P},i} \rightarrow \Sigma^{\ell_{\mathbf{P}(i)}})_{i \in [k]}$ are injective. Similarly, the binary encoding map $\text{bin}(\cdot)$, which is used to encode the salt, is injective. Hence for any input query in

$$\{0, 1\}^{\leq n} \times \{0, 1\}^{\delta^*} \times \mathcal{M}_{\mathbf{P},1} \times \dots \times \mathcal{M}_{\mathbf{P},i},$$

there exists at most one element in

$$\{0, 1\}^{\leq n} \times \Sigma^\delta \times \Sigma^{\ell_{\mathbf{P}(1)}} \times \dots \times \Sigma^{\ell_{\mathbf{P}(i)}}$$

that is associated with it. Therefore, while oracles in Hyb_3 store input queries in another format (using φ^{-1} and bin^{-1}), the malicious prover $\tilde{\mathcal{P}}$ cannot distinguish the two hybrids, since the input-output behavior is identical across Hyb_2 and Hyb_3 . The same holds for the verifier $\mathcal{V}$ in line 3. We conclude that

$$\Delta(\text{Hyb}_2, \text{Hyb}_3) = 0.$$

□

Hyb₄. This hybrid corresponds to the right-side experiment in Theorem 5.1:

- in line 1 we sample oracles $f \leftarrow \mathcal{D}_{\text{IP}}(\lambda, n)$, the same as the oracles f in Hyb_3 (see Equation 55);
- in line 2 the argument prover is run as $\text{D2SAlgo}^f(\tilde{\mathcal{P}})$.
- in line 3 the argument verifier is run as $\mathcal{V}_{\text{std}}^f$ instead of as $\mathcal{V}^{\text{D2SQuery}^{\psi^{-1} \circ f \circ \varphi^{-1}}}$ in Hyb_3 ; and
- in line 4 the query-answer trace is not translated as in Hyb_3 .

Claim 5.24. *The statistical distance between Hyb_3 and Hyb_4 is at most*

$$\frac{7(L+1)(2t_h + 1 + 2t_p + L + 2t_{p-1})}{2|\Sigma|^c} - \frac{5(L+1)}{|\Sigma|^c}.$$

Proof. The argument prover $\text{D2SAlgo}^f(\tilde{\mathcal{P}})$ equals the argument prover $\tilde{\mathcal{P}}^{\text{D2SQuery}^{\psi^{-1} \circ f \circ \varphi^{-1}}}$ in Hyb_3 (see Equation 17). Hence the only potential differences between the hybrids are the argument verifier (line 3) and the query-answer trace (line 4).

Let $E_{\mathcal{V}}$ denote the event in Hyb_3 where $\mathcal{V}^{\text{D2SQuery}}$ aborts but $\tilde{\mathcal{P}}^{\text{D2SQuery}}$ does not. We upper bound the probability of $E_{\mathcal{V}}$ as follows:

$$\begin{aligned}
& \Pr[E_{\mathcal{V}}] \\
&= \Pr \left[E(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}}) \wedge \overline{E(\text{tr}_{\tilde{\mathcal{P}}})} \mid \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}_{\text{IP}}(\lambda, n) \\ (\mathbb{x}, \pi) \xleftarrow{\text{tr}_{\tilde{\mathcal{P}}}} \tilde{\mathcal{P}}^{\text{D2SQuery}^{\psi^{-1} \circ \mathbf{f} \circ \varphi^{-1}}} \\ b \xleftarrow{\text{tr}_{\mathcal{V}}} \mathcal{V}^{\text{D2SQuery}^{\psi^{-1} \circ \mathbf{f} \circ \varphi^{-1}}} \end{array} \right] \\
&\leq \sum_{j=t_h+1+t_p+t_{p-1}+1}^{t_h+1+t_p+L+t_{p-1}} \left(\frac{2(j-1)}{|\Sigma|^c} + \frac{2j-1}{|\Sigma|^c} + \frac{2j-1}{|\Sigma|^c} + \frac{j-1}{|\Sigma|^c} \right) \quad (\text{by Equation 35}) \\
&= \frac{7(L+1)(2t_h+1+2t_p+L+2t_{p-1})}{2|\Sigma|^c} - \frac{5(L+1)}{|\Sigma|^c}.
\end{aligned} \tag{56}$$

Next, we argue that the two hybrids are identically distributed if $E_{\mathcal{V}}$ does not happen:

$$\Delta(\text{Hyb}_3 \mid \overline{E_{\mathcal{V}}}, \text{Hyb}_4) = 0. \tag{57}$$

We discuss changes in lines 1 to 3 in Hyb_4 in turn. Observe that:

- the decision bit of $\mathcal{V}_{\text{std}}^{\mathbf{f}}(\mathbb{x}, \pi)$, where $\pi = (\tau, (\alpha_i)_{i \in [k]})$, equals $\mathbf{V}(\mathbb{x}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]})$ with $\rho_i := f_i(\mathbb{x}, \tau, \alpha_1, \dots, \alpha_i)$ for every $i \in [k]$;
- the decision bit of $\mathcal{V}^{\text{D2SQuery}^{\psi^{-1} \circ \mathbf{f} \circ \varphi^{-1}}}(\mathbb{x}, \pi)$, where $\pi = (\tau, (\alpha_i)_{i \in [k]})$, equals $\mathbf{V}(\mathbb{x}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]})$ with $\rho_1, \dots, \rho_k$ answered by $\text{D2SQuery}^{\psi^{-1} \circ \mathbf{f} \circ \varphi^{-1}}$.

We show that, if $E_{\mathcal{V}}$ does not happen, then $\rho_1, \dots, \rho_k$ are identically distributed (when also given all the other quantities in the experiment). We argue this by induction on the round number $i \in [k]$.

- *Base case:* $i = 1$. The verifier $\mathcal{V}^{\psi_1^{-1} \circ \text{D2SQuery} \circ \varphi_1^{-1}}$ receives as input an instance $\mathbb{x} \in \{0, 1\}^{\leq n}$ and a NARG string $\pi = (\tau, (\alpha_i)_{i \in [k]})$ internally invokes $\mathbf{V}(\mathbb{x}, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]})$ where ρ_1 is generated as follows.
 - Invoke $\text{DS.Start}(\mathbb{x})$, which in turn makes an oracle query to h which will internally run D2SQuery , sample a uniformly-random capacity state in Item 2, and produce the initial state $\text{st}_0 := ((0^r, \mathbf{s}_C), 0, r)$.
 - Run $\text{DS.Absorb}(\text{st}_0, \tau \| \hat{\alpha}_1)$ using the encoded prover message $\hat{\alpha}_1 := \varphi_1(\alpha_1)$. Split the salt and prover message into $L_{\delta} + L_P(1)$ blocks (all in Σ^r except for the last one, which is in $\Sigma^{\leq r}$). Each query to the permutation function will have use a capacity segment that was not previously input (since $E_{\mathcal{V}}$ does not happen), and BackTrack will always return none since Item 4 will satisfy the length of candidates in $\mathcal{S}_{\text{BT}}$. By definition of DS, at the end of DS.Absorb , $\mathcal{V}$ produced capacity states $\mathbf{s}_{\text{in}}, \mathbf{s}_{\text{out}}$ where:
 1. $(\mathbb{x}, \mathbf{s}_{C,0})$ is the only element in tr_h ;
 2. $\mathbf{s}_{\text{in},0} = \mathbf{s}_{C,0}$ (i.e., the first capacity segment is the input query), by construction of DS;
 3. $\forall \iota \in [0, L_{\delta} + L_P(1)]$, $(\mathbf{s}_{\text{in},\iota}, \mathbf{s}_{\text{out},\iota}) \in \text{tr}_p$ (i.e., $\mathbf{s}_{\text{out},\iota} = p(\mathbf{s}_{\text{in},\iota})$ and $\mathbf{s}_{C,\text{out},\iota} = \mathbf{s}_{C,\text{in},\iota+1}$)
- Since $E_{\mathcal{V}}$ does not happen, E_{fork} does not hold, and thus $\mathcal{S}_{\text{BT}}$ contains exactly one element. Denote with st'_1 the state at the end of the Absorb procedure.

- Run $\text{DS.Squeeze}^{\psi_1^{-1} \circ \text{D2SQuery} \circ \varphi_1^{-1}}(\text{st}'_1, \ell_{\mathcal{V}}(i))$ which will internally run BackTrack in Item 4a. The backtracking algorithm will recover a unique sequence of permutation states in $\mathcal{S}_{\text{BT}}$ (see Equation 11), since tr contains $L_{\delta} + L_{\mathbf{P}}(1)$ distinct elements (as E does not happen).

BackTrack outputs $(\mathbb{x}, \tau, \hat{\alpha}_1)$ and query $f_1(\mathbb{x}, \text{bin}(\tau), \varphi_1^{-1}(\hat{\alpha}_1))$ which returns the verifier message ρ_1 . Then, D2SQuery will split $\hat{\rho}_1 := \psi_1(\rho_1)$ into chunks of r elements (except the last one, which is $\leq r$) and store them in a temporary query-answer trace list Cache_p using capacity states sampled uniformly at random (Item 4(e)iiiD). By construction of DS, the squeeze operation will query those exact locations which are then placed in tr (see Item 4(e)i) and $\mathcal{V}$ reconstructs the encoded verifier message $\hat{\rho}_1$ (since $\varphi_1^{-1} \circ \varphi_1$ is the identity).

Therefore, the verifier message is generated as

$$\rho_1 = \psi_1^{-1} \circ \psi_1 \circ f_1(\mathbb{x}, \text{bin}(\tau), \varphi_1^{-1} \circ \varphi_1(\alpha_1)) = \psi_1^{-1} \circ \psi_1 \circ f_1(\mathbb{x}, \tau, \alpha_1)$$

which is identically distributed to $f_1(\mathbb{x}, \tau, \alpha_1)$ by Theorem 3.2.

- *Inductive case.* Assume that $\rho_1, \dots, \rho_{i-1}$ are identically distributed; we show that also ρ_i is identically distributed across Hyb_3 and Hyb_4 . Let st_i be the duplex object resulting after squeezing the $(i-1)$ -th challenge (cf. Item 4d). By inductive hypothesis, the duplex object st'_i after squeezing ρ_{i-1} is identically distributed across hybrids.

The verifier absorbs the i -th prover message via $\text{DS.Absorb}^{\psi^{-1} \circ \text{D2SQuery} \circ \varphi^{-1}}(\text{st}_{i-1}, \hat{\alpha}_i)$ which in turn will make $L_{\mathbf{P}}(i)$ queries to D2SQuery, following **DSFS**. Since $E_{\mathcal{V}}$ does not happen, for each of the queries to p with input s , $E_{\text{fork},p}(\text{tr}, s) = 0$ and thus, during the ι -th query to the permutation oracle p , BackTrack will return none since $\mathcal{S}_{\text{BT}}$ will have only one candidate of length $L_{\delta} + \sum_{k < i} L_{\mathbf{P}}(k) + \iota$, and Item 4 will not satisfy the length requirements.

Afterwards, $\mathcal{V}$ invokes then $\text{DS.Squeeze}^{\psi^{-1} \circ \text{D2SQuery} \circ \varphi^{-1}}(\text{st}'_i, \ell_{\mathcal{V}}(i))$ which will internally run BackTrack. As before, the backtracking algorithm will recover a unique sequence of permutation states in $\mathcal{S}_{\text{BT}}$ (see Equation 11), since E does not happen, and output $(\mathbb{x}, \tau, \hat{\alpha}_1, \dots, \hat{\alpha}_i)$ and set:

$$\rho_i := \psi_i^{-1} \circ \psi_i \circ f_i(\mathbb{x}, \text{bin}(\tau), \varphi_1^{-1} \circ \varphi_1(\alpha_1), \dots, \varphi_i^{-1} \circ \varphi_i(\alpha_i)) = \psi_i^{-1} \circ \psi_i \circ f_i(\mathbb{x}, \tau, \alpha_1, \dots, \alpha_i).$$

which is identically distributed to $f_i(\mathbb{x}, \tau, \alpha_1, \dots, \alpha_i)$ by Theorem 3.2.

We conclude that

$$\begin{aligned} & \Delta(\text{Hyb}_3, \text{Hyb}_4) \\ & \leq \Delta(\text{Hyb}_3 \mid \overline{E_{\mathcal{V}}}, \text{Hyb}_4) + \Pr[E_{\mathcal{V}}] \\ & = 0 + \frac{7(L+1)(2t_h+1+2t_p+L+2t_{p^{-1}})}{2|\Sigma|^c} - \frac{5(L+1)}{|\Sigma|^c} \quad (\text{by Equation 57}) \\ & = \frac{7(L+1)(2t_h+1+2t_p+L+2t_{p^{-1}})}{2|\Sigma|^c} - \frac{5(L+1)}{|\Sigma|^c}. \quad (\text{by Equation 56}) \end{aligned}$$

□

Adding up all intermediate error terms, we obtain:

$$\begin{aligned}
& \Delta \left((b, \mathbb{x}, \pi, \text{tr}) \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathbb{S}}(\lambda, n) \\ (\mathbb{x}, \pi) \xleftarrow{\text{tr}_{\tilde{\mathcal{P}}}} \tilde{\mathcal{P}}^{h, p, p^{-1}} \\ b \xleftarrow{\text{tr}_{\mathcal{V}}} \mathcal{V}^{h, p}(\mathbb{x}, \pi) \\ \text{tr} := \text{D2STrace}(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}}) \end{array} \right. , (b, \mathbb{x}, \pi, \text{tr}) \left| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}_{\text{IP}}(\lambda, n) \\ (\mathbb{x}, \pi_{\tilde{\mathcal{P}}_*}) \xleftarrow{\text{tr}_{\tilde{\mathcal{P}}_*}} \text{D2SAlgo}^{\mathbf{f}}(\tilde{\mathcal{P}}) \\ b \xleftarrow{\text{tr}_{\mathcal{V}_{\text{std}}}} \mathcal{V}_{\text{std}}^{\mathbf{f}}(\mathbb{x}, \pi) \\ \text{tr} := \text{tr}_{\tilde{\mathcal{P}}_*} \| \text{tr}_{\mathcal{V}_{\text{std}}} \end{array} \right. \right) \\
& \leq \frac{7(t_h + 1 + t_p + L + t_{p^{-1}})^2 - 3(t_h + 1 + t_p + L + t_{p^{-1}})}{2|\Sigma|^c} \quad (\text{by Theorem 5.21}) \\
& \quad + \theta_*(t_h, t_p, t_{p^{-1}}) \cdot \max_{i \in [k]} \varepsilon_{\text{cdc}, i}(\lambda, n) + \sum_{i \in [k]} \varepsilon_{\text{cdc}, i}(\lambda, n) \quad (\text{by Theorem 5.22}) \\
& \quad + 0 \quad (\text{by Theorem 5.23}) \\
& \quad + \frac{7(L+1)(2t_h + 1 + 2t_p + L + 2t_{p^{-1}})}{2|\Sigma|^c} - \frac{5(L+1)}{|\Sigma|^c} \quad (\text{by Theorem 5.24}) \\
& \leq \frac{7(t_h + t_p + t_{p^{-1}})^2 + 28(L+1)(t_h + t_p + t_{p^{-1}}) + 14(L+1)^2 - 3(t_h + t_p + t_{p^{-1}}) - 13(L+1)}{2|\Sigma|^c} + \\
& \quad \theta_*(t_h, t_p, t_{p^{-1}}) \cdot \max_{i \in [k]} \varepsilon_{\text{cdc}, i}(\lambda, n) + \sum_{i \in [k]} \varepsilon_{\text{cdc}, i}(\lambda, n).
\end{aligned}$$

1 : $(h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathbb{S}}(\lambda, n)$
 $\mathbf{g} \leftarrow \mathcal{D}_{\Sigma}(\lambda, n)$
 $\mathbf{e} \leftarrow \mathcal{U}\left(\left(\{0, 1\}^{\leq n} \times \Sigma^{\delta} \times \Sigma^{\ell_{\mathbf{P}}(1)} \times \dots \times \Sigma^{\ell_{\mathbf{P}}(i)}\right) \rightarrow \mathcal{M}_{\mathbf{V}, i}\right)_{i \in [k]}$
 $\mathbf{f} \leftarrow \mathcal{U}\left(\left(\{0, 1\}^{\leq n} \times \{0, 1\}^{\delta_*} \times \mathcal{M}_{\mathbf{P}, 1} \times \dots \times \mathcal{M}_{\mathbf{P}, i}\right) \rightarrow \mathcal{M}_{\mathbf{V}, i}\right)_{i \in [k]}$
 $\mathbf{f} \leftarrow \mathcal{D}_{\text{IP}}(\lambda, n)$
2 : $(\mathbb{x}, \pi) \xleftarrow{\text{tr}_{\tilde{\mathcal{P}}_*} \text{tr}_{\tilde{\mathcal{P}}_*}} \tilde{\mathcal{P}}^{h, p, p^{-1}} \xrightarrow{\tilde{\mathcal{P}}^{\text{D2SQuery}}^{\mathbf{g}}} \tilde{\mathcal{P}}^{\text{D2SQuery}}^{\psi^{-1} \circ \mathbf{e}} \xrightarrow{\tilde{\mathcal{P}}^{\text{D2SQuery}}^{\psi^{-1} \circ \mathbf{f} \circ \varphi^{-1}}} \text{D2SAlgo}^{\mathbf{f}}(\tilde{\mathcal{P}})$
3 : $b \xleftarrow{\text{tr}_{\mathcal{V}_{\text{std}}} \text{tr}_{\mathcal{V}_{\text{std}}}} \mathcal{V}^{h, p}(\mathbb{x}, \pi) \xrightarrow{\mathcal{V}^{\text{D2SQuery}}^{\mathbf{g}}} \mathcal{V}^{\text{D2SQuery}}^{\psi^{-1} \circ \mathbf{e}}(\mathbb{x}, \pi) \xrightarrow{\mathcal{V}^{\text{D2SQuery}}^{\psi^{-1} \circ \mathbf{f} \circ \varphi^{-1}}} \mathcal{V}_{\text{std}}^{\mathbf{f}}(\mathbb{x}, \pi)$
4 : $\text{tr} := \text{D2STrace}(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}}) \xrightarrow{(\varphi^{-1}, \psi)(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}})} \varphi^{-1}(\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}}) \xrightarrow{\text{tr}_{\tilde{\mathcal{P}}} \| \text{tr}_{\mathcal{V}}} \text{tr}_{\tilde{\mathcal{P}}_*} \| \text{tr}_{\mathcal{V}_{\text{std}}}$
5 : **return** $(b, \mathbb{x}, \pi, \text{tr})$

Figure 4: Hybrid experiments Hyb_0 , Hyb_1 , Hyb_2 , Hyb_3 , Hyb_4 for Theorem 5.1.

6 Soundness and knowledge soundness

We formally state and prove theorems about soundness and knowledge soundness for our Fiat–Shamir transformation (in particular, the two theorems below formally restate the two bounds in Theorem 1). Let $\text{IP} = (\mathbf{P}, \mathbf{V})$ be a public-coin IP for a relation $\mathcal{R}$. Let Σ be a finite alphabet and cdc be a codec for IP over Σ with bias ε_{cdc} (see Theorem 4.1). Let $\mathcal{D}_\varepsilon$ be an ideal permutation distribution over Σ with capacity $c \in \mathbb{N}$ and rate $r \in \mathbb{N}$ (see Theorem 4.2). For a salt size $\delta \in \mathbb{N}$, let $\text{NARG} := \mathbf{DSFS}[\text{IP}, \text{cdc}, \delta]$ be the non-interactive argument for $\mathcal{R}$ in the $\mathcal{D}_\varepsilon$ -oracle model constructed in Theorem 4.3.

The theorems below rely on the following expressions for a query bound $\theta_\star(t)$, an additive error $\eta_\star(\lambda, t)$, and a privacy parameter $\delta_\star$:

$$\begin{aligned}\theta_\star(t) &:= t, \\ \eta_\star(\lambda, t) &:= \frac{25t^2}{|\Sigma|^c} + t \cdot \max_{i \in [k]} \varepsilon_{\text{cdc}, i} + \sum_{i \in [k]} \varepsilon_{\text{cdc}, i}, \\ \delta_\star &:= \delta \log |\Sigma|.\end{aligned}\tag{58}$$

Theorem 6.1 (soundness). *If IP has state-restoration soundness error $\varepsilon_{\text{IP}}^{\text{sr}}$ (see Theorem 3.14) then NARG has soundness error $\varepsilon_{\text{NARG}}$ (see Theorem 3.5) such that, for every security parameter $\lambda \in \mathbb{N}$, query bound $t \in \mathbb{N}$, and instance size bound $n \in \mathbb{N}$,*

$$\varepsilon_{\text{NARG}}(\lambda, t, n) \leq \varepsilon_{\text{IP}}^{\text{sr}}(\delta_\star, \theta_\star(t), n) + \eta_\star(\lambda, t).$$

Theorem 6.2 (knowledge soundness). *If IP has rewinding state-restoration knowledge soundness error $\kappa_{\text{IP}}^{\text{sr}}$ with extraction time $\mathbf{et}_{\text{IP}}$ (see Theorem 3.16) then NARG has rewinding knowledge soundness error $\kappa_{\text{NARG}}(\lambda, t, n, \delta_{\tilde{\mathcal{P}}}(\lambda, n))$ with extraction time $\mathbf{et}_{\text{NARG}}$ (see Theorem 3.8) such that, for every security parameter $\lambda \in \mathbb{N}$, query bound $t \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$,*

- $\kappa_{\text{NARG}}(\lambda, t, n, \delta_{\tilde{\mathcal{P}}}(\lambda, n)) \leq \kappa_{\text{IP}}^{\text{sr}}(\delta_\star, \theta_\star(t), n, \delta'_{\tilde{\mathcal{P}}}(\lambda, n)) + \eta_\star(\lambda, t)$, and
- $\mathbf{et}_{\text{NARG}}(\lambda, t, n, \delta_{\tilde{\mathcal{P}}}(\lambda, n), \tau_{\tilde{\mathcal{P}}}(\lambda, n)) \leq \mathbf{et}_{\text{IP}}^{\text{sr}}(\delta_\star, \theta_\star(t), n, \delta'_{\tilde{\mathcal{P}}}(\lambda, n), \tau'_{\tilde{\mathcal{P}}}(\lambda, n)) + t^2 \cdot \log t \cdot (n + (r + c) \cdot \log |\Sigma|) + t \cdot (t_{\varphi^{-1}} + t_{\psi} + \delta_\star)$.

Above,

$$\begin{aligned}\delta'_{\tilde{\mathcal{P}}}(\lambda, n) &:= \delta_{\tilde{\mathcal{P}}}(\lambda, n) + \eta_\star(\lambda, t) \\ \tau'_{\tilde{\mathcal{P}}}(\lambda, n) &:= \tau_{\tilde{\mathcal{P}}}(\lambda, n) + t^2 \cdot \log t \cdot (n + (r + c) \cdot \log |\Sigma|) + t \cdot (t_{\varphi^{-1}} + t_{\psi} + \delta_\star).\end{aligned}$$

Moreover, if the IP state-restoration extractor is straightline (see Theorem 3.14) then the NARG extractor is also straightline (see Theorem 3.6). In this case:

- the (straightline) knowledge soundness error is $\kappa_{\text{NARG}}(\lambda, t, n) \leq \kappa_{\text{IP}}^{\text{sr}}(\delta_\star, \theta_\star(t), n) + \eta_\star(\lambda, t)$; and
- the (straightline) extraction time is $\mathbf{et}_{\text{NARG}}(\lambda, t, n) \leq \mathbf{et}_{\text{IP}}^{\text{sr}}(\delta_\star, \theta_\star(t), n) + t^2 \cdot \log t \cdot (n + (r + c) \cdot \log |\Sigma|) + t \cdot (t_{\varphi^{-1}} + t_{\psi} + \delta_\star)$.

The proof of Theorem 6.1 is in Section 6.1, and the proof of Theorem 6.2 is in Section 6.2. Both theorems are proved by using Theorem 5.1 to reduce the security property (soundness or knowledge soundness) to the corresponding property for the basic variant of the Fiat–Shamir transformation (Section 3.6).

6.1 Proof of Theorem 6.1

We reduce the soundness of $(\mathcal{P}, \mathcal{V}) := \mathbf{DSFS}[\text{IP}, \text{cdc}, \delta]$ (Theorem 4.3) to the soundness of $(\mathcal{P}_{\text{std}}, \mathcal{V}_{\text{std}}) := \text{FS}[\text{IP}, \delta_\star]$ (Theorem 3.17). Specifically, by Theorem 5.1, every $(t_h, t_p, t_{p^{-1}})$ -query malicious argument

prover $\tilde{\mathcal{P}}$ for **DSFS**[IP, cdc, δ] can be translated to a $\theta_*(t_h, t_p, t_{p^{-1}})$ -query malicious argument prover $\tilde{\mathcal{P}}_{\text{std}} := \text{D2SAlgo}(\tilde{\mathcal{P}})$ for FS[IP, δ_*] that “behaves the same” up to an additive error $\eta_*(\lambda, (t_h, t_p, t_{p^{-1}}))$. The result then follows from Theorem 3.18, which states that the soundness of FS[IP, δ_*] is determined by the state-restoration soundness of IP. Specifically:

$$\begin{aligned}
& \varepsilon_{\text{NARG}}(\lambda, (t_h, t_p, t_{p^{-1}}), n) \\
&= \Pr \left[\begin{array}{l} |\mathbb{X}| \leq n \\ \wedge \mathbb{X} \notin \mathcal{L}(\mathcal{R}) \\ \wedge \mathcal{V}^{h,p}(\mathbb{X}, \pi) = 1 \end{array} \middle| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n) \\ (\mathbb{X}, \pi) \leftarrow \tilde{\mathcal{P}}^{h,p,p^{-1}} \end{array} \right] \quad (\text{by Theorem 3.5}) \\
&\leq \Pr \left[\begin{array}{l} |\mathbb{X}| \leq n \\ \wedge \mathbb{X} \notin \mathcal{L}(\mathcal{R}) \\ \wedge \mathcal{V}_{\text{std}}^f(\mathbb{X}, \pi) = 1 \end{array} \middle| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}_{\text{IP}}(\lambda, n) \\ (\mathbb{X}, \pi) \leftarrow \text{D2SAlgo}^f(\tilde{\mathcal{P}}) \end{array} \right] + \eta_*(\lambda, (t_h, t_p, t_{p^{-1}})) \quad (\text{by Theorem 5.1}) \\
&\leq \varepsilon_{\text{FS}[\text{IP}, \delta_*]}(\lambda, \theta_*(t_h, t_p, t_{p^{-1}}), n) + \eta_*(\lambda, (t_h, t_p, t_{p^{-1}})) \quad (\text{by Theorem 3.5}) \\
&\leq \varepsilon_{\text{IP}}^{\text{sr}}(\delta_*, \theta_*(t_h, t_p, t_{p^{-1}}), n) + \eta_*(\lambda, (t_h, t_p, t_{p^{-1}})). \quad (\text{by Theorem 3.18})
\end{aligned}$$

Finally, by combining the above derivation with the definition of $\eta_*(\lambda, (t_h, t_p, t_{p^{-1}}))$ in Equation 5 in Theorem 5.1, we obtain the upper bound claimed in the theorem:

$$\begin{aligned}
& \varepsilon_{\text{NARG}}(\lambda, (t_h, t_p, t_{p^{-1}}), n) \\
&\leq \varepsilon_{\text{IP}}^{\text{sr}}(\delta_*, \theta_*(t_h, t_p, t_{p^{-1}}), n) + \eta_*(\lambda, (t_h, t_p, t_{p^{-1}})) \quad (\text{by the above derivation}) \\
&= \varepsilon_{\text{IP}}^{\text{sr}}(\delta_*, \theta_*(t_h, t_p, t_{p^{-1}}), n) \\
&\quad + \frac{7(t_h + t_p + t_{p^{-1}})^2 + 28(L+1)(t_h + t_p + t_{p^{-1}}) + 14(L+1)^2 - 3(t_h + t_p + t_{p^{-1}}) - 13(L+1)}{2|\Sigma|^c} + \\
&\quad \theta_*(t_h, t_p, t_{p^{-1}}) \cdot \max_{i \in [k]} \varepsilon_{\text{cdc}, i}(\lambda, n) + \sum_{i \in [k]} \varepsilon_{\text{cdc}, i}(\lambda, n) \quad (\text{by Equation 5}) \\
&\leq \varepsilon_{\text{IP}}^{\text{sr}}(\delta_*, \theta_*(t), n) + \frac{25t^2}{|\Sigma|^c} + \theta_*(t) \cdot \max_{i \in [k]} \varepsilon_{\text{cdc}, i}(\lambda, n) + \sum_{i \in [k]} \varepsilon_{\text{cdc}, i}(\lambda, n) \quad (\text{since } t_h + t_p + t_{p^{-1}} \leq t) \\
&= \varepsilon_{\text{IP}}^{\text{sr}}(\delta_*, \theta_*(t), n) + \frac{25t^2}{|\Sigma|^c} + t \cdot \max_{i \in [k]} \varepsilon_{\text{cdc}, i} + \sum_{i \in [k]} \varepsilon_{\text{cdc}, i} \\
&= \varepsilon_{\text{IP}}^{\text{sr}}(\delta_*, \theta_*(t), n) + \eta_*(\lambda, t). \quad (\text{by definition of } \eta_*(\lambda, t))
\end{aligned}$$

6.2 Proof of Theorem 6.2

First we describe how to obtain a knowledge extractor $\mathcal{E}$ for $\mathcal{V}$ from the knowledge extractor $\mathcal{E}_{\text{std}}$ for $\mathcal{V}_{\text{std}}$; this additionally relies on the two algorithms D2SAlgo and D2STrace from Theorem 5.1.

Construction 6.3. Let $\mathcal{E}_{\text{std}}$ be the knowledge extractor for $\mathcal{V}_{\text{std}}$. The knowledge extractor $\mathcal{E}$ for $\mathcal{V}$ receives as input an instance $\mathbb{X}$, argument string π , query-answer trace tr of the argument prover $\tilde{\mathcal{P}}$, query-answer trace tr_v of the argument verifier, and (black-box access to) the IP prover $\tilde{\mathcal{P}}$, and works as follows.

$$\mathcal{E}(\mathbb{X}, \pi, \text{tr}, \text{tr}_v, \tilde{\mathcal{P}}):$$

1. Compute the query-answer trace $\text{tr}_{\text{std}} := \text{D2STrace}(\text{tr} \parallel \text{tr}_v)$.
2. Set $\text{tr}_{\text{std},v}$ to be the suffix of tr_{std} of length k .
3. Let $\tilde{\mathcal{P}}_{\text{std}} := \text{D2SAlgo}(\tilde{\mathcal{P}})$.
4. Compute $\mathbb{w} := \mathcal{E}_{\text{std}}(\mathbb{x}, \pi, \text{tr}_{\text{std}}, \text{tr}_{\text{std},v}, \tilde{\mathcal{P}}_{\text{std}})$.
5. Output $\mathbb{w}$.

We reduce the knowledge soundness of $(\mathcal{P}_{\text{std}}, \mathcal{V}_{\text{std}}) := \mathbf{DSFS}[\text{IP}, \text{cdc}, \delta]$ (Theorem 4.3) to the knowledge soundness of $(\mathcal{P}, \mathcal{V}) := \text{FS}[\text{IP}, \delta_\star]$ (Theorem 3.17). Specifically, by Theorem 5.1, every (t_h, t_p, t_{p-1}) -query malicious argument prover $\tilde{\mathcal{P}}$ for $\mathbf{DSFS}[\text{IP}, \text{cdc}, \delta]$ can be translated to a $\theta_\star(t_h, t_p, t_{p-1})$ -query malicious argument prover $\tilde{\mathcal{P}}_{\text{std}} := \text{D2SAlgo}(\tilde{\mathcal{P}})$ for $\text{FS}[\text{IP}, \delta_\star]$ that “behaves the same” up to an additive error $\eta_\star(\lambda, (t_h, t_p, t_{p-1}))$. The result then follows from Theorem 3.19, which states that the knowledge soundness of $\text{FS}[\text{IP}, \delta_\star]$ is determined by the state-restoration knowledge soundness of IP. Specifically:

$$\begin{aligned}
& \kappa_{\text{NARG}}(\lambda, (t_h, t_p, t_{p-1}), n, \delta_{\tilde{\mathcal{P}}}) \\
&= \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge (\mathbb{x}, \mathbb{w}) \notin \mathcal{R} \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathbb{S}}(\lambda, n) \\ (\mathbb{x}, \pi) \xleftarrow{\text{tr}} \tilde{\mathcal{P}}^{h,p,p^{-1}} \\ b \xleftarrow{\text{tr}_v} \mathcal{V}^{h,p}(\mathbb{x}, \pi) \\ \mathbb{w} \leftarrow \mathcal{E}(\mathbb{x}, \pi, \text{tr}, \text{tr}_v, \tilde{\mathcal{P}}) \end{array} \right] \quad (\text{by Theorem 3.8}) \\
&= \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge (\mathbb{x}, \mathbb{w}) \notin \mathcal{R} \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathbb{S}}(\lambda, n) \\ (\mathbb{x}, \pi) \xleftarrow{\text{tr}} \tilde{\mathcal{P}}^{h,p,p^{-1}} \\ b \xleftarrow{\text{tr}_v} \mathcal{V}^{h,p}(\mathbb{x}, \pi) \\ \text{tr}_{\text{std}} := \text{D2STrace}(\text{tr} \parallel \text{tr}_v) \\ \text{tr}_{\text{std},v} \text{ is the suffix of } \text{tr}_{\text{std}} \\ \mathbb{w} \leftarrow \mathcal{E}_{\text{std}}(\mathbb{x}, \pi, \text{tr}_{\text{std}}, \text{tr}_{\text{std},v}, \tilde{\mathcal{P}}_{\text{std}}) \end{array} \right] \quad (\text{by Theorem 6.3}) \\
&\leq \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge (\mathbb{x}, \mathbb{w}) \notin \mathcal{R} \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}_{\text{IP}}(\lambda, n) \\ (\mathbb{x}, \pi) \xleftarrow{\text{tr}} \tilde{\mathcal{P}}_{\text{std}}^{\mathbf{f}} \\ b \xleftarrow{\text{tr}_v} \mathcal{V}_{\text{std}}^{\mathbf{f}}(\mathbb{x}, \pi) \\ \mathbb{w} \leftarrow \mathcal{E}_{\text{std}}(\mathbb{x}, \pi, \text{tr}, \text{tr}_v, \tilde{\mathcal{P}}_{\text{std}}) \end{array} \right] + \eta_\star(\lambda, (t_h, t_p, t_{p-1})) \quad (\text{by Theorem 5.1}) \\
&\leq \kappa_{\text{FS}[\text{IP}, \delta_\star]}(\lambda, \theta_\star(t_h, t_p, t_{p-1}), n, \delta_{\tilde{\mathcal{P}}_{\text{std}}}) + \eta_\star(\lambda, (t_h, t_p, t_{p-1})) \quad (\text{by Theorem 3.8}) \\
&\leq \kappa_{\text{FS}[\text{IP}, \delta_\star]}(\lambda, \theta_\star(t_h, t_p, t_{p-1}), n, \delta_{\tilde{\mathcal{P}}} + \eta_\star(\lambda, (t_h, t_p, t_{p-1}))) + \eta_\star(\lambda, (t_h, t_p, t_{p-1})) \quad (\text{by Theorem 5.1}) \\
&\leq \kappa_{\text{IP}}^{\text{sr}}(\delta_\star, \theta_\star(t_h, t_p, t_{p-1}), n, \delta_{\tilde{\mathcal{P}}} + \eta_\star(\lambda, (t_h, t_p, t_{p-1}))) + \eta_\star(\lambda, (t_h, t_p, t_{p-1})). \quad (\text{by Theorem 3.19})
\end{aligned}$$

Finally, note that $\eta_\star(\lambda, (t_h, t_p, t_{p-1})) \leq \eta_\star(\lambda, t)$ (as argued in Section 6.1).

Next, we discuss the running time of $\mathcal{E}$ on $\tilde{\mathcal{P}}$. The extractor $\mathcal{E}$ runs D2SAlgo on both traces (a bound on the runtime is given in Equation 18), and then emulates the execution of $\mathcal{E}_{\text{std}}$ on $\tilde{\mathcal{P}}_{\text{std}} := \tilde{\mathcal{P}}_{\text{std}}(\tilde{\mathcal{P}})$. We deduce that

$$\begin{aligned}
& \text{et}_{\text{NARG}}(\lambda, (t_h, t_p, t_{p-1}), n, \delta_{\tilde{\mathcal{P}}}, \tau_{\tilde{\mathcal{P}}}) \\
&\leq \text{et}_{\text{IP}}^{\text{sr}}(\delta_\star, \theta_\star(t_h, t_p, t_{p-1}), n, \delta_{\tilde{\mathcal{P}}_{\text{std}}}, \tau_{\tilde{\mathcal{P}}_{\text{std}}}) + \\
&\quad (t_p + t_{p-1})^2 \cdot \log(t_h(t_p + t_{p-1})) \cdot (n + (r + c) \cdot \log |\Sigma|) + (t_p + t_{p-1}) \cdot (t_{\varphi^{-1}} + t_{\psi} + \delta \log |\Sigma|).
\end{aligned}$$

We simplify the expression by using:

$$\begin{aligned} t_h + t_p + t_{p^{-1}} &\leq t, \\ \delta_{\tilde{\mathcal{P}}_{\text{std}}} &\leq \delta_{\tilde{\mathcal{P}}} + \eta_{\star}(\lambda, t), \\ \tau_{\tilde{\mathcal{P}}_{\text{std}}} &\leq \tau_{\tilde{\mathcal{P}}} + t^2 \cdot \log t \cdot (n + (r + c) \cdot \log |\Sigma|) + t \cdot (t_{\varphi^{-1}} + t_{\psi} + \delta_{\star}), \end{aligned}$$

and using Theorem 3.19:

$$\begin{aligned} \mathbf{et}_{\text{NARG}}(\lambda, t, n, \delta_{\tilde{\mathcal{P}}}, \tau_{\tilde{\mathcal{P}}}) &\leq \\ \mathbf{et}_{\text{IP}}^{\text{sr}} \left(\delta_{\star}, \theta_{\star}(t), n, \delta_{\tilde{\mathcal{P}}} + \eta_{\star}(\lambda, t), \tau_{\tilde{\mathcal{P}}} + t^2 \cdot \log t \cdot (n + (r + c) \cdot \log |\Sigma|) + t \cdot (t_{\varphi^{-1}} + t_{\psi} + \delta_{\star}) \right) \\ &\quad + t^2 \cdot \log t \cdot (n + (r + c) \cdot \log |\Sigma|) + t \cdot (t_{\varphi^{-1}} + t_{\psi} + \delta_{\star}). \end{aligned}$$

The straightline case. The above analysis directly specializes to the straightline case. Suppose that $\mathbf{E}^{\text{sr}}$ is a straightline extractor (Theorem 3.14): it is a deterministic algorithm that does not need access to the IP state-restoration prover $\tilde{\mathbf{P}}^{\text{sr}}$. Then, by Theorem 3.19, $\mathcal{E}_{\text{std}}$ is a straightline extractor (Theorem 3.6): it is a deterministic algorithm that does not need access to the argument prover $\tilde{\mathcal{P}}_{\text{std}}$. In turn, $\mathcal{E}$ in Theorem 6.3 is a straightline extractor (Theorem 3.6): it is a deterministic algorithm that does not need access to the argument prover $\tilde{\mathcal{P}}$. In this case, the knowledge soundness error bound does not depend on the failure probability of the prover, and simplifies to

$$\begin{aligned} \kappa_{\text{NARG}}(\lambda, (t_h, t_p, t_{p^{-1}}), n) &\leq \kappa_{\text{IP}}^{\text{sr}}(\delta_{\star}, \theta_{\star}(t_h, t_p, t_{p^{-1}}), n) + \eta_{\star}(\lambda, (t_h, t_p, t_{p^{-1}})) \\ &\leq \kappa_{\text{IP}}^{\text{sr}}(\delta_{\star}, \theta_{\star}(t_h, t_p, t_{p^{-1}}), n) + \eta_{\star}(\lambda, t). \end{aligned}$$

Similarly, the extraction time bound does not depend on the failure probability or running time of the prover, and simplifies to

$$\begin{aligned} \mathbf{et}_{\text{NARG}}(\lambda, (t_h, t_p, t_{p^{-1}}), n) &\leq \mathbf{et}_{\text{IP}}^{\text{sr}}(\delta_{\star}, \theta_{\star}(t_h, t_p, t_{p^{-1}}), n) \\ &\quad + (t_p + t_{p^{-1}})^2 \cdot \log \left(t_h(t_p + t_{p^{-1}}) \right) \cdot (n + (r + c) \cdot \log |\Sigma|) + (t_p + t_{p^{-1}}) \cdot (t_{\varphi^{-1}} + t_{\psi} + \delta \log |\Sigma|). \end{aligned}$$

We simplify the expression using $t_h + t_p + t_{p^{-1}} \leq t$:

$$\mathbf{et}_{\text{NARG}}(\lambda, t, n) \leq \mathbf{et}_{\text{IP}}^{\text{sr}}(\delta_{\star}, \theta_{\star}(t), n) + t^2 \cdot \log t \cdot (n + (r + c) \cdot \log |\Sigma|) + t \cdot (t_{\varphi^{-1}} + t_{\psi} + \delta_{\star}).$$

7 Zero knowledge

We formally state and prove the theorem about zero knowledge for our Fiat–Shamir transformation. Let $\text{IP} = (\mathbf{P}, \mathbf{V})$ be a public-coin IP for a relation $\mathcal{R}$. Let Σ be a finite alphabet and cdc be a codec for IP over Σ with bias ε_{cdc} (see Theorem 4.1). Let $\mathcal{D}_{\Sigma}$ be an ideal permutation distribution over Σ with capacity $c \in \mathbb{N}$ and rate $r \in \mathbb{N}$ (see Theorem 4.2). For a salt size $\delta \in \mathbb{N}$, let $\text{NARG} := \mathbf{DSFS}[\text{IP}, \text{cdc}, \delta]$ be the non-interactive argument for $\mathcal{R}$ in the $\mathcal{D}_{\Sigma}$ -oracle model constructed in Theorem 4.3.

Theorem 7.1. *If IP has honest-verifier zero-knowledge error z_{IP} (see Theorem 7.3) then NARG has zero-knowledge error z_{NARG} (see Theorem 7.4) such that, for every security parameter $\lambda \in \mathbb{N}$, query bound $t \in \mathbb{N}$, and instance size bound $n \in \mathbb{N}$,*

$$z_{\text{NARG}}(\lambda, t, n) \leq z_{\text{IP}}(n) + \frac{t}{|\Sigma|^{\min\{\delta, c\}}} + \frac{t \cdot \sum_{i \in [k]} \lceil \ell_{\mathbf{V}}(i)/r \rceil}{|\Sigma|^{r+c}} + \max_{i \in [k]} \varepsilon_{\text{cdc}, i}(\lambda, n).$$

The definitions of honest-verifier zero-knowledge for IPs and zero-knowledge for non-interactive arguments are in Section 7.1. The simulator is in Section 7.2. The analysis of the simulator, establishing the theorem, is in Section 7.3.

7.1 Definitions for zero knowledge

Definition 7.2. *The IP verifier’s view in $\text{IP} = (\mathbf{P}, \mathbf{V})$ on the instance-witness pair $(\mathbb{x}, \mathbb{w})$, denoted $\text{View}_{\text{IP}}(\mathbf{P}, \mathbf{V}, \mathbb{x}, \mathbb{w})$, is the random variable $(\mathbb{x}, \rho, (\alpha_i)_{i \in [k]})$ where:*

- ρ is a random choice of randomness for the IP verifier $\mathbf{V}$; and
- $(\alpha_i)_{i \in [k]}$ are the prover messages received in an interaction between $\mathbf{P}(\mathbb{x}, \mathbb{w})$ and $\mathbf{V}(\mathbb{x}, \rho)$.

Note that the honest IP prover $\mathbf{P}(\mathbb{x}, \mathbb{w})$ may use its own private randomness (and is not part of the IP verifier’s view). If the IP is public-coin then the view shows each round’s randomness: $(\mathbb{x}, (\rho_i)_{i \in [k]}, (\alpha_i)_{i \in [k]})$.

Definition 7.3. $\text{IP} = (\mathbf{P}, \mathbf{V})$ for a relation $\mathcal{R}$ has **honest-verifier zero-knowledge error** z_{IP} if there exists a polynomial-time probabilistic algorithm $\mathbf{S}$ such that for every instance-witness pair $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ the following random variables are $z_{\text{IP}}(\mathbb{x})$ -close in statistical distance:

$$\text{View}_{\text{IP}}(\mathbf{P}, \mathbf{V}, \mathbb{x}, \mathbb{w}) \text{ and } \mathbf{S}(\mathbb{x}).$$

We additionally define $z_{\text{IP}}(n) := \max_{\substack{(\mathbb{x}, \mathbb{w}) \in \mathcal{R} \\ |\mathbb{x}| \leq n}} z_{\text{IP}}(\mathbb{x})$.

Definition 7.4. A non-interactive argument $\text{NARG} = (\mathbf{P}, \mathbf{V})$ for a relation $\mathcal{R}$ has **(adaptive) zero-knowledge error** z_{NARG} (in the EPROM) if there exists a probabilistic polynomial-time simulator $\mathcal{S}$ such that, for every security parameter $\lambda \in \mathbb{N}$, query bound $t \in \mathbb{N}$, t -query admissible adversary $\mathcal{A}$, and instance bound $n \in \mathbb{N}$, the following two distributions are $z_{\text{NARG}}(\lambda, t, n)$ -close in statistical distance:

$$\mathcal{D}_{\text{real}} := \left\{ \text{out} \left| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \leftarrow \mathcal{A}^{\mathbf{f}} \\ \pi \leftarrow \mathcal{P}^{\mathbf{f}}(\mathbb{x}, \mathbb{w}) \\ \text{out} \leftarrow \mathcal{A}^{\mathbf{f}}(\text{aux}, \pi) \end{array} \right. \right\} \text{ and } \mathcal{D}_{\text{sim}} := \left\{ \text{out} \left| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \leftarrow \mathcal{A}^{\mathbf{f}} \\ (\pi, \mu) \leftarrow \mathcal{S}^{\mathbf{f}}(\mathbb{x}) \\ \text{out} \leftarrow \mathcal{A}^{\mathbf{f}[\mu]}(\text{aux}, \pi) \end{array} \right. \right\}.$$

Above, $\mathcal{A}$ is admissible if it always outputs $\mathbb{x}, \mathbb{w}$ such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ and $|\mathbb{x}| \leq n$. With $\mathbf{f}[\mu]$, we denote the function $\mathbf{f}$ modified to be consistent with the query-answer list μ .

In the simulated-world experiment (the one on the right), the simulator has access to the random oracle, and “programs” the random oracle via a list μ of query-answer pairs.

7.2 The simulator

We describe the simulator $\mathcal{S}$ that we use to establish zero knowledge for NARG.

Construction 7.5. *The simulator is an algorithm $\mathcal{S}^{h,p}(\mathbb{x})$ that works as follows. Below we denote by $\mathbf{S}$ the honest-verifier zero-knowledge simulator of IP (see Theorem 7.3).*

$\mathcal{S}^{h,p}(\mathbb{x})$:

1. Sample a simulated view of the IP verifier: $(\mathbb{x}, (\rho_i)_{i \in [k]}, (\alpha_i)_{i \in [k]}) \leftarrow \mathbf{S}(\mathbb{x})$.
2. Initialize the sponge state with the instance: $\text{st}'_0 := \text{DS.Start}^h(\mathbb{x})$.
3. Sample a random salt $\tau \in \Sigma^\delta$.
4. Absorb the salt: $\text{st}_0 := \text{DS.Absorb}^p(\text{st}'_0, \tau)$.
5. For $i = 1, \dots, k$:
 - (a) Encode the prover message: $\hat{\alpha}_i := \varphi_i(\alpha_i)$.
 - (b) Absorb the encoded prover message: $\text{st}'_i := \text{DS.Absorb}^p(\text{st}_{i-1}, \hat{\alpha}_i)$.
 - (c) Sample an encoded verifier message: $\hat{\rho}_i \leftarrow \psi_i^{-1}(\rho_i)$.
 - (d) Sample $(\mu_{p,i}, \mu_{p^{-1},i}, \text{st}_i) \leftarrow \text{ProgramBlocks}(i, \text{st}'_i, \hat{\rho}_i)$ (see below).
6. Set the argument string $\pi := (\tau, (\alpha_i)_{i \in [k]})$.
7. Let μ_h be the empty list. (The oracle h is not programmed.)
8. Set μ_p to be the concatenation of the query-answer lists $(\mu_{p,i})_{i \in [k]}$.
9. Set $\mu_{p^{-1}}$ to be the concatenation of the query-answer lists $(\mu_{p^{-1},i})_{i \in [k]}$.
10. Set $\mu := (\mu_h, \mu_p, \mu_{p^{-1}})$ to be the list of all programmed locations for the oracles.
11. Output (π, μ) .

Note that $\mathcal{S}$ does not query the inverse permutation p^{-1} and does not program h .

Programming blocks. The auxiliary procedure `ProgramBlocks` used in Theorem 7.5 outputs the permutation blocks to be programmed: it receives as input the round index $i \in [k]$, a sponge state $\text{st} \in \Sigma^{r+c} \times [0, r] \times [0, r]$, and a message $\hat{\rho}_i \in \Sigma^{\ell_{\mathbf{V}}(i)}$, and outputs a query-answer list $\mu_{p,i}$ for the permutation oracle p , a corresponding query-answer list $\mu_{p^{-1},i}$ for the inverse permutation oracle p^{-1} , and a new sponge state st_i corresponding to the sponge state for the next round.

`ProgramBlocks`($i, \text{st}, \hat{\rho}_i$):

1. Set $L_{\mathbf{V}}(i) := \left\lceil \frac{\ell_{\mathbf{V}}(i)}{r} \right\rceil$.
2. Parse the sponge state st as a tuple $((s_{R,0}, s_{C,0}), i_A, i_S)$.
3. For every $j \in [L_{\mathbf{V}}(i)]$, sample random $s_{C,j} \leftarrow \mathcal{U}(\Sigma^c)$.
4. Sample random $z_i \leftarrow \mathcal{U}(\Sigma^{r-(\ell_{\mathbf{V}}(i) \bmod r)})$.
5. Parse $\hat{\rho}_i \| z_i$ into rage segments $s_{R,1}, s_{R,2}, \dots, s_{R,L_{\mathbf{V}}(i)} \in \Sigma^r$.
6. Set $\mu_{p,i} := ((s_{R,i-1}, s_{C,i-1}), (s_{R,i}, s_{C,i}))_{i \in [L_{\mathbf{V}}(i)]}$ and $\mu_{p^{-1},i} := ((s_{R,i}, s_{C,i}), (s_{R,i-1}, s_{C,i-1}))_{i \in [L_{\mathbf{V}}(i)]}$.
7. Set $\text{st}_i := ((s_{R,L_{\mathbf{V}}(i)}, s_{C,L_{\mathbf{V}}(i)}), i_A, i_S)$ where $i_A := r$ and $i_S := \ell_{\mathbf{V}}(i) \bmod r$.
8. Return $(\mu_{p,i}, \mu_{p^{-1},i}, \text{st}_i)$.

7.3 Proof of Theorem 7.1

We prove the theorem via a hybrid argument; hybrids are summarized in Figure 5. Throughout we consider an (admissible) adversary $\mathcal{A}$ that makes at most t_h queries to h , at most t_p queries to p , and at most $t_{p^{-1}}$ queries

to p^{-1} . We show that the statistical distance between $\mathcal{D}_{\text{real}}$ and $\mathcal{D}_{\text{sim}}$, by adding the errors across all hybrids, is at most

$$z_{\text{IP}}(n) + \frac{t_p + t_{p^{-1}}}{|\Sigma|^{\min\{\delta, c\}}} + \frac{(t_p + t_{p^{-1}}) \cdot \sum_{i \in [k]} \lceil \ell_{\mathbf{v}}(i)/r \rceil}{|\Sigma|^{r+c}} + \max_{i \in [k]} \varepsilon_{\text{cdc}, i}(\lambda, n).$$

The bound in the theorem statement follows from the fact that $t_h + t_p + t_{p^{-1}} \leq t$.

Hyb₀. This is the distribution $\mathcal{D}_{\text{real}}$. The argument prover is displayed in Figure 5 in the non-boxed lines. To simplify hybrid exposition, the argument prover also computes the last IP verifier message ρ_k (even though it does not use it).

Hyb₁. In this hybrid we program the permutation p . We alter line 11 and, instead of using DS to produce encoded verifier messages, we sample fresh $\hat{\rho}_1 \leftarrow \mathcal{U}(\Sigma^{\ell_{\mathbf{v}}(1)}), \dots, \hat{\rho}_k \leftarrow \mathcal{U}(\Sigma^{\ell_{\mathbf{v}}(k)})$ uniformly at random and use the procedure ProgramBlocks (see line 6) to obtain query-answer lists used to program the oracles p and p^{-1} . Overall, $L_{\mathbf{v}} := \sum_{i \in [k]} L_{\mathbf{v}}(i) = \sum_{i \in [k]} \lceil \ell_{\mathbf{v}}(i)/r \rceil$ locations are programmed.

Claim 7.6. *The statistical distance between Hyb₀ and Hyb₁ is at most*

$$\frac{t_p + t_{p^{-1}}}{|\Sigma|^{\min\{\delta, c\}}} + \frac{(t_p + t_{p^{-1}}) \cdot \sum_{i \in [k]} \lceil \ell_{\mathbf{v}}(i)/r \rceil}{|\Sigma|^{r+c}}.$$

We prove the claim by showing that Hyb₀ and Hyb₁ are perfectly indistinguishable conditioned on a certain bad event not happening, and and upper bounding the probability of this bad event.

Consider the following two predicates.

- E_{start} . For a query-answer trace tr and permutation state $\mathbf{s}_0 \in \Sigma^{r+c}$, we define the predicate

$$E_{\text{start}}(\text{tr}, \mathbf{s}_0) := “\exists \mathbf{s}'_1 \in \Sigma^{r+c}: (\langle p, \mathbf{s}_0, \mathbf{s}'_1 \rangle \in \text{tr} \vee \langle p^{-1}, \mathbf{s}'_1, \mathbf{s}_0 \rangle \in \text{tr})”. \quad (59)$$

- E_{squeeze} . For a query-answer trace tr and permutation states $\mathbf{s}_1, \dots, \mathbf{s}_{L_{\mathbf{v}}} \in \Sigma^{r+c}$, we define the predicate

$$E_{\text{squeeze}}(\text{tr}, (\mathbf{s}_1, \dots, \mathbf{s}_{L_{\mathbf{v}}})) := “\exists \iota \in [L_{\mathbf{v}} - 1], \mathbf{s}_{\text{out}} \in \Sigma^{r+c}: \mathbf{s}_{\text{out}} \neq \mathbf{s}_{\iota+1} \wedge (\langle p, \mathbf{s}_{\iota}, \mathbf{s}_{\text{out}} \rangle \in \text{tr})”. \quad (60)$$

Next, consider the following two events.

- In Hyb₀, E_0 is the (bad) event that $E_{\text{start}}(\text{tr}, \mathbf{s}_0) \vee E_{\text{squeeze}}(\text{tr}, (\mathbf{s}_1, \dots, \mathbf{s}_{L_{\mathbf{v}}}))$ holds, where:
 - tr is the query-answer trace of $\mathcal{A}^{h, p, p^{-1}}$ at the end of line 2;
 - $\mathbf{s}_0$ is the permutation state in st_0 after absorbing the salt τ at the end of line 5; and
 - $\mathbf{s}_1, \dots, \mathbf{s}_{L_{\mathbf{v}}}$ are the permutations states in the encoded verifier messages $\hat{\rho}_1, \dots, \hat{\rho}_k$ produced by DS.Squeeze ^{p} in line 11 (across all k rounds).
- In Hyb₁, E_1 is the (bad) event that $E_{\text{start}}(\text{tr}, \mathbf{s}_0) \vee E_{\text{squeeze}}(\text{tr}, (\mathbf{s}_1, \dots, \mathbf{s}_{L_{\mathbf{v}}}))$ holds, where:
 - tr is as above (tr is the same in Hyb₁ and Hyb₀);
 - $\mathbf{s}_0$ is as above ($\mathbf{s}_0$ is the same in Hyb₁ and Hyb₀);
 - $\mathbf{s}_1, \dots, \mathbf{s}_{L_{\mathbf{v}}}$ are the permutations states contained in the query-answer lists $(\mu_{p, i})_{i \in [k]}$ and $(\mu_{p^{-1}, i})_{i \in [k]}$ produced by ProgramBlocks in line 11 (across all k rounds).

We argue that $\Pr[E_0] \leq \Pr[E_1]$. Observe that:

- E_{start} is identical across Hyb_0 and Hyb_1 ; moreover,
- E_{squeeze} never holds in Hyb_0 because there is no programming (p is a function so for every $(p', s_\iota, s_{\text{out}})$ and $(p', s'_\iota, s'_{\text{out}})$ in tr it holds that $s_\iota = s'_\iota \implies s_{\text{out}} = s'_{\text{out}}$).

Therefore:

$$\begin{aligned}
& \Pr[E_0] \\
& \leq \Pr \left[E_{\text{start}}(\text{tr}, s_0) \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_\mathbb{E}(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^\delta) \\ \text{st}'_0 := \text{DS.Start}^h(\mathbb{x}) \\ \text{st}_0 := \text{DS.Absorb}^p(\text{st}'_0, \tau) \end{array} \right. \right] \\
& \quad + \Pr \left[E_{\text{squeeze}}(\text{tr}, (s_1, \dots, s_{L_V})) \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_\mathbb{E}(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^\delta) \\ \text{st}'_0 := \text{DS.Start}^h(\mathbb{x}) \\ \text{st}_0 := \text{DS.Absorb}^p(\text{st}'_0, \tau) \\ \text{for } i = 1, \dots, k \\ \quad (\alpha_i, \text{aux}_i) := \mathbf{P}(\mathbb{x}, \mathbb{w}) \text{ if } i = 1 \text{ else } \mathbf{P}(\text{aux}_{i-1}, \rho_{i-1}) \\ \quad \hat{\alpha}_i := \varphi_i(\alpha_i) \\ \quad \text{st}'_i := \text{DS.Absorb}^p(\text{st}_{i-1}, \hat{\alpha}_i) \\ \quad (\hat{\rho}_i, \text{st}'_i) := \text{DS.Squeeze}^p(\text{st}'_i, \ell_V(i)); \quad \rho_i := \psi_i(\hat{\rho}_i) \end{array} \right. \right] \\
& = \Pr \left[E_{\text{start}}(\text{tr}, s_0) \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_\mathbb{E}(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^\delta) \\ \text{st}'_0 := \text{DS.Start}^h(\mathbb{x}) \\ \text{st}_0 := \text{DS.Absorb}^p(\text{st}'_0, \tau) \end{array} \right. \right] + 0 \\
& \leq \Pr[E_1].
\end{aligned}$$

Next, we argue that $\text{Hyb}_0 \mid \overline{E_0}$ and $\text{Hyb}_1 \mid \overline{E_1}$ have the same distribution:

- In Hyb_0 , there is no programming, and the permutation p and its inverse p^{-1} answer random and consistent elements in Σ^{r+c} .
- In Hyb_1 , the permutation states $s_1, \dots, s_{L_V}$ are uniformly and independently sampled in ProgramBlocks . The event E_1 does not hold, so for every $\iota \in [0, L_V - 1]$ it holds that $(p', s_\iota, s_{\text{out}}) \in \text{tr} \implies s_{\text{out}} = s_{\iota+1}$, which implies that p 's answers are consistent.

We deduce that

$$\begin{aligned}
\Delta(\text{Hyb}_0, \text{Hyb}_1) & \leq \Delta(\text{Hyb}_0 \mid \overline{E_0}, \text{Hyb}_1 \mid \overline{E_1}) + \max\{\Pr[E_0], \Pr[E_1]\} \quad (\text{by [CY24, Claim 1.2.10]}) \\
& = 0 + \Pr[E_1] \quad (\text{argued above}) \\
& = \Pr[E_1].
\end{aligned}$$

We are left to upper bound the probability of E_1 . We do this by separately upper bounding the probability of E_{start} in Theorem 7.7 and of E_{squeeze} in Theorem 7.8, which implies the claimed upper bound:

$$\frac{t_p + t_{p^{-1}}}{|\Sigma|^{\min\{\delta, c\}}} + \frac{(t_p + t_{p^{-1}}) \cdot \sum_{i \in [k]} \lceil \ell_{\mathbf{v}}(i)/r \rceil}{|\Sigma|^{r+c}}.$$

Claim 7.7. *The following holds:*

$$\Pr \left[E_{\text{start}}(\text{tr}, \mathbf{s}_0) \mid \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^{\delta}) \\ \text{st}'_0 := \text{DS.Start}^h(\mathbb{x}) \\ \text{st}_0 := \text{DS.Absorb}^p(\text{st}'_0, \tau) \end{array} \right] \leq \frac{t_p + t_{p^{-1}}}{|\Sigma|^{\min\{\delta, c\}}}.$$

Proof. Let $L_{\delta} := \lceil \delta/r \rceil$ be the number of blocks in the salt string τ . We distinguish two cases.

- *Case 1:* $L_{\delta} = 1$ (i.e. $\delta \leq r$). The permutation state $\mathbf{s}_0$ is $(\tau \| 0^{r-\delta}, h(\mathbb{x})) \in \Sigma^{r+c}$. Therefore:

$$\begin{aligned} & \Pr \left[E_{\text{start}}(\text{tr}, \mathbf{s}_0) \mid \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^{\delta}) \\ \text{st}'_0 := \text{DS.Start}^h(\mathbb{x}) \\ \text{st}_0 := \text{DS.Absorb}^p(\text{st}'_0, \tau) \end{array} \right] \\ & \leq \Pr \left[(\cdot, \mathbf{s}_0, \cdot) \in \text{tr} \mid \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^{\delta}) \\ \mathbf{s}_0 := (\tau \| 0^{r-\delta}, h(\mathbb{x})) \end{array} \right] + \Pr \left[(\cdot, \mathbf{s}_0, \cdot) \in \text{tr} \mid \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^{\delta}) \\ \mathbf{s}_0 := (\tau \| 0^{r-\delta}, h(\mathbb{x})) \end{array} \right] \\ & \leq \frac{t_p}{|\Sigma|^{\delta}} + \frac{t_{p^{-1}}}{|\Sigma|^{\delta}}. \end{aligned}$$

- *Case 2:* $L_{\delta} > 1$ (i.e. $\delta > r$). We split the analysis of this case into two parts: there exists a sequence of query-answers to p that match an absorb of the salt τ , or no sequence matching the salt can be found.

Let $\mathbf{s}_{C, -L_{\delta}+1} := h(\mathbb{x}) \in \Sigma^c$. Parse $\tau \| 0^{r-(\delta \bmod r)}$ into the sequence of L_{δ} rate segments $(\mathbf{s}_{R, -L_{\delta}+1} \| \dots \| \mathbf{s}_{R, 0})$. We consider the following statement: there exist capacity segments $\mathbf{s}_{C, -L_{\delta}+2}, \dots, \mathbf{s}_{C, 0} \in \Sigma^c$ such that

$$\forall j \in \{-L_{\delta} + 1, \dots, 0\}: \quad \bigvee \begin{array}{l} (\cdot, (\mathbf{s}_{R, j-1}, \mathbf{s}_{C, j-1}), (\mathbf{s}_{R, j}, \mathbf{s}_{C, j})) \in \text{tr} \\ (\cdot, (\mathbf{s}_{R, j}, \mathbf{s}_{C, j}), (\mathbf{s}_{R, j+1}, \mathbf{s}_{C, j+1})) \in \text{tr} \end{array} \quad (61)$$

Then:

- (a) If Equation 61 holds:

$$\Pr \left[E_{\text{start}}(\text{tr}, \mathbf{s}_0) \mid \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^{\delta}) \\ \text{st}'_0 := \text{DS.Start}^h(\mathbb{x}) \\ \text{st}_0 := \text{DS.Absorb}^p(\text{st}'_0, \tau) \end{array} \right]$$

$$\begin{aligned}
&\leq \Pr \left[\begin{array}{l} \forall j \in \{-L_\delta + 1, \dots, 0\}: \\ ('p', (s_{R,j-1}, \cdot), (s_{R,j}, \cdot)) \in \text{tr} \end{array} \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_\ominus(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h,p,p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^\delta) \\ \text{parse } \tau \| 0^{r-(\delta \bmod r)} \text{ into blocks } s_{R,-L_\delta+1}, \dots, s_{R,0} \end{array} \right. \right] \\
&\quad + \Pr \left[\begin{array}{l} \forall j \in \{-L_\delta + 1, \dots, 0\}: \\ ('p^{-1}', (s_{R,j}, \cdot), (s_{R,j-1}, \cdot)) \in \text{tr} \end{array} \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_\ominus(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h,p,p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^\delta) \\ \text{parse } \tau \| 0^{r-(\delta \bmod r)} \text{ into blocks } s_{R,-L_\delta+1}, \dots, s_{R,0} \end{array} \right. \right] \\
&= \left(1 - \left(1 - \frac{1}{|\Sigma|^\delta} \right)^{t_p} \right) + \left(1 - \left(1 - \frac{1}{|\Sigma|^\delta} \right)^{t_{p^{-1}}} \right) \\
&\leq \frac{t_p}{|\Sigma|^\delta} + \frac{t_{p^{-1}}}{|\Sigma|^\delta}. \quad (\text{by Bernoulli's inequality})
\end{aligned}$$

(b) Else (Equation 61 does not hold), let

$$j^* := \arg \max_{j \in \{-L_\delta+1, \dots, 0\}} : \begin{array}{l} ('p', (s_{R,j-1}, s_{C,j-1}), (s_{R,j}, s_{C,j})) \notin \text{tr} \\ \wedge ('p^{-1}', (s_{R,j}, s_{C,j}), (s_{R,j-1}, s_{C,j-1})) \notin \text{tr} \end{array} \quad (62)$$

In this case, the capacity state s_{C,j^*+1} is sampled uniformly at random from Σ^c (see line 5), and has been correctly guessed from the adversary. That is,

$$\begin{aligned}
&\Pr \left[E_{\text{start}}(\text{tr}, s_0) \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_\ominus(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h,p,p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^\delta) \\ \text{st}'_0 := \text{DS.Start}^h(\mathbb{x}) \\ \text{st}_0 := \text{DS.Absorb}^p(\text{st}'_0, \tau) \end{array} \right. \right] \\
&\leq \Pr \left[\begin{array}{l} \exists j^* \text{ as per Equation 62} \\ \wedge ('p', \cdot, (\cdot, s_{C,j^*})) \in \text{tr} \end{array} \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_\ominus(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h,p,p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^\delta) \\ s_{C,-L_\delta+1} := h(\mathbb{x}) \\ s_{C,-L_\delta+2}, \dots, s_{C,0} \leftarrow \mathcal{U}(\Sigma^c) \\ (s_{R,-L_\delta+1} \| \dots \| s_{R,0}) := (\tau \| 0^{r-(\delta \bmod r)}) \end{array} \right. \right] \\
&\quad + \Pr \left[\begin{array}{l} \exists j^* \text{ as per Equation 62} \\ \wedge ('p^{-1}', \cdot, (\cdot, s_{C,j^*})) \in \text{tr} \end{array} \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_\ominus(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h,p,p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^\delta) \\ s_{C,-L_\delta+1} := h(\mathbb{x}) \\ s_{C,-L_\delta+2}, \dots, s_{C,0} \leftarrow \mathcal{U}(\Sigma^c) \\ (s_{R,-L_\delta+1} \| \dots \| s_{R,0}) := (\tau \| 0^{r-(\delta \bmod r)}) \end{array} \right. \right] \\
&\leq \frac{t_p}{|\Sigma|^c} + \frac{t_{p^{-1}}}{|\Sigma|^c}.
\end{aligned}$$

Putting together all above (mutually-exclusive) cases, we conclude that:

$$\Pr \left[E_{\text{start}}(\text{tr}, s_0) \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n) \\ (\mathfrak{x}, \mathfrak{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ \tau \leftarrow \mathcal{U}(\Sigma^\delta) \\ \text{st}'_0 := \text{DS.Start}^h(\mathfrak{x}) \\ \text{st}_0 := \text{DS.Absorb}^p(\text{st}'_0, \tau) \end{array} \right. \right] \\ \leq \max \left\{ \frac{t_p}{|\Sigma|^\delta} + \frac{t_{p^{-1}}}{|\Sigma|^\delta}, \frac{t_p}{|\Sigma|^\delta} + \frac{t_{p^{-1}}}{|\Sigma|^\delta}, \frac{t_p}{|\Sigma|^c} + \frac{t_{p^{-1}}}{|\Sigma|^c} \right\} = \frac{t_p + t_{p^{-1}}}{|\Sigma|^{\min\{\delta, c\}}}.$$

□

Claim 7.8. *The following holds:*

$$\Pr \left[E_{\text{squeeze}}(\text{tr}, (s_1, \dots, s_{L_V})) \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n) \\ (\mathfrak{x}, \mathfrak{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ s_1, \dots, s_{L_V} \leftarrow \mathcal{U}(\Sigma^{r+c}) \end{array} \right. \right] \leq (t_p + t_{p^{-1}}) \cdot \frac{L_V}{|\Sigma|^{r+c}}.$$

Proof. The adversary $\mathcal{A}$ makes at most $t_p, t_{p^{-1}}$ queries to the oracles p, p^{-1} respectively, and has to guess any of the L_V states that have been sampled uniformly at random. In other words:

$$\Pr \left[E_{\text{squeeze}}(\text{tr}, (s_1, \dots, s_{L_V})) \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n) \\ (\mathfrak{x}, \mathfrak{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ s_1, \dots, s_{L_V} \leftarrow \mathcal{U}(\Sigma^{r+c}) \end{array} \right. \right] \\ = \Pr \left[\exists \iota \in [L_V - 1], s_{\text{out}} \in \Sigma^{r+c} : \begin{array}{l} s_{\text{out}} \neq s_{\iota+1} \\ \wedge (\text{'p'}, s_\iota, s_{\text{out}}) \in \text{tr} \end{array} \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n) \\ (\mathfrak{x}, \mathfrak{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ s_1, \dots, s_{L_V} \leftarrow \mathcal{U}(\Sigma^{r+c}) \end{array} \right. \right] \\ \quad \quad \quad \text{(by Equation 60)} \\ \leq \Pr \left[\exists \iota \in [L_V - 1] : (\text{'p'}, s_\iota, \cdot) \in \text{tr} \left| \begin{array}{l} (h, p, p^{-1}) \leftarrow \mathcal{D}_{\mathfrak{S}}(\lambda, n) \\ (\mathfrak{x}, \mathfrak{w}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h, p, p^{-1}} \\ s_1, \dots, s_{L_V} \leftarrow \mathcal{U}(\Sigma^{r+c}) \end{array} \right. \right] \\ \leq (t_p + t_{p^{-1}}) \left(1 - \left(1 - \frac{1}{|\Sigma|^{r+c}} \right)^{L_V} \right) \\ \leq (t_p + t_{p^{-1}}) \cdot \frac{L_V}{|\Sigma|^{r+c}}.$$

□

Hyb₂. In this hybrid we modify line 11 (relative to Hyb₁). In each round $i \in [k]$, we sample $\hat{\rho}_i \in \Sigma^{\ell_V(i)}$ in a different way: sample $\hat{\rho}'_i \leftarrow \mathcal{U}(\Sigma^{\ell_V(i)})$, set $\rho_i := \psi_i(\hat{\rho}'_i)$, and sample $\hat{\rho}_i \leftarrow \psi_i^{-1}(\rho_i)$.

Claim 7.9. *Hyb₁ and Hyb₂ are perfectly indistinguishable.*

Proof. We argue that, for every $i \in [k]$, the distributions of $\hat{\rho}_i \in \Sigma^{\ell_{\mathbf{V}}(i)}$ in Hyb_2 and in Hyb_3 are identical. By definition of statistical distance, we have:

$$\begin{aligned} & \Delta \left(\mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)}), (\psi_i^{-1} \circ \psi_i \circ \mathcal{U})(\Sigma^{\ell_{\mathbf{V}}(i)}) \right) \\ &= \sum_{\mathbf{x} \in \Sigma^{\ell_{\mathbf{V}}(i)}} \left| \Pr \left[\mathbf{x}' = \mathbf{x} \mid \mathbf{x}' \leftarrow \mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)}) \right] - \Pr \left[\mathbf{x}'' = \mathbf{x} \mid \begin{array}{l} \mathbf{x}' \leftarrow \mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)}) \\ y := \psi_i(\mathbf{x}') \\ \mathbf{x}'' \leftarrow \psi_i^{-1}(y) \end{array} \right] \right| \\ &= 0. \end{aligned} \quad (\text{by Theorem 3.2})$$

□

Hyb₃. In this hybrid we modify line 11 (relative to Hyb_2). In each round $i \in [k]$, we sample $\hat{\rho}_i \in \Sigma^{\ell_{\mathbf{V}}(i)}$ in a different way: sample a verifier message $\rho_i \leftarrow \mathcal{U}(\mathcal{M}_{\mathbf{V},i})$ and set $\hat{\rho}_i \leftarrow \psi_i^{-1}(\rho_i)$.

Claim 7.10. *The statistical distance between Hyb_3 and Hyb_2 is at most $\sum_{i \in [k]} \varepsilon_{\text{cdc},i}(\lambda, n)$.*

Proof. The IP prover uses the verifier messages $(\rho_i)_{i \in [k]}$ to compute its messages $(\alpha_i)_{i \in [k]}$. (The last verifier message ρ_k does not affect the prover messages.) Observe that:

- In Hyb_2 , $(\rho_i)_{i \in [k]}$ are decodings of random encoded verifier messages: $\rho_i := \psi_i(\hat{\rho}_i)$ for $\hat{\rho}_i \leftarrow \mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)})$.
- In Hyb_3 , $(\rho_i)_{i \in [k]}$ are random verifier messages: $\rho_i \leftarrow \mathcal{U}(\mathcal{M}_{\mathbf{V},i})$.

The claim follows from the fact that each ψ_i is $\varepsilon_{\text{cdc},i}(\lambda, n)$ -biased. □

Hyb₄. In this hybrid we replace the IP prover $\mathbf{P}$ with the IP simulator $\mathbf{S}$. Instead of computing the prover messages $(\alpha_i)_{i \in [k]}$ using the IP prover $\mathbf{P}$ as in hybrid Hyb_3 (line 8), we compute $(\alpha_i)_{i \in [k]}$ using the IP simulator $\mathbf{S}$ (line 6); the verifier messages sampled in line 11 are replaced with the simulated verifier messages from line 6. By inspection, this hybrid corresponds to the distribution $\mathcal{D}_{\text{sim}}$.

Claim 7.11. *The statistical distance between Hyb_4 and Hyb_3 is at most $z_{\text{IP}}(n)$.*

Proof. The statistical distance is the distance between real and simulated messages, hence is at most $z_{\text{IP}}(n)$ by the definition of honest-verifier zero knowledge for IPs (Theorem 7.3). □

```

1 :  $(h, p, p^{-1}) \leftarrow \mathcal{D}_\Theta(\lambda, n)$ 
2 :  $(\mathbb{X}, \mathbb{W}, \text{aux}) \xleftarrow{\text{tr}} \mathcal{A}^{h,p,p^{-1}}$ 
3 :  $\tau \leftarrow \mathcal{U}(\Sigma^\delta)$ 
4 :  $\text{st}'_0 := \text{DS.Start}^h(\mathbb{X})$ 
5 :  $\text{st}_0 := \text{DS.Absorb}^p(\text{st}'_0, \tau)$ 
6 :  $(\mathbb{X}, (\rho_i)_{i \in [k]}, (\alpha_i)_{i \in [k]}) \leftarrow \mathbf{S}(\mathbb{X})$ 
7 : for  $i = 1, \dots, k$ 
8 :  $(\alpha_i, \text{aux}_i) := \mathbf{P}(\mathbb{X}, \mathbb{W})$  if  $i = 1$  else  $\mathbf{P}(\text{aux}_{i-1}, \rho_{i-1})$ 
9 :  $\hat{\alpha}_i := \varphi_i(\alpha_i)$ 
10 :  $\text{st}'_i := \text{DS.Absorb}^p(\text{st}_{i-1}, \hat{\alpha}_i)$ 
11 :  $(\hat{\rho}_i, \text{st}'_i) := \text{DS.Squeeze}^p(\text{st}'_i, \ell_{\mathbf{V}}(i)); \quad \rho_i := \psi_i(\hat{\rho}_i)$ 
    $\hat{\rho}_i \leftarrow \mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)}); \rho_i := \psi_i(\hat{\rho}_i); \quad (\mu_{p,i}, \mu_{p^{-1},i}, \text{st}_i) := \text{ProgramBlocks}(i, \text{st}'_i, \hat{\rho}_i)$ 
    $\hat{\rho}'_i \leftarrow \mathcal{U}(\Sigma^{\ell_{\mathbf{V}}(i)}); \rho_i := \psi_i(\hat{\rho}'_i); \hat{\rho}_i \leftarrow \psi_i^{-1}(\rho_i); (\mu_{p,i}, \mu_{p^{-1},i}, \text{st}_i) := \text{ProgramBlocks}(i, \text{st}'_i, \hat{\rho}_i)$ 
    $\rho_i \leftarrow \mathcal{U}(\mathcal{M}_{\mathbf{V},i}); \hat{\rho}_i \leftarrow \psi_i^{-1}(\rho_i); \quad (\mu_{p,i}, \mu_{p^{-1},i}, \text{st}_i) := \text{ProgramBlocks}(i, \text{st}'_i, \hat{\rho}_i)$ 
    $\hat{\rho}_i \leftarrow \psi_i^{-1}(\rho_i); \quad (\mu_{p,i}, \mu_{p^{-1},i}, \text{st}_i) := \text{ProgramBlocks}(i, \text{st}'_i, \hat{\rho}_i)$ 
12 :  $\mu_h := ()$ 
13 :  $\mu_p := \mu_{p,1} \parallel \dots \parallel \mu_{p,k}$ 
14 :  $\mu_{p^{-1}} := \mu_{p^{-1},1} \parallel \dots \parallel \mu_{p^{-1},k}$ 
15 :  $\pi := (\tau, (\alpha_i)_{i \in [k]})$ 
16 :  $y \leftarrow \mathcal{A}^{(h,p,p^{-1})}(\text{aux}, \pi) \quad \mathcal{A}^{(h,p,p^{-1})}[\mu_h, \mu_p, \mu_{p^{-1}}](\text{aux}, \pi)$ 

```

Figure 5: Hybrid experiments Hyb_0 , Hyb_1 , Hyb_2 , Hyb_3 , Hyb_4 for Theorem 7.1.

8 Implementation

We implemented **DSFS**[IP, cdc, δ] (Theorem 4.3) in Rust as an open-source library called

`spongef`ish (duplex sponge Fiat–Shamir),

released under a BSD-3-Clause license and available at github.com/arkworks-rs/spongef.

The library is type-safe and relies on Rust’s traits, using generics, associated types, and trait bounds to offer a generic implementation that behaves consistently across different permutation functions.

8.1 Core library and software stack

The core of our library, illustrated as a software stack in Figure 6, consists of the following components.

- A trait `Permutation<U: Unit>` defines the permutation state, of size `Permutation::R` (the rate) plus `Permutation::C` (the capacity), in units `U`. The trait is publicly exposed, so that one can use arbitrary permutation functions. As examples, we provide a Keccak implementation that relies on the existing keccak crate, and a Poseidon implementation that relies on the arkworks library. Moreover, hash function designers can implement their own permutation functions (without having to re-implement the duplex sponge construction and the corresponding duplex-sponge Fiat–Shamir transformation).
- A trait `Unit` defines the alphabet Σ for the permutation (and hash function). The only requirements for `Unit` are: constant size known at compile time (`Sized`), copy (`Clone`), secure deletion (via `zeroize::Zeroize`), and serialization/deserialization (via functions `write` and `read`).
- A `DuplexSponge<P: Permutation>` implementation is given, which implements the duplex construction in overwrite mode as described in Theorem 3.3, based on a given arbitrary permutation `P`.

On top of the duplex sponge implementation, we build two structures to implement the argument prover and argument verifier in Theorem 4.3.

- A `ProverState<P: Permutation>` structure that contains the NARG prover state. This structure allows absorbing/squeezing units, and internally updating the argument string. It contains: (i) the current sponge state, (ii) the NARG string serialized so far, and (iii) the private coins of the argument prover.
- A `VerifierState<P: Permutation>` structure that implements the NARG verifier. This structure takes as input an argument string, and during the verifier execution, it deserializes the string into prover messages (as units), and feeds them into the duplex sponge to derive the IP verifier messages.

We exploit some of Rust’s type safety features to make the library resilient against misuse. For instance, it is not possible to clone the prover state during the execution. Another example: by design, the NARG string contains only those prover messages that have been included in the Fiat–Shamir transformation.

Codecs. Via *extension traits*¹³ we add the ability for prover and verifier to absorb and squeeze from arbitrary domains. In particular, a submodule `codecs` implements the following.

- An extension `FieldToUnit<F>` that provides an encoding map φ translating prover messages from a field `F` into units `U` (the permutation alphabet). Supported field types are either in `ark-ff` (within `arkworks-rs/algebra`) or in `ff` (within `zkcrypto/ff`).

¹³Extension traits are a programming pattern that enables adding methods to an existing type outside of the crate defining that type.

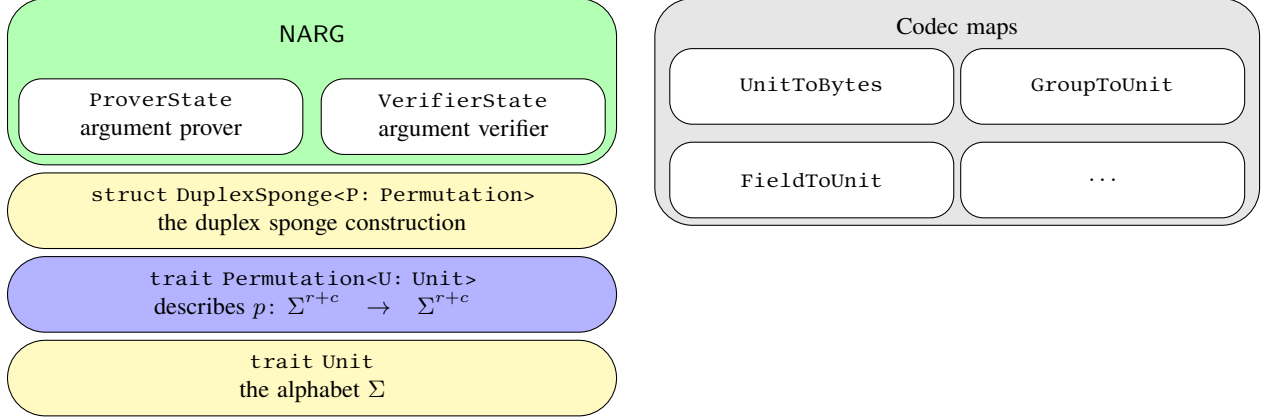


Figure 6: Overview of the software stack in our library.

- An extension `UnitToField<F>` that provides a negligibly-biased map ψ that sends uniformly-distributed units into (almost) uniformly-distributed elements of a field F following Theorem C.1. The extension supports units that are bytes ($U=u8$) and field elements ($U=F$). The extension is implemented for `arkworks` and `zkcrypto/ff`.
- An extension `GroupToUnit<G>` that provides an encoding map φ that compresses and serializes elliptic curve group elements into units. We support elliptic curves types from `arkworks-rs/algebra` and `zkcrypto/ff`.
- An extension `UnitToBytes` that allows to map units (squeezed from the duplex sponge) into bytes. In the case of binary units the implementation is trivial, as it relies on squeezing field native units. In the case of algebraic permutations, we follow Theorem C.1 to ensure that the squeezed IP verifier messages are negligibly close to uniformly distributed ones.

8.2 Concrete security and ergonomics

We discuss some design choices and trade-offs in security and usability.

Byte-level interface. Cryptographic libraries typically expose cryptographic objects as “opaque” byte payloads (e.g., see PKCS#11 and the NaCL cryptography library). We take a similar approach in our library, and let `ProverState` build the argument string, without having to worry about serialization/deserialization of the NARG string. On the other end, `VerifierState` reads incrementally the IP prover messages from the NARG string recovering the IP messages and recomputing the IP verifier messages in the correct sequence. This is beneficial for two reasons: (i) the argument string object is guaranteed to contain all messages that have been absorbed and to include those messages in the correct round;¹⁴ (ii) serialization is internally made by the library, offloading the burden of delicately re-mapping verifier messages without introducing noticeable biases, in a way that can be used also by other libraries.

Session identifiers. The focus of this paper is single-theorem (knowledge) soundness and zero-knowledge. Real-world protocols, however, demand more complex security notions, and the gold standard for security is *universal composability* [Can01]. For protocols that rely on random oracles, this means establishing universal composability in (a suitable flavor of) the global random oracle model [CDGLN18]. zkSNARKs based on the

¹⁴In most implementations today, the prover returns a “NARG string” that contains all prover messages, but ensuring their inclusion in the Fiat–Shamir transformation is left to the programmer.

BCS transformation [BCS16] are known to be UC-secure [CF24] and, since the Fiat–Shamir transformation is a building block of the BCS transformation (see Section 2.6), we believe it is plausible that **DSFS** can also be shown to satisfy universal composability (given suitable properties for the underlying IP) in a suitably defined *global random permutation model*. While the formal study of the UC-security of **DSFS** is left to future work (see Section 1.3), our library includes features intended to facilitate composability.

Specifically UC-security in the global random oracle model demands domain separation of the oracle by way of session identifiers. There is a natural place to put such information in **DSFS**: the initialization of the sponge state via the oracle h , which can be pre-processed and chosen separately from the permutation oracle p used during the online phase of the protocol. Specifically, the initialization step $st'_0 := \text{DS.Start}^h(\mathbb{x})$ in Theorem 4.3 is replaced by $st'_0 := \text{DS.Start}^h(\text{sid} \parallel \mathbb{x})$ where sid is a fixed-length string that can be viewed as (the hash of) a session identifier. Our library provides an additional structure `DomainSeparator` that wraps the `String` type and exposes helper functions for embedding additional information within the initial oracle call. This information is then hashed using Keccak to produce sid .

The user is responsible for choosing a session identifier sid that is unique, for example, including: (i) a protocol identifier, which uniquely determines the non-interactive argument including information such as the codec and the hash functions used; (ii) a context identifier, providing information such as a timestamp and a URL identifying the resource in which the proof is being used; (iii) a serialization of the relation being proved.

Private randomness of the argument prover. The argument prover `ProverState` uses `DuplexSponge<U>` as part of **DSFS** in order to derive the IP verifier messages. In addition, `ProverState` uses a separate duplex sponge `DuplexSponge<u8>` for its own randomness: to derive the salt string $\tau \in \Sigma^\delta$ (included in the argument string) and to derive the private randomness of the IP prover (if any, and typically used for zero knowledge). This latter duplex sponge is seeded by operating system randomness and, optionally, external randomness provided by the user. (This helps in constrained environments lacking a source of high-quality randomness.)

A On indifferntiability

In Section A.1 we discuss oracle implementations and indifferntiability [MRH04]. In Section A.2 we discuss how different flavors of indifferntiability affect the properties of a non-interactive argument (including the limitations of the standard flavor). A comprehensive discussion of how different flavors of indifferntiability affect the security properties of a non-interactive argument can be found in [CY24].

A.1 Oracle constructions and indifferntiability

We define *oracle constructions* (Theorem A.1) and *indifferntiability* (Theorem A.2). We also define a notion of *strong indifferntiability* (Theorem A.3), and show that it is implied via a certain loss (Theorem A.4). In Theorem A.5 we describe how to capture **DSFS** via the abstraction of oracle constructions.

Below $\mathcal{D}$ and $\mathcal{D}_\diamond$ are oracle distributions (families of distributions indexed by a security parameter $\lambda \in \mathbb{N}$ and an instance size bound $n \in \mathbb{N}$).

Definition A.1. A $(\mathcal{D}, \mathcal{D}_\diamond)$ -**construction** is a probabilistic stateful algorithm $\mathcal{C}$ that works as follows: $\mathcal{C}$ receives as input a security parameter $\lambda \in \mathbb{N}$ and instance size bound $n \in \mathbb{N}$ and oracle access to functions $g \in \mathcal{D}_\diamond(\lambda, n)$, and answers queries with inputs and outputs consistent with functions $f \in \mathcal{D}(\lambda, n)$.

Definition A.2. A $(\mathcal{D}, \mathcal{D}_\diamond)$ -construction $\mathcal{C}$ is **indifferntiable with error** ε_{ind} if there exists a probabilistic stateful algorithm $\mathcal{S}_\mathcal{C}$ such that for every t -query T -time algorithm D :

$$\left| \Pr \left[D^{\mathcal{C}^g, g} = 1 \mid g \leftarrow \mathcal{D}_\diamond(\lambda, n) \right] - \Pr \left[D^{f, \mathcal{S}_\mathcal{C}^f} = 1 \mid f \leftarrow \mathcal{D}(\lambda, n) \right] \right| \leq \varepsilon_{\text{ind}}(\lambda, n, t, T).$$

Definition A.3. A $(\mathcal{D}, \mathcal{D}_\diamond)$ -construction $\mathcal{C}$ is **strongly indifferntiable with error** ε_{ind} if there exists a probabilistic stateful algorithm $\mathcal{S}_\mathcal{C}$ such that for every t -query T -time ℓ -bit-output algorithm A :

$$\Delta \left(A^{\mathcal{C}^g, g}, A^{f, \mathcal{S}_\mathcal{C}^f} \right) \leq \varepsilon_{\text{ind}}(\lambda, n, t, T, \ell).$$

Lemma A.4. If $\mathcal{C}$ is indifferntiable with error ε_{ind} then $\mathcal{C}$ is strongly indifferntiable with error ε_{ind} such that

$$\varepsilon_{\text{ind}}(\lambda, n, t, T, \ell) \leq \ell \cdot \varepsilon_{\text{ind}}(\lambda, n, t, T).$$

Proof. Let $\mathcal{S}_\mathcal{C}$ be the probabilistic stateful algorithm guaranteed by Theorem A.2. Fix a t -query T -time ℓ -bit-output A , and let $(A_i)_{i \in [\ell]}$ be the algorithms that each output a bit of A 's ℓ -bit output. We can write:

$$\begin{aligned} & \Delta \left(A^{\mathcal{C}^g, g}, A^{f, \mathcal{S}_\mathcal{C}^f} \right) \\ & \leq \sum_{i \in [\ell]} \Delta \left(A_i^{\mathcal{C}^g, g}, A_i^{f, \mathcal{S}_\mathcal{C}^f} \right) && \text{(by sub-additivity of statistical distance)} \\ & = \sum_{i \in [\ell]} \left| \Pr \left[A_i^{\mathcal{C}^g, g} = 1 \mid g \leftarrow \mathcal{D}_\diamond(\lambda, n) \right] - \Pr \left[A_i^{f, \mathcal{S}_\mathcal{C}^f} = 1 \mid f \leftarrow \mathcal{D}(\lambda, n) \right] \right| \\ & && \text{(by definition of statistical distance)} \\ & \leq \sum_{i \in [\ell]} \varepsilon_{\text{ind}}(\lambda, n, t, T) && \text{(by applying Theorem A.2 to each distinguisher } A_i) \end{aligned}$$

$$= \ell \cdot \varepsilon_{\text{ind}}(\lambda, n, t, T).$$

□

Remark A.5 (oracle construction for the duplex sponge). Let $\text{IP} = (\mathbf{P}, \mathbf{V})$ be a public-coin IP and let $\mathcal{D}_{\text{IP}}$ be the corresponding oracle distribution for some salt size δ (see Equation 4). Let $\mathcal{D}_{\text{S}}$ be an ideal permutation oracle distribution (see Theorem 4.2). Let cdc a codec for IP over the alphabet Σ of $\mathcal{D}_{\text{S}}$ (see Theorem 4.1). We describe a $(\mathcal{D}_{\text{IP}}, \mathcal{D}_{\text{S}})$ -construction $\mathcal{C}_{\text{DSFS}}$ that realizes **DSFS**[IP, cdc, δ] in Theorem 4.3. Below we use syntax from the ideal duplex sponge described in Section 3.3.

$\mathcal{C}_{\text{DSFS}}^{h,p}:$

1. Initialize k empty lists $(L_i)_{i \in [k]}$.
2. All queries to h or p below are memoized so any duplicate queries are answered internally.
3. Upon receiving a query x for f_i , if there is y such that $(x, y) \in L_i$ then answer with y .

Otherwise (the query is new so) proceed as follows:

- (a) Parse x as $(\mathbb{x}, \tau, \alpha_1, \dots, \alpha_i)$ (abort if this cannot be done).
- (b) If parsing is possible do the following:
 - i. Initialize the sponge state with the instance: $\text{st}'_0 := \text{DS.Start}^h(\mathbb{x})$.
 - ii. Absorb the salt: $\text{st}_0 := \text{DS.Absorb}^p(\text{st}'_0, \tau)$ where τ is τ represented over Σ .
 - iii. For $j = 1, \dots, i$:
 - A. Encode the prover message: $\hat{\alpha}_j := \varphi_j(\alpha_j)$.
 - B. Absorb the encoded prover message: $\text{st}'_j := \text{DS.Absorb}^p(\text{st}_{j-1}, \hat{\alpha}_j)$.
 - C. Squeeze the encoded verifier message: $(\hat{\rho}_j, \text{st}_j) := \text{DS.Squeeze}^p(\text{st}'_j, \ell_{\mathbf{V}}(j))$.
 - D. Decode the verifier message: $\rho_j := \psi_j(\hat{\rho}_j)$.
- (c) Add (x, ρ_i) to L_i , and answer the query with ρ_i .

Note that

$$\mathcal{V}^{h,p}(\mathbb{x}, \pi) \equiv \mathcal{V}_{\text{std}}^{\mathcal{C}_{\text{DSFS}}^{h,p}}(\mathbb{x}, \pi).$$

A.2 Limitations of indistinguishability

Let $\text{NARG} = (\mathcal{P}, \mathcal{V})$ be a non-interactive argument for relation $\mathcal{R}$ in the $\mathcal{D}$ -oracle model (see Section 3.4) and let $\mathcal{C}$ be a $(\mathcal{D}, \mathcal{D}_{\diamond})$ -construction (see Theorem A.1). Given an oracle construction $\mathcal{C}$, define a new non-interactive argument $\text{NARG}_{\diamond} := (\mathcal{P}_{\diamond}, \mathcal{V}_{\diamond})$ in the $\mathcal{D}_{\diamond}$ -oracle model where, for every $\mathbf{g} \in \mathcal{D}_{\diamond}(\lambda, n)$,

$$\begin{aligned} \mathcal{P}_{\diamond}^{\mathbf{g}}(\mathbb{x}, \mathbb{w}) &:= \mathcal{P}^{\mathcal{C}^{\mathbf{g}}}(\mathbb{x}, \mathbb{w}), \\ \mathcal{V}_{\diamond}^{\mathbf{g}}(\mathbb{x}, \pi) &:= \mathcal{V}^{\mathcal{C}^{\mathbf{g}}}(\mathbb{x}, \pi). \end{aligned}$$

We discuss how security properties of NARG impact security properties of $\text{NARG}_{\diamond}$, by reasoning about the indistinguishability of $\mathcal{C}$ (see Theorem A.2).

- In Section A.2.1 we see that indistinguishability suffices to establish **soundness**.
- In Section A.2.2 we see that indistinguishability *does not suffice* to establish **knowledge soundness**. Instead, we introduce *extraction-friendly indistinguishability*, a stronger flavor that suffices for knowledge soundness.
- In Section A.2.3 we see that indistinguishability *does not suffice* to establish **zero knowledge**.

Subsequent to this work, [CY24] adopted our extraction-friendly indifferentiability as part of their general treatment of non-interactive arguments across oracle settings, and also introduced a different flavor called *simulation-friendly indifferentiability* that instead suffices to establish zero knowledge.

Throughout, we assume that $\mathcal{S}_c$ (from Theorem A.2) is query-preserving, i.e., $\mathcal{S}_c^f$ makes at most the same number of queries as it answers. (The discussion can be straightforwardly adapted to account for a parameter that accounts for $\mathcal{S}_c^f$'s query complexity in terms of the queries it answers.)

A.2.1 Soundness

The soundness error of $\text{NARG}_\diamond$ can be upper bounded by the soundness error of NARG plus the indifferentiability error ε_{ind} . More precisely, two different arguments yield incomparable upper bounds with different parameters: (i) a direct argument relies on a distinguisher whose running time depends on the time to decide the language $\mathcal{L}(\mathcal{R})$ (possibly exponential time); and (ii) an alternate argument, going through strong indifferentiability (Theorem A.3), incurs a multiplicative loss of n (the instance size bound).

Lemma A.6. *If NARG has soundness error $\varepsilon_{\text{NARG}}(\lambda, t, n)$ then $\text{NARG}_\diamond$ has soundness error $\varepsilon_{\text{NARG}_\diamond}(\lambda, t, n)$ that satisfies two (incomparable) upper bounds:*

$$\begin{aligned} \varepsilon_{\text{NARG}_\diamond}(\lambda, t, n) &\leq \varepsilon_{\text{NARG}}(\lambda, t, n) + \varepsilon_{\text{ind}}(\lambda, n, t + t_v, T + T_v + T_c), \\ &\text{and} \\ \varepsilon_{\text{NARG}_\diamond}(\lambda, t, n) &\leq \varepsilon_{\text{NARG}}(\lambda, t, n) + \varepsilon_{\text{ind}}(\lambda, n, t + t_v, T + T_v, n + 1) \\ &\leq \varepsilon_{\text{NARG}}(\lambda, t, n) + (n + 1) \cdot \varepsilon_{\text{ind}}(\lambda, n, t + t_v, T + T_v). \end{aligned}$$

Proof. We prove the two bounds separately; the second inequality in the second bound is by Theorem A.4.

Bound via ε_{ind} . The soundness property (Theorem 3.5) is a predicate applied to the tuple $(\mathbb{x}, \pi, b)$ sampled according to the experiment. Fix a t -query T -time malicious argument prover $\tilde{\mathcal{P}}_\diamond$ against $\mathcal{V}_\diamond$. Define the adversary D that, given two oracles $\mathcal{O}_1, \mathcal{O}_2$, works as follows.

$D^{\mathcal{O}_1, \mathcal{O}_2}$:

1. Compute $(\mathbb{x}, \pi) \leftarrow \tilde{\mathcal{P}}_\diamond^{\mathcal{O}_2}$.
2. Compute $b \leftarrow \mathcal{V}_\diamond^{\mathcal{O}_1}(\mathbb{x}, \pi)$.
3. Check that $|\mathbb{x}| \leq n, \mathbb{x} \notin \mathcal{L}(\mathcal{R}), b = 1$.
4. If so, output 1; else output 0.

Note that D makes $t + t_v$ queries and runs in time $T + T_v + T_c$. Therefore we can write

$$\begin{aligned} &\Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge \mathbb{x} \notin \mathcal{L}(\mathcal{R}) \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} \mathbf{g} \leftarrow \mathcal{D}_\diamond(\lambda, n) \\ (\mathbb{x}, \pi) \leftarrow \tilde{\mathcal{P}}_\diamond^{\mathbf{g}} \\ b \leftarrow \mathcal{V}_\diamond^{\mathbf{g}}(\mathbb{x}, \pi) \end{array} \right] \\ &= \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge \mathbb{x} \notin \mathcal{L}(\mathcal{R}) \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} \mathbf{g} \leftarrow \mathcal{D}_\diamond(\lambda, n) \\ (\mathbb{x}, \pi) \leftarrow \tilde{\mathcal{P}}_\diamond^{\mathbf{g}} \\ b \leftarrow \mathcal{V}_\diamond^{\mathbf{g}}(\mathbb{x}, \pi) \end{array} \right] \quad (\text{by construction of } \mathcal{V}_\diamond) \\ &\leq \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge \mathbb{x} \notin \mathcal{L}(\mathcal{R}) \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}(\lambda, n) \\ (\mathbb{x}, \pi) \leftarrow \tilde{\mathcal{P}}_\diamond^{\mathcal{S}_c^f} \\ b \leftarrow \mathcal{V}_\diamond^{\mathbf{f}}(\mathbb{x}, \pi) \end{array} \right] + \varepsilon_{\text{ind}}(\lambda, n, t + t_v, T + T_v + T_c) \\ &\quad (\text{by applying Theorem A.2 to } D) \end{aligned}$$

$$= \varepsilon_{\text{NARG}}(\lambda, t, n) + \varepsilon_{\text{ind}}(\lambda, n, t + t_v, T + T_v + T_c).$$

Bound via ε_{ind} . The soundness property (Theorem 3.5) is a predicate applied to the tuple $(\mathbb{x}, b)$ sampled according to the experiment. Fix a t -query T -time malicious argument prover $\tilde{\mathcal{P}}_\diamond$ against $\mathcal{V}_\diamond$. Define the adversary A that, given two oracles $\mathcal{O}_1, \mathcal{O}_2$, works as follows.

$A^{\mathcal{O}_1, \mathcal{O}_2}$:

1. Compute $(\mathbb{x}, \pi) \leftarrow \tilde{\mathcal{P}}_\diamond^{\mathcal{O}_2}$.
2. Compute $b \leftarrow \mathcal{V}_\diamond^{\mathcal{O}_1}(\mathbb{x}, \pi)$.
3. Output $(\mathbb{x}, b)$.

Note that A makes $t + t_v$ queries, runs in time $T + T_v$, and outputs at most $n + 1$ bits. Therefore we can write

$$\begin{aligned} & \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge \mathbb{x} \notin \mathcal{L}(\mathcal{R}) \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} \mathbf{g} \leftarrow \mathcal{D}_\diamond(\lambda, n) \\ (\mathbb{x}, \pi) \leftarrow \tilde{\mathcal{P}}_\diamond^{\mathbf{g}} \\ b \leftarrow \mathcal{V}_\diamond^{\mathbf{g}}(\mathbb{x}, \pi) \end{array} \right] \\ &= \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge \mathbb{x} \notin \mathcal{L}(\mathcal{R}) \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} \mathbf{g} \leftarrow \mathcal{D}_\diamond(\lambda, n) \\ (\mathbb{x}, \pi) \leftarrow \tilde{\mathcal{P}}_\diamond^{\mathbf{g}} \\ b \leftarrow \mathcal{V}_\diamond^{\mathbf{g}}(\mathbb{x}, \pi) \end{array} \right] \quad (\text{by construction of } \mathcal{V}_\diamond) \\ &\leq \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge \mathbb{x} \notin \mathcal{L}(\mathcal{R}) \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}(\lambda, n) \\ (\mathbb{x}, \pi) \leftarrow \tilde{\mathcal{P}}_\diamond^{\mathbf{f}} \\ b \leftarrow \mathcal{V}_\diamond^{\mathbf{f}}(\mathbb{x}, \pi) \end{array} \right] + \varepsilon_{\text{ind}}(\lambda, n, t + t_v, T + T_v, n + 1) \\ &\quad (\text{by applying Theorem A.3 to } A) \\ &= \varepsilon_{\text{NARG}}(\lambda, t, n) + \varepsilon_{\text{ind}}(\lambda, n, t + t_v, T + T_v, n + 1). \end{aligned}$$

□

A.2.2 Knowledge soundness

For simplicity, we focus on straightline knowledge soundness; a similar discussion holds for rewinding knowledge soundness. Indifferentiability (Theorem A.2) as well as strong indifferentiability (Theorem A.3) do not offer capabilities for translating a query-answer trace from the $\mathcal{D}_\diamond$ -oracle model to the $\mathcal{D}$ -oracle model. This is necessary to build a knowledge extractor for $\text{NARG}_\diamond$ by relying on a knowledge extractor for NARG . To address this, we propose *extraction-friendly indifferentiability* as a notion that precisely captures what we need; abstractly, this is what we prove in Theorem 5.1 for deducing the knowledge soundness of **DSFS**.

Definition A.7. A $(\mathcal{D}, \mathcal{D}_\diamond)$ -construction $\mathcal{C}$ is **extraction-friendly indifferentiable with error $\varepsilon_{\text{eind}}$** if there exist a probabilistic stateful algorithm $\mathcal{S}_\mathcal{C}$ and a deterministic algorithm $\mathcal{T}_\mathcal{C}$ such that for every t -query T -time ℓ -bit-output algorithm A :

$$\Delta \left(\left\{ (z, \mathcal{T}_\mathcal{C}(\text{tr}_\diamond)) \mid z \xleftarrow{\text{tr}_\diamond} A^{\mathcal{C}^{\mathbf{g}} \cdot \mathbf{g}} \right\}, \left\{ (z, \text{tr}) \mid z \xleftarrow{\text{tr}} A^{\mathbf{f}, \mathcal{S}_\mathcal{C}^{\mathbf{f}}} \right\} \right) \leq \varepsilon_{\text{eind}}(\lambda, n, t, T, \ell).$$

We prove that the above notion enables upper bounding the knowledge soundness error of $\text{NARG}_\diamond$ in terms of the knowledge soundness error of NARG plus the extraction-friendly indifference error $\varepsilon_{\text{eind}}$.

Lemma A.8. *If NARG has straightline knowledge soundness error $\kappa_{\text{NARG}}(\lambda, t, n)$ then $\text{NARG}_\diamond$ has straightline knowledge soundness error $\varepsilon_{\text{NARG}_\diamond}(\lambda, t, n)$ such that*

$$\begin{aligned} & \kappa_{\text{NARG}_\diamond}(\lambda, t, n) \\ & \leq \kappa_{\text{NARG}}(\lambda, t, n) + \varepsilon_{\text{eind}}(\lambda, n, t + t_\nu, T + T_\nu, n + |\pi| + 1). \end{aligned}$$

Proof. The straightline knowledge soundness property (Theorem 3.6) is a predicate applied to the tuple $(\mathbb{x}, \pi, b, \text{tr})$ sampled according to the experiment. Fix a t -query T -time malicious argument prover $\tilde{\mathcal{P}}_\diamond$ against $\mathcal{V}_\diamond$. Letting $\mathcal{E}$ be the straightline extractor for NARG, we define a straightline extractor $\mathcal{E}_\diamond$ for $\text{NARG}_\diamond$ as follows.

- $\mathcal{E}_\diamond(\mathbb{x}, \pi, \text{tr}_\diamond)$:
1. Compute $\text{tr} \leftarrow \mathcal{T}_c(\text{tr}_\diamond)$.
 2. Compute $\mathbb{w} \leftarrow \mathcal{E}(\mathbb{x}, \pi, \text{tr})$.
 3. Output $\mathbb{w}$.

Define the adversary A that, given two oracles $\mathcal{O}_1, \mathcal{O}_2$, works as follows.

- $A^{\mathcal{O}_1, \mathcal{O}_2}$:
1. Compute $(\mathbb{x}, \pi) \xleftarrow{\text{tr}_\diamond} \tilde{\mathcal{P}}_\diamond^{\mathcal{O}_2}$.
 2. Compute $b \leftarrow \mathcal{V}^{\mathcal{O}_1}(\mathbb{x}, \pi)$.
 3. Output $(\mathbb{x}, \pi, b)$.

Note that A makes $t + t_\nu$ queries, runs in time $T + T_\nu$, and outputs at most $n + |\pi| + 1$ bits. Therefore we can write

$$\begin{aligned} & \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge (\mathbb{x}, \mathbb{w}) \notin \mathcal{R} \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} \mathbf{g} \leftarrow \mathcal{D}_\diamond(\lambda, n) \\ (\mathbb{x}, \pi) \xleftarrow{\text{tr}_\diamond} \tilde{\mathcal{P}}_\diamond^{\mathbf{g}} \\ \mathbb{w} \leftarrow \mathcal{E}_\diamond(\mathbb{x}, \pi, \text{tr}_\diamond) \\ b \leftarrow \mathcal{V}_\diamond^{\mathbf{g}}(\mathbb{x}, \pi) \end{array} \right] \\ &= \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge (\mathbb{x}, \mathbb{w}) \notin \mathcal{R} \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} \mathbf{g} \leftarrow \mathcal{D}_\diamond(\lambda, n) \\ (\mathbb{x}, \pi) \xleftarrow{\text{tr}_\diamond} \tilde{\mathcal{P}}_\diamond^{\mathbf{g}} \\ \text{tr} \leftarrow \mathcal{T}_c(\text{tr}_\diamond) \\ \mathbb{w} \leftarrow \mathcal{E}(\mathbb{x}, \pi, \text{tr}) \\ b \leftarrow \mathcal{V}^{\mathbf{g}}(\mathbb{x}, \pi) \end{array} \right] \quad (\text{by construction of } \mathcal{V}_\diamond \text{ and } \mathcal{E}_\diamond) \\ &\leq \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n \\ \wedge (\mathbb{x}, \mathbb{w}) \notin \mathcal{R} \\ \wedge b = 1 \end{array} \middle| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}(\lambda, n) \\ (\mathbb{x}, \pi) \xleftarrow{\text{tr}} \tilde{\mathcal{P}}_\diamond^{\mathbf{f}} \\ \mathbb{w} \leftarrow \mathcal{E}(\mathbb{x}, \pi, \text{tr}) \\ b \leftarrow \mathcal{V}^{\mathbf{f}}(\mathbb{x}, \pi) \end{array} \right] + \varepsilon_{\text{eind}}(\lambda, n, t + t_\nu, T + T_\nu, n + |\pi| + 1). \\ & \quad (\text{by applying Theorem A.7 to } A) \end{aligned}$$

□

A.2.3 Zero knowledge

Let NARG and $\text{NARG}_\diamond$ be as in Section A.2. The zero-knowledge property of NARG states that the following two distributions are $z_{\text{NARG}}(\lambda, t, n)$ -close in statistical distance (for every t -query admissible adversary $\mathcal{A}$):

$$\left\{ \text{out} \left| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \leftarrow \mathcal{A}^{\mathbf{f}} \\ \pi \leftarrow \mathcal{P}^{\mathbf{f}}(\mathbb{x}, \mathbb{w}) \\ \text{out} \leftarrow \mathcal{A}^{\mathbf{f}}(\text{aux}, \pi) \end{array} \right. \right\} \text{ and } \left\{ \text{out} \left| \begin{array}{l} \mathbf{f} \leftarrow \mathcal{D}(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \leftarrow \mathcal{A}^{\mathbf{f}} \\ (\pi, \mu) \leftarrow \mathcal{S}^{\mathbf{f}}(\mathbb{x}) \\ \text{out} \leftarrow \mathcal{A}^{\mathbf{f}[\mu]}(\text{aux}, \pi) \end{array} \right. \right\}.$$

Analogously, the zero-knowledge property of $\text{NARG}_\diamond$ states that the following two distributions are $z_{\text{NARG}_\diamond}(\lambda, t, n)$ -close in statistical distance (for every t -query admissible adversary $\mathcal{A}$):

$$\left\{ \text{out} \left| \begin{array}{l} \mathbf{g} \leftarrow \mathcal{D}_\diamond(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \leftarrow \mathcal{A}_\diamond^{\mathbf{g}} \\ \pi \leftarrow \mathcal{P}_\diamond^{\mathbf{g}}(\mathbb{x}, \mathbb{w}) \\ \text{out} \leftarrow \mathcal{A}_\diamond^{\mathbf{g}}(\text{aux}, \pi) \end{array} \right. \right\} \text{ and } \left\{ \text{out} \left| \begin{array}{l} \mathbf{g} \leftarrow \mathcal{D}_\diamond(\lambda, n) \\ (\mathbb{x}, \mathbb{w}, \text{aux}) \leftarrow \mathcal{A}_\diamond^{\mathbf{g}} \\ (\pi, \mu_\diamond) \leftarrow \mathcal{S}_\diamond^{\mathbf{g}}(\mathbb{x}) \\ \text{out} \leftarrow \mathcal{A}_\diamond^{\mathbf{g}[\mu_\diamond]}(\text{aux}, \pi) \end{array} \right. \right\}.$$

Our goal is to derive an upper bound on $z_{\text{NARG}_\diamond}$ in terms of z_{NARG} plus some additive error associated to the oracle construction $\mathcal{C}$. This would also entail (somehow) constructing the new simulator $\mathcal{S}_\diamond$ for $\text{NARG}_\diamond$ in terms of the old simulator $\mathcal{S}$ for NARG, for example as follows:

$\mathcal{S}_\diamond^{\mathbf{g}}(\mathbb{x})$:

1. Compute $(\pi, \mu) \leftarrow \mathcal{S}^{\mathbf{g}}(\mathbb{x})$.
2. Remap (somehow) the programmed locations μ for $\mathbf{f}$ to programmed locations $\mu_\diamond$ for $\mathbf{g}$.
3. Output $(\pi, \mu_\diamond)$.

However, indistinguishability does not provide any guarantees about the behavior of an adversary when an oracle is programmed (nor do the strengthenings we considered in Theorem A.3 and Theorem A.7).

One way to address this would be to augment the notion of indistinguishability to include programming (this is done in [CY24] via the notion of simulation-friendly indistinguishability). However this would yield an additive error that grows at least quadratically in t , whereas the “right” additive error is a linear growth in t . We do not know of a property that allows for such a loss while generically transferring the zero knowledge property from NARG to $\text{NARG}_\diamond$. This is why we prove the zero knowledge property of **DSFS** directly.

B Example: codecs for Schnorr's protocol

We review Schnorr's protocol: we describe a codec for binary permutation functions (where the alphabet is $\Sigma = \{0, 1\}$), and a codec for algebraic permutation functions (where the alphabet is a field).

Let $\mathbb{G}$ be an additive elliptic curve group of prime order r where DL is hard, and let $G \in \mathbb{G}$ be a generator for $\mathbb{G}$.¹⁵ Let $\mathbb{Z}_p$ be the coordinate field of the elliptic curve. Schnorr's protocol is a 3-message IP for the relation $\mathcal{R} := \{(X, x) \in \mathbb{G} \times \mathbb{Z}_p : X = xG\}$. In particular, $k = 2$ and the second round consists of a prover message and no verifier message. To prove knowledge of $x \in \mathbb{Z}_r$ such that $X = xG$, the prover sends $K := kG$, the verifier sends a challenge $c \in \mathbb{Z}_r$, and the prover sends $s := k + cx \bmod r$. The verifier accepts if $sG = K + cX$. In our notation:

- $\mathbf{x} = X \in \mathbb{G}$ with $n := \lceil \log_2 p \rceil + 1$, the size of the “x” coordinate and the sign of the “y” coordinate;
- $\mathcal{M}_{P,1} = \mathbb{Z}_p^2$ is the message space of the first prover message, seen as the affine representation of an elliptic curve point;
- $\mathcal{M}_{V,1} = \mathbb{Z}_r$ is the message space of the verifier message;
- $\mathcal{M}_{P,2} = \mathbb{Z}_r$ is the message space of the second prover message.

Binary codec. An example of a binary permutation function is KECCAK- $f[1600]$ [Sha], which has alphabet $\Sigma = \{0, 1\}$, capacity 512, and rate 1088. The binary codec for Schnorr's protocol is a tuple $(\ell_P, \ell_V, \varphi, \psi)$ where:

- $\varphi_1: \mathbb{Z}_p^2 \rightarrow \{0, 1\}^{\ell_P(1)}$, with $\ell_P(1) := \lceil \log p \rceil + 1$, is the big-endian binary encoding of the “x” coordinate and the sign of the “y” coordinate. Note that φ_1 is injective, and its preimage is efficiently computable using the elliptic curve's equation.
- $\psi_1: \{0, 1\}^{\ell_V(1)} \rightarrow \mathbb{Z}_r$, with $\ell_V(1) = \lceil \log r \rceil + \lambda$, is the big-endian decoding of the given binary string interpreted as an integer modulo r (i.e., $\mathbf{b} \in \{0, 1\}^{\ell_V(1)}$ is mapped to $\sum_{i=1}^{\ell_V(1)} b_i 2^{i-1} \bmod r$). The additive term λ ensures that ψ_1 has bias at most $2^{-\lambda}$ (see Theorem C.1).
- $\varphi_2: \mathbb{Z}_r \rightarrow \{0, 1\}^{\ell_P(2)}$, with $\ell_P(2) := \lceil \log r \rceil$ is the big-endian binary encoding of the second prover message.

Algebraic codec. Algebraic permutation functions typically have a larger alphabet. An example is Poseidon [GKRRS21], whose alphabet can be set to be the field $\mathbb{Z}_p$ (the elliptic curve's field of definition), with capacity 1 and rate 2. The algebraic codec for Schnorr's protocol is a tuple $(\ell_P, \ell_V, \varphi, \psi)$ where:

- $\varphi_1: \mathbb{Z}_p^2 \rightarrow \mathbb{Z}_p^2$ (with $\ell_P(1) = 2$) is the identity function.
- $\psi_1: \mathbb{Z}_p \rightarrow \mathbb{Z}_r$ (with $\ell_V(1) = 1$) maps x to $x \bmod r$. The bias of this map is $2^{\frac{p \bmod r}{pr}}(p - (p \bmod r))$, by Theorem C.1. If we consider a pairing-friendly elliptic curve such as BLS12-381, the bias is at most 2^{-126} .
- $\varphi_2: \mathbb{Z}_r \rightarrow \{0, 1\}^{\ell_P(2)}$, with $\ell_P(2) = \lceil \log r / \log p \rceil$, is the big-endian binary encoding of the response. In typical pairing-friendly elliptic curves, such as BLS12-381, this map is the “identity” function, mapping an element of $\mathbb{Z}_r$, seen as an integer $[0, r-1]$, into $\mathbb{Z}_q$.

A common concern, when using algebraic hashes, is the number of invocations of the permutation function. In this example, the permutation function is invoked only once: the construction **DSFS** absorbs two $\mathbb{Z}_p$ elements, writing two elements in the rate (see Item 3a in Theorem 3.3). At the end of the execution, the state has i_A and i_S equal to the rate of the sponge (and no call yet to the permutation function). Then, **DSFS** squeezes one $\mathbb{Z}_p$ element from the duplex sponge, which invokes the permutation function, sets $i_S = 0$, and then reads the first element of the rate segment in the permutation state.

¹⁵The group choice is merely an example, and captures, for instance, the elliptic curves in the SEC2 standards [Bro10]. Other choices are possible (e.g., the subgroup of squares in $\mathbb{Z}_q^*$, where q is a safe prime).

C Bias of modular reduction

We state and prove a simple lemma about the bias of modular reduction, which is useful for bounding the bias of the distribution that arises from a common decoding strategy from binary strings to prime field elements.

Consider the setting where the verifier message is a random field element in a prime field $\mathbb{F}_p$, equivalently, a random integer in $[0, p-1]$. Moreover, suppose that the function (or permutation) used in the Fiat–Shamir transformation is binary, which means that one must somehow decode a field element from a (random) binary string x in $\{0, 1\}^m$, for a sufficiently large m .

A common decoding strategy [Hao] is to interpret x as a base-2 integer and outputting its remainder modulo p , i.e., the decoding function $\psi: \{0, 1\}^m \rightarrow [0, p-1]$ is

$$\psi(x) := \left(\sum_{i=1}^m x_i 2^{i-1} \right) \bmod p.$$

The lemma below directly implies that, for $m := \lceil \log p \rceil + \lambda$, the bias of ψ is at most $2^{-\lambda}$. The extra length λ ensures that $\mathcal{U}(\mathbb{F}_p) = \mathcal{U}([0, p-1])$ and $\psi(\mathcal{U}(\{0, 1\}^m))$ are close enough. More generally, for positive integers a, b with $b \geq a$, the lemma below upper bounds the statistical distance between $\mathcal{U}([0, a-1])$ (the target distribution) and the distribution arising from $\mathcal{U}([0, b-1])$ reduced modulo a .

Lemma C.1. *Let a, b be positive integers with $b \geq a$, and set $r := b \bmod a$. Let $\psi_{a,b}: [0, b-1] \rightarrow [0, a-1]$ be the function that maps x to $x \bmod a$. Then*

$$\Delta(\mathcal{U}([0, a-1]), \psi_{a,b}(\mathcal{U}([0, b-1]))) \leq \frac{2r}{ab}(a-r).$$

In particular:

- if $a \mid b$ then the statistical distance is 0, and
- if $\lceil \log_2 b \rceil \geq \lceil \log_2 a \rceil + \lambda$ then the statistical distance is at most $2^{-\lambda}$.

Proof. Let $b = q \cdot a + r$ with $0 \leq r < a$. For every $y \in [0, a-1]$, the probability that $\psi_{a,b}(\mathcal{U}([0, b-1]))$ is equal to y is $\frac{q+1}{b}$ if $0 \leq y < r$, and $\frac{q}{b}$ if $r \leq y < a$. Therefore:

$$\begin{aligned} & \Delta(\mathcal{U}([0, a-1]), \psi_{a,b}(\mathcal{U}([0, b-1]))) \\ &= \sum_{k \in [0, r-1]} \left| \frac{1}{a} - \frac{q+1}{b} \right| + \sum_{k \in [r, a-1]} \left| \frac{1}{a} - \frac{q}{b} \right| \\ &= r \left| \frac{b - a(q+1)}{ab} \right| + (a-r) \left| \frac{b - qa}{ab} \right| \\ &= \frac{1}{ab} \cdot (r|b - qa - a| + (a-r)|r|) \\ &= \frac{2r}{ab}(a-r). \end{aligned}$$

The case where $a \mid b$ is straightforward since it implies $r = 0$. The case where $\lceil \log_2 b \rceil \geq \lceil \log_2 a \rceil + \lambda$ follows the fact that $r(a-r) \leq a^2/2$:

$$\frac{2r}{ab}(a-r) \leq \frac{a^2}{a^2 2^\lambda} \leq \frac{1}{2^\lambda}.$$

□

Acknowledgments

This work was partially supported by the Ethereum Foundation, and by Agence Nationale de la Recherche (ANR) project ANR-24-CE39-6061-01A2C. We are grateful to Quang Dao (CMU) and Kasra Abbaszadeh (University of Maryland) for suggestions and comments.

References

- [AB26] T. Ashur and A. S. Bhati. “Generalized Indifferentiable Sponge and its Application to Polygon Miden VM”. In: *Designs, Codes and Cryptography* 94.1 (2026), p. 9. doi: 10.1007/S10623-025-01742-5.
- [AFK22] T. Attema, S. Fehr, and M. Klooß. “Fiat-Shamir Transformation of Multi-round Interactive Proofs”. In: *Proceedings of the 20th International Conference on Theory of Cryptography*. Vol. 13747. TCC ’22. 2022, pp. 113–142. doi: 10.1007/978-3-031-22318-1_5.
- [AGRRT16] M. R. Albrecht, L. Grassi, C. Rechberger, A. Roy, and T. Tiessen. “MiMC: Efficient Encryption and Cryptographic Hashing with Minimal Multiplicative Complexity”. In: *Proceedings of the 22nd International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’16. 2016, pp. 191–219. doi: 10.1007/978-3-662-53887-6_7.
- [AKMQ23] J. Aumasson, D. Khovratovich, B. Mennink, and P. Quine. *SAFE: Sponge API for Field Elements*. Cryptology ePrint Archive, Paper 2023/522. 2023.
- [Ark] *A library of useful cryptographic primitives*. <https://github.com/arkworks-rs/cryptoprimitives>.
- [ark] arkworks. *arkworks: an ecosystem for developing and programming with zkSNARKs*. URL: <https://github.com/arkworks-rs>.
- [AY25] G. Arnon and E. Yogev. “Towards a White-Box Secure Fiat-Shamir Transformation”. In: *Proceedings of the 45th Annual International Cryptology Conference*. CRYPTO ’25. 2025, pp. 27–56. doi: 10.1007/978-3-032-01887-8_2.
- [Azt] Aztec. *Aztec monorepo*. <https://github.com/AztecProtocol/aztec-packages/>.
- [Bar01] B. Barak. “How to Go Beyond the Black-Box Simulation Barrier”. In: *Proceedings of the 42nd IEEE Symposium on Foundations of Computer Science*. FOCS ’01. 2001, pp. 106–115.
- [BBHMR19] J. Bartusek, L. Bronfman, J. Holmgren, F. Ma, and R. D. Rothblum. “On the (In)security of Kilian-Based SNARGs”. In: *Proceedings of the 14th Theory of Cryptography Conference*. TCC ’19. 2019, pp. 522–551. ISBN: 978-3-030-36033-7.
- [BCS16] E. Ben-Sasson, A. Chiesa, and N. Spooner. “Interactive Oracle Proofs”. In: *Proceedings of the 14th Theory of Cryptography Conference*. TCC ’16. 2016, pp. 31–60.
- [BDHPVAVK18] G. Bertoni, J. Daemen, S. Hoffert, M. Peeters, G. Van Assche, and R. Van Keer. “Farfalle: parallel permutation-based cryptography”. In: *FSE ’18* (2018).
- [BDPV08] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche. “On the Indifferentiability of the Sponge Construction”. In: *Proceedings of the 27th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’08. 2008, pp. 181–197. doi: 10.1007/978-3-540-78967-3_11.
- [BDPVA12] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche. “Duplexing the Sponge: Single-Pass Authenticated Encryption and Other Applications”. In: *Selected Areas in Cryptography*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 320–337. ISBN: 978-3-642-28496-0.

- [BKM20] Z. Brakerski, V. Koppula, and T. Mour. “NIZK from LPN and Trapdoor Hash via Correlation Intractability for Approximable Relations”. In: *Proceedings of the 40th Annual International Cryptology Conference*. CRYPTO ’20. 2020, pp. 738–767. doi: 10.1007/978-3-030-56877-1_26.
- [BLHG] S. Bowe, Y. T. Lai, D. E. Hopwood, and J. Grigg. *The Halo2 zero-knowledge proving system*. <https://github.com/zcash/halo2>.
- [Bou+23] C. Bouvier et al. “New Design Techniques for Efficient Arithmetization-Oriented Hash Functions: Anemoi Permutations and Jive Compression Mode”. In: *Proceedings of the 43th Annual International Cryptology Conference*. CRYPTO ’23. 2023. ISBN: 978-3-031-38547-6. doi: 10.1007/978-3-031-38548-3_17.
- [BR06] M. Bellare and P. Rogaway. “The Security of Triple Encryption and a Framework for Code-Based Game-Playing Proofs”. In: *Proceedings of the 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’06. 2006. doi: 10.1007/11761679_25.
- [Bro10] D. L. R. Brown. *Standards for Efficient Cryptography — SEC 2: Recommended Elliptic Curve Domain Parameters*. Ed. by Certicom Research. <https://www.secg.org/sec2-v2.pdf>. Version 2.0. 2010.
- [Can01] R. Canetti. “Universally Composable Security: A New Paradigm for Cryptographic Protocols”. In: *Proceedings of the 42nd Annual IEEE Symposium on Foundations of Computer Science*. FOCS ’01. 2001, pp. 136–145.
- [Can+19] R. Canetti et al. “Fiat-Shamir: from practice to theory”. In: *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*. STOC ’19. 2019, pp. 1082–1090. doi: 10.1145/3313276.3316380.
- [CCHLRR18] R. Canetti, Y. Chen, J. Holmgren, A. Lombardi, G. N. Rothblum, and R. D. Rothblum. “Fiat-Shamir From Simpler Assumptions”. In: *IACR Cryptol. ePrint Arch.* (2018), p. 1004. URL: <https://eprint.iacr.org/2018/1004>.
- [CCR16] R. Canetti, Y. Chen, and L. Reyzin. “On the Correlation Intractability of Obfuscated Pseudorandom Functions”. In: *Proceedings of the 14th Theory of Cryptography Conference*. TCC ’16. 2016, pp. 389–415. doi: 10.1007/978-3-662-49096-9_17.
- [CCRR18] R. Canetti, Y. Chen, L. Reyzin, and R. D. Rothblum. “Fiat-Shamir and Correlation Intractability from Strong KDM-Secure Encryption”. In: *Proceedings of the 37th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’18. 2018, pp. 91–122. doi: 10.1007/978-3-319-78381-9_4.
- [CDGLN18] J. Camenisch, M. Drijvers, T. Gagliardoni, A. Lehmann, and G. Neven. “The Wonderful World of Global Random Oracles”. In: *Proceedings of the 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT’18. 2018, pp. 280–312. doi: 10.1007/978-3-319-78381-9_11.
- [CF24] A. Chiesa and G. Fenzi. “zkSNARKs in the ROM with Unconditional UC-Security”. In: *Proceedings of the 22nd International Theory of Cryptography Conference*. TCC ’24. 2024, pp. 67–89. doi: 10.1007/978-3-031-78011-0_3.
- [CGH04] R. Canetti, O. Goldreich, and S. Halevi. “The random oracle methodology, revisited”. In: *Journal of the ACM* 51.4 (2004), pp. 557–594.
- [CGJJZ23] A. R. Choudhuri, S. Garg, A. Jain, Z. Jin, and J. Zhang. “Correlation Intractability an SNARGs from Sub-exponential DDH”. In: *Proceedings of the 43rd Annual International Cryptology Conference*. CRYPTO ’23. 2023, pp. 635–668. doi: 10.1007/978-3-031-38551-3_20.

- [CJS14] R. Canetti, A. Jain, and A. Scafuro. “Practical UC security with a Global Random Oracle”. In: *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’14. 2014, pp. 597–608. doi: 10.1145/2660267.2660374.
- [CKGW] D. Connolly, C. Komlo, I. Goldberg, and C. A. Wood. *The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures*. RFC 9591. doi: 10.17487/RFC9591. URL: <https://www.rfc-editor.org/info/rfc9591>.
- [CMS19] A. Chiesa, P. Manohar, and N. Spooner. “Succinct Arguments in the Quantum Random Oracle Model”. In: *17th International Theory of Cryptography Conference*. TCC 2019. 2019, pp. 1–29. doi: 10.1007/978-3-030-36033-7_1.
- [CY24] A. Chiesa and E. Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024. URL: <https://snargsbook.org/>.
- [DFHSW] A. Davidson, A. Faz-Hernandez, N. Sullivan, and C. A. Wood. *Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups*. RFC 9497. doi: 10.17487/RFC9497. URL: <https://www.rfc-editor.org/info/rfc9497>.
- [DFMS19] J. Don, S. Fehr, C. Majenz, and C. Schaffner. “Security of the Fiat-Shamir Transformation in the Quantum Random-Oracle Model”. In: *Proceedings of the 39th Annual International Cryptology Conference*. CRYPTO ’19. Springer, 2019, pp. 356–383. doi: 10.1007/978-3-030-26951-7_13.
- [dus] dusk-network. *safe: Sponge API for Field Elements*. URL: <https://github.com/dusk-network/safe>.
- [Eth24] Ethereum Foundation. *zkEVM Formal Verification Project*. <https://verified-zkevm.org/>. 2024.
- [FS86] A. Fiat and A. Shamir. “How to prove yourself: practical solutions to identification and signature problems”. In: *Proceedings of the 6th Annual International Cryptology Conference*. CRYPTO ’86. 1986, pp. 186–194.
- [GK03] S. Goldwasser and Y. T. Kalai. “On the (In)security of the Fiat-Shamir Paradigm”. In: *Proceedings of the 44th Annual IEEE Symposium on Foundations of Computer Science*. FOCS ’03. 2003, pp. 102–113.
- [GKRRS21] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schafneger. “Poseidon: A New Hash Function for Zero-Knowledge Proof Systems”. In: *30th USENIX Security Symposium*. USENIX Security ’21. 2021, pp. 519–535.
- [Ham17] M. Hamburg. “The STROBE protocol framework”. In: *IACR Cryptol. ePrint Arch.* (2017), p. 3. URL: <http://eprint.iacr.org/2017/003>.
- [Hao] F. Hao. *Schnorr Non-interactive Zero-Knowledge Proof*. RFC 8235. doi: 10.17487/RFC8235. URL: <https://www.rfc-editor.org/info/rfc8235>.
- [HL18] J. Holmgren and A. Lombardi. “Cryptographic Hashing from Strong One-Way Functions (Or: One-Way Product Functions and Their Applications)”. In: *Proceedings of the 59th Annual IEEE Symposium on Foundations of Computer Science*. FOCS ’18. 2018, pp. 850–858. doi: 10.1109/FOCS.2018.00085.
- [HLR21] J. Holmgren, A. Lombardi, and R. D. Rothblum. “Fiat-Shamir via list-recoverable codes (or: parallel repetition of GMW is not zero-knowledge)”. In: *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*. STOC ’21. 2021, pp. 750–760. doi: 10.1145/3406325.3451116.
- [JJ21] A. Jain and Z. Jin. “Non-interactive Zero Knowledge from Sub-exponential DDH”. In: *Proceedings of the 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’21. 2021, pp. 3–32. doi: 10.1007/978-3-030-77870-5_1.

- [JKKZ21] R. Jawale, Y. T. Kalai, D. Khurana, and R. Zhang. “SNARGs for bounded depth computations and PPAD hardness from sub-exponential LWE”. In: *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*. STOC ’21. 2021, pp. 708–721. doi: 10.1145/3406325.3451055.
- [KLV23] Y. T. Kalai, A. Lombardi, and V. Vaikuntanathan. “SNARGs and PPAD Hardness from the Decisional Diffie-Hellman Assumption”. In: *Proceedings of the 41th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’23. 2023, pp. 470–498. doi: 10.1007/978-3-031-30617-4_16.
- [KMM23] D. Khovratovich, M. Marhuenda Beltrán, and B. Mennink. “Generic Security of the SAFE API and Its Applications”. In: *Proceedings of the 29th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT 2023. 2023, pp. 301–327.
- [KRR17] Y. T. Kalai, G. N. Rothblum, and R. D. Rothblum. “From Obfuscation to the Security of Fiat-Shamir for Proofs”. In: *Proceedings of the 37th Annual International Conference on Advances in Cryptology*. CRYPTO ’17. 2017, pp. 224–251. ISBN: 978-3-319-63715-0.
- [KRS25] D. Khovratovich, R. D. Rothblum, and L. Soukhanov. “How to Prove False Statements: Practical Attacks on Fiat-Shamir”. In: *Proceedings of the 45th Annual International Cryptology Conference*. CRYPTO ’25. 2025, pp. 3–26. doi: 10.1007/978-3-032-01887-8_1.
- [LKWL] T. Looker, V. Kalos, A. Whitehead, and M. Lodder. *The BBS Signature Scheme*. Internet-Draft draft-irtf-cfrg-bbs-signatures-07. Work in Progress. Internet Engineering Task Force. URL: <https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/07/>.
- [LZ19] Q. Liu and M. Zhandry. “Revisiting Post-quantum Fiat-Shamir”. In: *Proceedings of the 39th Annual International Cryptology Conference*. CRYPTO ’19. 2019, pp. 326–355. doi: 10.1007/978-3-030-26951-7_12.
- [MF21] A. Mittelbach and M. Fischlin. *The Theory of Hash Functions and Random Oracles - An Approach to Modern Cryptography*. Information Security and Cryptography. 2021. ISBN: 978-3-030-63286-1. doi: 10.1007/978-3-030-63287-8.
- [MRH04] U. M. Maurer, R. Renner, and C. Holenstein. “Indifferentiability, Impossibility Results on Reductions, and Applications to the Random Oracle Methodology”. In: *Proceedings of the first Theory of Cryptography Conference*. TCC ’04. 2004, pp. 21–39. doi: 10.1007/978-3-540-24638-1_2.
- [Pas03] R. Pass. “On Deniability in the Common Reference String and Random Oracle Model”. In: *Proceedings of the 23rd Annual International Cryptology Conference*. CRYPTO ’03. 2003, pp. 316–337.
- [PS19] C. Peikert and S. Shiehian. “Noninteractive Zero Knowledge for NP from (Plain) Learning with Errors”. In: *Proceedings of the 39th Annual International Cryptology Conference*. CRYPTO ’19. 2019, pp. 89–114. doi: 10.1007/978-3-030-26948-7_4.
- [Set] S. Setty. *Nova: High-speed recursive arguments from folding schemes*. <https://github.com/microsoft/Nova/>.
- [Sha] SHA-3 Standard: *Permutation-Based Hash and Extendable-Output Functions*. National Institute of Standards and Technology, NIST FIPS PUB 202, U.S. Department of Commerce. 2015.
- [Sig] Signal Foundation. *libsignal’s proof of knowledge stateful hash object*. <https://github.com/signalapp/libsignal/tree/main/rust/poksho/>.
- [Sta] StarkWare. *Cairo*. <https://github.com/starkware-libs/cairo-lang/>.
- [Val] H. de Valence. *Merlin: Composable proof transcripts for public-coin arguments of knowledge*. <https://github.com/dalek-cryptography/merlin>. Version 1.0.
- [Wee09] H. Wee. “Zero Knowledge in the Random Oracle Model, Revisited”. In: *Proceedings of the 15th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’09. Springer Berlin Heidelberg, 2009, pp. 417–434. doi: 10.1007/978-3-642-10366-7_25.

- [Wri] O. Wright. *Decree Fiat Shamir Library*. <https://github.com/trailofbits/decree>. 0.1.0.
- [YZ21] T. Yamakawa and M. Zhandry. “Classical vs Quantum Random Oracles”. In: *Proceedings of the 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT’21. 2021, pp. 568–597. doi: 10.1007/978-3-030-77886-6_20.